

**STEGOLOCK: A CLOUD-BASED WEB APPLICATION BUILT ON A
RECONSTRUCTION-DEPENDENT SECURITY ARCHITECTURE FOR
DIGITAL DOCUMENT STORAGE**



An Information Technology Capstone Project
Presented to the Faculty of the College of Information and Computing
University of Southeastern Philippines
Bo. Obrero, Davao City

In Partial Fulfillment of the Requirements for the Degree of
BACHELOR OF SCIENCE IN INFORMATION TECHNOLOGY

Corpin, Jinnelyn M.
Icalina, Joshua R.
Pasaporte, John Christian N.
Ranas, Elezabeth R.

Adviser
Reyes, Ariel Roy L., DIT

May 2025

APPROVAL SHEET

The Information Technology Capstone Project, titled **STEGOLOCK: A CLOUD-BASED WEB APPLICATION BUILT ON A RECONSTRUCTION-DEPENDENT SECURITY ARCHITECTURE FOR DIGITAL DOCUMENT STORAGE**, prepared and submitted by **Jinnelyn M. Corpin, Joshua R. Icalina, Elezabeth R. Ranas, and John Christian N. Pasaporte**, in partial fulfillment of the requirements for the degree of Bachelor of Science in Information Technology, has been examined and is hereby recommended for oral examination, acceptance, and approval.

Ariel Roy L. Reyes, DIT
Adviser

Approved by the Committee on Oral Defense

Hermoso J. Tupas Jr., MIT
Member
Date signed: _____

Jamal Kay B. Rogers, PhD
Member
Date signed: _____

Commented [J11]: Changed from MEP-ECE

Accepted in partial fulfillment of the requirements for the degree of Bachelor of Science in Information Technology.

Cristina E. Dum Dumaya, Ph.D.
Dean

DECLARATION OF ORIGINALITY

We, **Jinnelyn M. Corpin, Elezabeth R. Ranas, Joshua R. Icalina, and John Christian N. Pasaporte**, candidates for the degree of **Bachelor of Science in Information Technology** at the University of Southeastern Philippines, hereby declare that this capstone, titled

"STEGOLOCK: A CLOUD-BASED WEB APPLICATION BUILT ON A RECONSTRUCTION-DEPENDENT SECURITY ARCHITECTURE FOR DIGITAL DOCUMENT STORAGE"

is our original work. This capstone has been prepared and written by us, and except where otherwise indicated or acknowledged in the references, it does not contain the work of any other person, nor has it been submitted in any form to any other academic institution or for any other degree, diploma, or qualification.

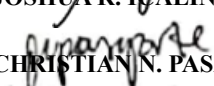
We further affirm that proper citation and acknowledgement have been made wherever the work of others has been used, in accordance with the academic and ethical standards of the University.

In making this declaration, we understand that any violation of this declaration will be subject to the University's policies on academic integrity and research ethics.


JINNELYN M. CORPIN


ELEZABETH R. RANAS


JOSHUA R. ICALINA


JOHN CHRISTIAN N. PASAPORTE

ACKNOWLEDGEMENT

This study stands as a testament of a journey defined by persistence and the resolve to see a difficult project through to its completion. We, the researchers, wish to express our sincere gratitude to those who supported us throughout this process.

To our adviser, **Dr. Ariel Roy L. Reyes**, we extend our deepest appreciation. We are profoundly grateful that you opened your doors to us when your schedule was already full. You met our many revisions and consultations with consistent patience and dedication, never making us feel like a burden regardless of how many rounds of refinements it took. Thank you for your guidance and for seeing the potential in our work.

We also express our heartfelt gratitude to the **University of Southeastern Philippines** for providing the supportive learning environment and the valuable resources that made this study possible. Specifically, we thank the faculty of the **College of Information and Computing** for the professional and academic guidance we have received since we first stepped into the university. Your collective expertise and the high standards of your instruction have been instrumental in our growth, providing us with the technical foundation and inspiration needed to complete this study.

To the research team, thank you for the shared perseverance. This journey tested our resolve and technical capacity, but together, we learned what it takes to see a difficult project through to the end.

Finally, we thank our families and friends for their encouragement and understanding during this challenging journey. Most importantly, we offer our deepest gratitude to **Jesus Christ, Our Lord**, for the wisdom, strength, and grace that sustained us to the very end.

Thank you for helping us get here.

EXECUTIVE SUMMARY

The widespread adoption of cloud storage platforms has revolutionized document management, enabling individuals and organizations to store and share sensitive data with unprecedented ease; however, this centralized accessibility also introduces significant security risks due to the limitations of traditional "key-dependent" security models where a single cryptographic key often serves as the only line of defense. While traditional encryption provides a robust first layer of data protection, it remains vulnerable to adversaries who target the storage backend or key management infrastructure directly, potentially exposing digital assets once the primary defense is bypassed. To address these vulnerabilities, this study introduced and developed StegoLock, a system designed to protect sensitive documents by integrating encryption, file segmentation, and steganography into a unified security framework. The system first secures documents using AES-256-GCM encryption and a KDF-based key management process before employing a segmentation algorithm to divide the ciphertext into independent fragments. These fragments are then concealed within ordinary image, audio, and text files using Least Significant Bit (LSB) steganography and distributed across S3-compatible object storage, ensuring that no single stored object carries the complete encrypted document. Systematically evaluated against ISO/IEC 25010 quality standards through technical audits and a user study with 30 participants, StegoLock achieved an overall General Weighted Mean of 4.53, demonstrating that the application effectively balances technical security requirements with operational simplicity. Notably, the system maintained a 100% data integrity success rate, perfectly reconstructing every document without bit-loss during the retrieval process. By combining these defensive layers, StegoLock offers a technical method for users to maintain absolute control over their sensitive digital documents, providing a robust defense that remains secure even in the event of partial cloud or credential compromise.

Commented [J12]: Inserted "of data protection"

TABLE OF CONTENTS

COVER PAGE	
APPROVAL SHEET.....	ii
DECLARATION OF ORIGINALITY	iii
ACKNOWLEDGEMENT	iv
EXECUTIVE SUMMARY.....	v
TABLE OF CONTENTS.....	vi
LIST OF TABLES	vii
LIST OF FIGURES	viii
LIST OF ABBREVIATIONS	xii
CHAPTER I INTRODUCTION.....	1
1.1 BACKGROUND OF THE STUDY	1
1.2 OBJECTIVES OF THE STUDY	4
1.3 SIGNIFICANCE OF THE STUDY	5
1.4 SCOPE AND LIMITATIONS.....	6
1.5 DEFINITION OF TERMS.....	7
CHAPTER II REVIEW OF LITERATURE.....	10
2.1 RELATED LITERATURE.....	10
2.2 RELATED SYSTEMS	27
CHAPTER III FRAMEWORK AND METHODOLOGY	32
3.1 CONCEPTUAL FRAMEWORK	36
3.2 METHODOLOGY.....	55
3.3 ETHICAL CONSIDERATION.....	59
CHAPTER IV RESULTS AND DISCUSSIONS	61
CHAPTER V SUMMARY, CONCLUSIONS, AND RECOMMENDATIONS	164
REFERENCES	170

LIST OF TABLES

Table	Description	Page No.
1	Operational and Deployment Characteristics.....	32
2	Cryptographic and Security Characteristics.....	32
3	General Testing Environment Setup Summary	118
4	User Participation and Response Summary	119
5	Standard User Profile Responses Summary.....	120
6	Summary of ISO/IEC 25010 Quality Characteristic Ratings	123
7	Summary of Survey Ratings for Reliability.....	127
8	Distribution of Respondents' Internet Access	128
9	Document Locking and Unlocking Success Ratios	129
10	Document Unlock Attempts	130
11	Summary of Survey Ratings for Functional Suitability	135
12	Technical Audit of StegoLock Functional Processes	136
13	Summary of Survey Ratings for Security	139
14	Summary of Survey Ratings for Usability	145
15	Time Interval from User Registration to First Successful Document Locking	148
16	Time Interval from First Document Lock to First Unlock.....	150
17	Summary of Survey Ratings for Performance Efficiency.....	152
18	Average Locking Process Latency by Sub-process.....	154
19	Average Unlocking Process Latency by Sub-process	155
20	Unlocking and Locking Process Duration Distribution Across File Size Ranges	158
21	Stego-Expansion Analysis (Payload vs. Cloud Size).....	159

LIST OF FIGURES

Figure	Description	Page No.
1	StegoLock Conceptual Framework.....	37
2	StegoLock System Architecture.....	38
3	StegoLock Document Locking Process	41
4	StegoLock Document Unlocking Process.....	43
5	StegoLock User Account Creation Process.....	44
6	StegoLock User Login Process	45
7	StegoLock Document Sharing Process	46
8	StegoLock Document Deletion Process.....	47
9	The Reconstruction-Dependent Security Model.....	50
10	ISO/IEC 25010 Quality Characteristics used for Evaluating StegoLock [132]	52
11	Agile Methodology.....	55
12	PBKDF2 on Password Code	63
13	Encryption Key Derivation with PBKDF2 Code.....	64
14	Wrapping Key Derivation Code.....	65
15	Master Key Encryption and Database Storage Code	68
16	Master Key Decryption During User Session Code	69
17	Wrapping Key Encryption Code	71
18	Document File Encryption Code.....	72
19	Document Encryption Key Re-Wrapping Process Code	73
20	Document Encryption Key Re-Wrapping Upon Share Acceptance Code	74
21	Cover Selection Method Code 1	77
22	Cover Selection Method Code 2	78

23	Cover Fetching from Cloud Method Code	79
24	Dynamic Capacity-Based Segmentation Logic Code 1	80
25	Dynamic Capacity-Based Segmentation Logic Code 2	81
26	Dynamic Capacity-Based Segmentation Logic Code 3	82
27	Cover Text Generation Code.....	84
28	Text LSB Embedding Engine Code	86
29	Audio LSB Embedding Engine Code	88
30	Image LSB Embedding Engine Code	90
31	Stego Files Capacity Gate and Parallel Cloud Storage Distribution Code	92
32	StegoLock Home/Welcome Page.....	95
33	User Registration Interface	96
34	User Login Interface	97
35	MyDocuments Page — Main User Dashboard.....	98
36	Application Sidebar Navigation	99
37	Implementation of Secure Password Hashing	101
38	Authenticated Document Routes and Middleware Protection	102
39	Document Upload and Locking Interface	104
40	Segmented Document Storage in Cloud Buckets (Part 1)	105
41	Segmented Document Storage in Cloud Buckets (Part 2)	105
42	Document Retrieval and Reconstruction Interface	106
43	Document Sharing Interface — Owner's Perspective.....	108
44	Role-Based Access: Owner View.....	109
45	Role-Based Access: Shared User View	109
46	Access Denial for Unauthorized User	110
47	Active Document Sharing — Recipient Access Confirmed	111

48	Active Document Sharing — Owner Revoked Access Confirmed.....	111
49	Railway Production Environment — Application and Database Services Online	115
50	Railway Database Browser — stegolock_app MySQL Schema in Production	116
51	Railway Deployment History — Active and Historical Deployments via GitHub CI/CD	117
52	Database Query for Overall Evaluation Mean	121
53	Database Query for Categorical Weighted Means	121
54	Visual Profile of StegoLock Evaluation Scores	125
55	File History and Audit Log showing a Failed Encryption Process	132
56	UI display showing Ghost Files in the Cloud Storage	133
57	Database Management page in Admin Access. System Integrity Audit feature in red box	134
58	Share File view when a user shares a file to recipient	141
59	Accept Shared File in recipient's view	141
60	Accepted Shared File in recipient's view, showing the accessibility on file operations	142
61	Sample document_shares records	143
62	Database Query for Learnability Metric: Time to First Document Lock after Registration	147
63	Database Query for Learnability Metric: Time from First Lock to First Unlock	149
64	Database Query for Average Locking Process Latency by Sub-process	154
65	Database Query for Average Unlocking Process Latency by Sub-process..	155
66	Database Query for Locking Process Distribution Across File Size Ranges	156
67	Database Query for Unlocking Process Distribution Across File Size Ranges	157

68 Database Query for Stego-Expansion Analysis159

LIST OF ABBREVIATIONS

AES-GCM	Advanced Encryption Standard in Galois/Counter Mode
API	Application Programming Interface
DMS	Document Management System
HKDF	HMAC-based Extract-and-Expand Key Derivation Function
IPO	Input- Process-Output
KDF	Key Derivation Function
LSB	Least Significant Bit
PSNR	Peak Signal-to-Noise Ratio
S3	Simple Storage Service
NIST	National Institute of Standard and Technology
SNR	Signal-to-Noise Ratio

CHAPTER I INTRODUCTION

1.1 BACKGROUND OF THE STUDY

Documents have long been used to record information, establish ownership, communicate decisions, and preserve records for future reference. Early documents existed in physical forms such as clay tablets, papyrus, parchment, and paper, which were stored in archives, libraries, and record rooms and shared through manual copying, physical transfer, or controlled institutional access [1][2][3]. While effective at the time, these methods were constrained by limited physical space, susceptibility to damage, slow retrieval, and restricted accessibility, highlighting the need for more efficient ways to manage information. This need led to the process of digitalization, transforming physical records into digital documents, enabling information to be created, stored, and accessed electronically. Digital documents are typically stored in local systems, organizational servers, and cloud-based platforms and are shared through digital networks, such as email and web-based applications [28][29].

Despite improved efficiency and accessibility, digital document storage introduces challenges such as centralized storage risks, unauthorized access, data breaches, and data integrity concerns. Reports indicate that the global average cost of a data breach has reached \$4.4 million, while many organizations still lack proper access controls, leaving them vulnerable to ransomware, social engineering, and third-party attacks [5][6]. The Thales Data Threat Report 2025 further emphasizes that data breaches remain prevalent in increasingly complex digital environments [4]. These challenges underscore the growing need for secure, controlled, and reliable digital document management, which has driven the adoption of cloud-based storage systems offering scalability, accessibility, and support for decision-making [7][8][9].

While cloud-based platforms offer significant benefits, persistent security challenges remain, particularly in terms of system reliability, data confidentiality, and

user authentication [10]. Despite the efficiency of widely used platforms such as Google Drive, recent phishing-related breaches and security warnings highlight ongoing risks [11]. Incidents involving major providers demonstrate that even platforms with advanced protections can be compromised. For example, a Google Drive breach caused by social engineering affected nearly 40% of files containing sensitive information, exposing gaps in access controls and data privacy management [12][13]. Similarly, iCloud experienced large-scale credential exposures through malware-infected devices [30][31], Dropbox suffered a breach of its Dropbox Sign service, exposing user account data [32][33], and OneDrive, in one of its security reports, documented design flaws that could allow unauthorized access [34][35]. Collectively, these cases highlight persistent vulnerabilities in cloud-based document storage and the ongoing need for robust security measures.

In response to the risks inherent in cloud environments, organizations have adopted various security measures, among which encryption has emerged as a prominent mechanism for protecting digital documents. Hybrid models integrating encryption with access controls are widely implemented to ensure that even if unauthorized access occurs, the confidentiality of stored files remains preserved [14][19]. Other complementary approaches, such as blockchain-based auditing [15][16] and machine learning–driven monitoring [17][18], further strengthen layered defenses. Nevertheless, encryption remains the foundational safeguard in most cloud storage systems [39], operating under the assumption that intercepted or breached files remain unreadable without the corresponding decryption keys. While this assumption enhances confidentiality, it does not entirely eliminate risk.

Despite its strengths, encryption alone does not fully prevent unauthorized recovery of sensitive information. Cybersecurity research has acknowledged that encrypted data, once obtained by adversaries, may still be subjected to unauthorized decryption attempts through reverse engineering, brute-force strategies, or exploitation

of implementation weaknesses [40][41]. In cloud environments, encrypted files may be exfiltrated through credential compromise, insider threats, or system vulnerabilities [42][43], after which decryption efforts can occur outside the protected infrastructure. In such situations, encryption continues to preserve ciphertext form, yet the security boundary shifts from the protected system to the encrypted artifact itself. When ciphertext becomes independently accessible, adversarial efforts may focus on exposing the original content without interacting further with the originating environment [44][45]. This reveals a structural limitation in conventional encryption-based models, where confidentiality depends primarily on cryptographic strength, even when encrypted files are no longer contained within the secured platform.

Beyond encryption, cryptography–steganography (crypto-stego) integration has emerged as a promising approach for secure document storage by simultaneously encrypting sensitive content and concealing its existence [20][21]. By hiding encrypted data within benign cover files, crypto-stego systems add an additional layer of obscurity beyond traditional cryptographic protection. However, these systems face practical challenges. Steganographic techniques perform best with small payloads; however, the embedding of substantial data volumes or high-entropy encrypted payloads can strain the system's capacity and reduce overall efficiency [22][23][24]. Moreover, the increased technical complexity can affect usability, as users may experience difficulties performing secure upload, retrieval, or file management tasks, as observed in recent web-based and application-level platforms [25][26][27]. Hybrid systems also complicate cryptographic key handling. Poor key management has been documented in real-world cases, such as the Codefinger ransomware attack on AWS S3 buckets, API key exposure in Dropbox Sign, and unrotated compromised keys in Microsoft SharePoint, which allowed unauthorized access to encrypted files and sensitive information [36][37][38]. Although crypto-stego systems improve confidentiality and concealment, issues such as embedding large payloads, maintaining user-friendly

operation, and ensuring secure key handling continue to pose obstacles, highlighting the need for innovative security models that address these operational complexities.

To address the limitations identified in current cloud-based document security approaches, this study proposes a cloud-based web application built on a reconstruction-dependent security architecture for digital document storage. This architecture ensures that decryption is structurally dependent on successful reconstruction of distributed encrypted data fragments, while incorporating the integration of cryptography and steganography and implementing lightweight key management to enhance security and mitigate practical challenges related to payload capacity, usability, and key handling.

1.2 OBJECTIVES OF THE STUDY

The primary objective of this study is to develop StegoLock, a cloud-based web application for secure digital document storage that implements a reconstruction-dependent security model integrating encryption, steganography, and enhanced key management mechanisms. The system aims to reduce reliance on traditional key-dependent security by ensuring that decryption is not solely dependent on key possession but also on the successful reconstruction of distributed encrypted data fragments. Through this approach, the application seeks to address vulnerabilities associated with key exposure and unauthorized reconstruction of encrypted data. Additionally, the system will provide an efficient, secure, and user-friendly platform for sharing digital documents, while protecting both the confidentiality and concealment of sensitive information, optimizing steganographic payload capacity, maintaining compatibility with widely used document formats, and ensuring reliable performance in resource-constrained environments.

Commented [J13]: Changed from “steganographic encryption”

Commented [J14]: Changed from “segments”

To be specific, this capstone project aims to:

1. Implement AES-based encryption with a KDF-based key management process to ensure the confidentiality and integrity of a document file.

2. Design and implement a segmentation process that splits the encrypted document into multiple segments and hides them through a steganographic embedding process into cover files, which are scattered across the application's cloud storage to enhance security.
3. Develop a web-based application that implements and integrates the AES-based encryption, segmentation, access control and authentication, and sharing mechanisms to a document storage platform.
4. Evaluate the application based on ISO/IEC 25010 quality characteristics to assess the effectiveness in terms of functional suitability, security, reliability, and measure usability and performance efficiency.

1.3 SIGNIFICANCE OF THE STUDY

The main objective of this study is to develop StegoLock, a cloud-hosted web application for secure digital document storage and sharing that employs a reconstruction-dependent security model. The primary beneficiaries of the project are the following:

1. **Organizations Handling Sensitive Digital Documents** - Businesses, educational institutions, healthcare providers, legal offices, and government agencies that manage confidential records will benefit from the application's reconstruction-dependent security model. The system not only encrypts sensitive documents but also conceals and disperses their segmented data across multiple cover files, reducing the risk of data breaches, interception, and unauthorized access in cloud storage environments. This segmentation also allows the steganographic embedding to handle larger document payloads without compromising security or performance.
2. **End Users and Professionals** - Individual users and professionals who frequently store or share confidential documents online will benefit from a secure yet user-friendly design. By enforcing access control and requiring full

Commented [J15]: Removed:

The system combines AES encryption with KDF-based key management, segmentation of a document's encrypted data into multiple fragments, and concealment of these fragments using multi-technique steganographic embedding across multiple cover files, which are then scattered within the cloud storage environment. Unlike conventional document storage systems, StegoLock requires all encrypted data fragments to be successfully extracted from the cover files and reconstructed before decryption is possible, ensuring that unauthorized access to partial data provides no meaningful information.

Commented [J16]: Changed from "StegoLock's"

reconstruction of all encrypted data fragments, the application protects the confidentiality and integrity of sensitive information during storage and sharing, ensuring that partial access provides no meaningful data.

Commented [J17]: Changed from “StegoLock»

3. **Future Researchers** - This study provides a reference for researchers in information security and cryptography-steganography integration. The reconstruction-dependent model, together with segmentation and lightweight key management, offers insights into designing hybrid security systems that balance robust protection, usability, and practical key management.

Commented [J18]: Removed:

Additionally, segmenting the encrypted data improves the payload capacity for steganographic embedding, allowing large documents to be securely concealed across multiple cover files without exceeding embedding limits.

1.4 SCOPE AND LIMITATIONS

StegoLock is implemented as a cloud-based web application accessible through modern web browsers, allowing users to create accounts, log in, manage, and share documents within a secure environment.

Upon upload, documents are compressed, encrypted using AES in Galois Counter Mode (AES-GCM) with KDF-based key management, and segmented into multiple fragments. These fragments are then concealed via steganographic LSB embedding across a set of cover files. Decryption is permitted only after successful authorization, followed by the complete extraction and reconstruction of all fragments. This segmentation strategy enables the system to handle original files up to 5MB by distributing the encrypted payload across multiple cover media. For clarity, this entire security workflow is defined as '**locking**,' while the retrieval process is termed '**unlocking**.' Additionally, the system provides a hierarchical folder structure to ensure organized and user-friendly document management experience.

Commented [J19]: Changed from “diverse”

Furthermore, StegoLock supports **.pdf, .doc/.docx, and .txt** document formats. The steganographic cover pool is **system-provided** and administrator-managed, encompassing **image (.png), text (.txt), and audio (.wav) formats**. Upon registration, each user is allotted an initial storage quota of **225MB**, which is calculated based on the **cumulative 'in-cloud' size of the resulting stego-files** rather than the original

document size. Due to the overhead inherent in steganographic embedding, the total storage footprint will significantly exceed that of the source documents. To maintain system stability and ensure efficient **resource allocation**, the application is optimized for limited concurrent processing; consequently, larger files or multiple simultaneous operations may result in extended processing durations. Furthermore, access is strictly limited to authenticated users, with a secure access-control mechanism ensuring that shared documents are restricted to designated collaborators. Finally, while the system is built for modern web standards, usage on legacy or unsupported browsers may result in reduced performance or visual inconsistencies.

1.5 DEFINITION OF TERMS

This section provides operational definitions of key terms used in this study, particularly those directly related to the design, implementation, and evaluation of the application. These definitions clarify how specific concepts are applied within the system, ensuring a shared understanding of the terminology throughout the study.

1. Digital Document Storage - a component of a document management system responsible for holding and organizing digital files, ensuring that documents are maintained and accessible to authorized users.
2. Reconstruction-Dependent Security Architecture – refers to a security framework where decryption is structurally impossible unless all distributed encrypted data fragments are successfully retrieved and reassembled.
3. Cryptography-Steganography System – also known as a crypto-stego system, is a hybrid security approach that combines the principles of cryptography and steganography, emphasizing a protection mechanism that encrypts sensitive content and conceals its existence within ordinary-looking cover files.
4. AES-GCM (Advanced Encryption Standard in Galois/Counter Mode) – an authenticated encryption method employed by StegoLock to secure documents, providing confidentiality, data integrity, and protection against tampering.

5. Key Derivation Function (KDF) – a cryptographic algorithm used in StegoLock to generate strong encryption keys from user-provided secrets, supporting lightweight and secure key management.
6. Lightweight Key Management – the system within StegoLock that handles the creation, distribution, and storage of encryption keys efficiently, minimizing complexity while maintaining strong security for users.
7. Steganography – the method used to conceal encrypted document data within cover files, ensuring that the existence of sensitive information is hidden from unauthorized parties.
8. Steganographic Embedding – the operational process of integrating encrypted document fragments into cover files (images, text, or audio), such that the hidden data remains protected from detection.
9. Cover File – the file used to hold embedded encrypted document fragments; it appears ordinary to external observers but contains a hidden payload accessible only by authorized users.
10. Segmentation – the systematic process in StegoLock that divides an encrypted document into smaller pieces or fragments prior to steganographic embedding, ensuring that sensitive information is distributed across multiple cover files.
11. Fragments – the individual pieces of encrypted data generated during the segmentation process; each fragment is stored within a separate cover file and is essential for the reconstruction of the original document.
12. Stego File – the final digital artifact stored in the cloud upon completion of the locking process. It serves as the structural anchor for the unlocking workflow, requiring both its retrieval and valid user authentication credentials to initiate the extraction of the hidden encrypted fragments stored within.
13. Locking – the integrated multi-stage security pipeline in StegoLock encompassing document compression, AES-GCM encryption, segmentation, steganographic embedding, and secure cloud storage.

14. Unlocking – the retrieval and reconstruction process that fetches stego files from the cloud, extracts hidden fragments, reassembles the encrypted payload, and performs authorized decryption and decompression to restore the document to its original state.

CHAPTER II

REVIEW OF RELATED LITERATURE

This chapter presents a review of existing literature and systems relevant to the objectives of this project. The review of related literature examines foundational and recent studies on digital document storage, cloud security, encryption, key derivation, steganography, encrypted file segmentation, multi-location cloud storage, reconstruction-dependent security models, and usability in secure web applications. The review of related systems evaluates existing platforms and security architectures based on their technical capabilities, encompassing cloud-based document storage applications, crypto-steganographic security systems, and web-based steganographic and secure document applications. Together, these reviews establish the theoretical and technical context for the proposed system's conceptual framework and provide a comprehensive comparative analysis that contextualizes its architectural requirements.

2.1 RELATED LITERATURE

2.1.1 EVOLUTION OF DIGITAL DOCUMENT STORAGE

2.1.1.1 FROM PHYSICAL ARCHIVES TO DIGITAL DOCUMENT MANAGEMENT SYSTEMS

The transition from physical to digital document storage marks a major development in modern information management. Traditional paper-based systems were limited by storage capacity, slow retrieval processes, and vulnerability to physical damage. To address these limitations, organizations adopted Digital Document Management Systems (DMS) that enable electronic storage, structured organization, and efficient retrieval of documents through indexing and search mechanisms [28][46][47].

Studies on digital document management highlight several advantages over traditional archives. Digital systems improve document accessibility, support remote collaboration, and reduce operational costs

associated with physical storage and manual document handling. They also enable faster information retrieval through metadata tagging and keyword indexing, allowing organizations to manage large volumes of information more efficiently [47][48]. In addition, automated workflows integrated into digital archives further enhance document processing, classification, and retrieval efficiency compared with manual record-keeping systems [46][47].

2.1.1.2 CLOUD-BASED DOCUMENT STORAGE

The evolution of digital storage was further accelerated by the emergence of cloud computing, which significantly transformed how documents are stored, accessed, and shared. According to the National Institute of Standards and Technology (NIST), cloud computing is defined as a model that enables ubiquitous, on-demand network access to a shared pool of configurable computing resources such as servers, storage, applications, and services that can be rapidly provisioned with minimal management effort [49].

Cloud storage platforms provide several advantages over traditional on-premises systems. Research highlights scalability, remote accessibility, and collaborative capabilities as primary factors driving cloud adoption [49][133][134]. Organizations can dynamically adjust storage resources according to demand while allowing users to access documents from any internet-connected device. Cloud repositories also enable centralized storage and real-time collaboration, allowing distributed teams to share and co-edit documents more efficiently [51][52].

2.1.2 CLOUD-BASED STORAGE AND SECURITY CHALLENGES

Despite its benefits, cloud computing introduces significant security challenges. Storing data in third-party infrastructures reduces direct organizational control over physical storage resources and expands the potential attack surface. Research identifies several risks affecting cloud-based document storage, including misconfigurations, unauthorized access, and vulnerabilities associated with shared infrastructure and multi-tenant environments [53]. These security concerns highlight the need for stronger data protection mechanisms when adopting cloud-based document storage systems, particularly in environments where sensitive or confidential information is involved [97][121].

Cloud storage services such as Google Drive, iCloud, Dropbox, and Microsoft OneDrive are widely used due to their accessibility and integrated security features, including encryption, authentication, and access control mechanisms [97]. These platforms enable users to store, synchronize, and share documents conveniently across devices and locations. However, security evaluations and academic studies reveal persistent vulnerabilities that may expose stored data to potential threats [97][122].

Research conducted by Sultan Majid Alshamsi highlights that cloud storage providers may remain vulnerable to OAuth misconfigurations, API exposures, and weaknesses in authentication flows, indicating that data stored in these platforms can still be at risk despite standard security measures [54]. Additional security analyses emphasize that improper encryption implementation, delayed system patching, inadequate access control policies, and insufficient monitoring mechanisms may lead to unintended data exposure or security breaches [55].

Furthermore, studies on man-in-the-cloud attack techniques demonstrate that compromised authentication tokens can allow attackers to gain unauthorized

access to cloud accounts without stealing traditional login credentials. Forensic investigations also reveal the presence of residual artifacts on local devices, challenging assumptions regarding complete data isolation in cloud storage environments. User-trust surveys similarly indicate that concerns regarding privacy and unauthorized access remain prevalent even when cloud services provide built-in encryption and security controls [56][57].

Traditional cloud security frameworks largely rely on encryption and perimeter protection. However, once encrypted files are exfiltrated from the storage environment, the protection shifts from system boundaries to the encrypted data itself, exposing limitations in conventional security strategies. Studies on cloud storage security emphasize that encryption alone cannot guarantee comprehensive protection, particularly when infrastructure configurations, access controls, or key management practices are inadequate. As a result, researchers highlight the need for additional security mechanisms to complement encryption to better protect sensitive data stored in cloud environments [58].

2.1.3 ENCRYPTION AS A FOUNDATIONAL SECURITY MECHANISM

2.1.3.1 ADVANCED ENCRYPTION STANDARD (AES) FOR DATA PROTECTION

Encryption remains a fundamental mechanism for protecting digital information, particularly in cloud-based environments where sensitive data is transmitted and stored across distributed infrastructures. Among modern encryption standards, the Advanced Encryption Standard is widely recognized as one of the most secure and extensively used symmetric encryption algorithms in contemporary cybersecurity. Standardized by the National Institute of Standards and Technology (NIST), AES operates as a symmetric block cipher capable of encrypting and decrypting data using key

sizes of 128, 192, or 256 bits [123]. This design enables AES to secure sensitive information across a wide range of applications [123][124].

Encryption protects confidentiality by transforming plaintext into ciphertext that cannot be interpreted without the correct cryptographic key. In cloud environments, encryption may be applied either at the client side or the server side. Client-side encryption, where data is encrypted before transmission to the cloud, is often considered more secure because it reduces exposure to external threats and potential service-provider vulnerabilities. AES replaced the earlier Data Encryption Standard due to its stronger key lengths and improved resistance to brute-force and cryptanalytic attacks. Because of its security strength and computational efficiency, AES is commonly recommended for protecting both data-at-rest and data-in-transit in cloud storage systems [59][60].

2.1.3.2 LIMITATIONS OF ENCRYPTION AND THE IMPORTANCE OF KEY MANAGEMENT

Despite its effectiveness, encryption alone cannot eliminate all security risks. Encrypted systems may still be vulnerable to attacks such as side-channel attacks, cryptographic implementation flaws, and brute-force attempts. Security analyses show that even mathematically strong encryption algorithms can be compromised through physical information leakage, including variations in power consumption, processing time, or electromagnetic emissions during cryptographic operations [61]. These findings highlight that the overall security of encrypted systems depends not only on the algorithm itself but also on how it is implemented and deployed.

Furthermore, the effectiveness of encryption is highly dependent on proper cryptographic key management. Research emphasizes that weak key

management practices can undermine the theoretical security of encryption algorithms such as AES. If encryption keys are exposed, improperly stored, or poorly generated, the protection provided by encryption can be completely negated. Consequently, secure key management practices that include key generation, distribution, storage, and rotation are essential for maintaining the confidentiality and integrity of protected data [62][63].

2.1.4 KEY DERIVATION FUNCTIONS AND KEY MANAGEMENT

2.1.4.1 KEY DERIVATION FUNCTIONS FOR SECURE KEY GENERATION

In encryption-based systems, effective key management is essential for maintaining overall security. One widely used mechanism for strengthening cryptographic keys is the Key Derivation Function (KDF), which derives secure keys from passwords or shared secrets. Commonly adopted KDFs include PBKDF2 and HKDF, both of which enhance password-based encryption by incorporating salting mechanisms and configurable computational costs [125][126]. These features significantly increase resistance to brute-force and dictionary attacks [125].

Recent research highlights the evolving role of KDFs in modern security architectures. Abdullah proposed an adaptive model of PBKDF2 that dynamically adjusts iteration counts according to contextual factors such as password strength and device capability. By increasing computational cost, when necessary, the model improves resilience against offline brute-force attacks while maintaining system performance. Similarly, formal cryptographic analysis by Hoang and colleagues demonstrates that HKDF produces secure pseudorandom outputs suitable for cryptographic key generation when the initial keying material or salt contains sufficient randomness [64][65]. The theoretical basis of HKDF

originates from the work of Hugo Krawczyk, who introduced the extract-then-expand paradigm, explaining how entropy can be securely extracted from weak input material and expanded into strong cryptographic keys. This framework has since been widely adopted in modern cryptographic standards [66].

Practical implementations further support these findings. For example, the Linux Unified Key Setup utilizes PBKDF2 with salting and configurable iteration counts to strengthen disk encryption. Increasing the number of iterations significantly raises the computational cost of brute-force and dictionary attacks while maintaining usability for legitimate users [67].

2.1.4.2 IMPORTANCE OF CRYPTOGRAPHIC KEY MANAGEMENT

Despite the strength of modern encryption algorithms, improper key management remains a major source of security vulnerabilities. Studies indicate that weak key storage practices, inadequate access control mechanisms, and infrequent key rotation can compromise encrypted systems regardless of the algorithm used. As a result, effective key management practices including secure key generation, distribution, storage, and periodic rotation are critical components of secure document management systems [68].

Integrating KDF-based key generation with controlled authorization mechanisms can further enhance system security. By adjusting computational workload based on contextual factors such as device capability and data sensitivity, adaptive KDF implementations strengthen derived keys when higher protection is required while minimizing overhead in lower-risk scenarios. This approach helps balance security and usability

while improving resistance to password-guessing and brute-force attacks [64][65].

2.1.5 STEGANOGRAPHY IN DIGITAL SECURITY

Steganography refers to the practice of concealing information within a non-secret file or medium to obscure the very existence of communication. Unlike encryption, which protects the content of data, steganography hides the presence of the message itself. Recent digital steganography studies demonstrate that various media types — including text, audio, and images — can serve as cover objects for embedding hidden information [69][70]. One commonly used technique is the modification of least significant bits (LSBs) in digital data, such as pixel values in images or sample values in audio files. Through altering these LSBs, secret bits can be embedded in a way that remains imperceptible to human users while still encoding concealed information [70].

To further strengthen security, crypto-steganography systems integrate encryption with steganographic techniques [127]. In this layered approach, data is first encrypted and then embedded within a cover medium [128]. This dual-protection mechanism enhances both confidentiality and concealment: even if the hidden data is detected or extracted, it remains encrypted and unreadable without the appropriate cryptographic key. Such integration provides an additional security layer beyond traditional encryption or steganography alone, reinforcing protection against interception and unauthorized access [127][129].

Despite its advantages, steganographic systems are subject to several technical limitations. Embedding large, encrypted payloads can degrade the quality of the cover media and reduce overall system efficiency. Commonly cited challenges include limited payload capacity and increase in computational overhead, particularly when advanced embedding techniques are employed. A 2025 review published in *Scientific Reports* analyzes modern steganographic

approaches and identifies constraints such as low embedding capacity and increased computational complexity, especially in deep-learning-based methods [71]. Although these advanced techniques may enhance security and robustness, they often introduce heavier processing requirements and difficulties in balancing payload size, imperceptibility, and resistance to detection.

Additional peer-reviewed studies further confirm that increasing payload size tends to negatively affect the perceptual quality of steganographic media, reinforcing a fundamental trade-off in many embedding strategies [71]. These findings highlight that steganographic capacity is inherently constrained by the size of the cover medium and the need to preserve visual or auditory quality. Consequently, achieving an optimal balance between security, capacity, efficiency, and imperceptibility remains a central challenge in steganographic system design.

2.1.6 ENCRYPTED FILE SEGMENTATION AND MULTI-LOCATION CLOUD STORAGE

2.1.6.1 ENCRYPTED FILE SEGMENTATION AS A SECURITY MECHANISM

In secure document storage systems, segmentation refers to the deliberate splitting of an already-encrypted file into multiple separate parts referred to here as segments prior to storage. This approach is distinct from encryption itself: while encryption protects the *content* of data by rendering it unreadable without a key, segmentation addresses a different threat ensuring that no single stored file contains enough information to be useful to an attacker, even if the file is obtained. A file-splitting approach for cloud data security distributes encrypted file segments across multiple cloud service providers, significantly reducing the likelihood of unauthorized access, as each individual segment in isolation contains insufficient information for reconstruction [50].

This principle is formalized in cryptographic literature through the concept of secret sharing — also called secret splitting — wherein a secret is divided such that no individual piece reveals any intelligible information on its own. Secret sharing distributes a secret among a group such that no individual holds any intelligible information about the secret, but when enough individuals combine their shares, the secret may be reconstructed. Secure secret sharing is effectively "all or nothing" a single share is entirely useless without the others. Waseem et al. (2025) demonstrate that splitting a file into encrypted segments dispersed across multiple cloud providers precludes unauthorized access by a single provider, as authenticated access and reassembly of all segments are required for secure recovery meaning no individual segment yields recoverable content on its own [98].

As validated in a study [74], threshold secret sharing provides mathematical certainty that no individual share reveals any information about the original secret on its own, meaning that an attacker who obtains a portion of segments gains no meaningful data unless they acquire all segments required to meet the reconstruction threshold. This structural guarantee is what distinguishes segmentation as a security mechanism from conventional single-file encrypted storage. Supporting this, practical implementations of secret sharing in cloud storage systems demonstrate that cryptographic keys can be divided into shares distributed across separate cloud locations, such that each location holds only one share improving the security of sensitive data because no single cloud provider can access the complete secret [97].

2.1.6.2 MULTI-LOCATION STORAGE OF ENCRYPTED SEGMENTS

Once an encrypted file is split into segments, distributing those segments across separate, independent cloud storage locations introduces

an additional structural security layer [50][79]. This is not the same as distributed computing infrastructure — the concept here is straightforward: each encrypted segment is stored as a separate file in a distinct cloud location, so that no single location holds the complete encrypted file [130]. The security benefit is architectural in the sense that even if one storage location is compromised, the attacker retrieves only one segment, which carries no reconstructible content without the other remaining pieces [50][130].

Frameworks that split data into chunks and encrypt each portion separately demonstrate that this approach strengthens user-side data security — combining the protection of encryption with the structural protection of splitting, ensuring that the compromise of any single chunk does not expose the full original data [100]. This is directly aligned with the architecture of the proposed system, wherein each encrypted segment is embedded within a steganographic carrier file and stored at a separate cloud location meaning an adversary must successfully retrieve all carrier files, extract all embedded segments, and reassemble them before any decryption attempt can occur.

The security value of this multi-location approach is further supported by Tang and Li [79], who propose a multi-layer secure cloud storage model that includes secure data partitioning and dispersed storage across several cloud platforms, improving privacy-preserving capabilities in data-sensitive environments. Similarly, Sudha and Ramesh [78] establish that integrating file splitting, encryption, and access control mechanisms adds a meaningful security layer with insignificant system overhead which the security and confidentiality benefits far outweigh the computational costs introduced. Collectively, these works confirm that splitting an

encrypted file across multiple independent storage locations reduces reliance solely on cryptographic strength and incorporates structural protection mechanisms, a principle central to the reconstruction-dependent security model of the proposed system.

2.1.7 RECONSTRUCTION-DEPENDENT SECURITY MODELS

2.1.7.1 ASSEMBLY-DEPENDENT DECRYPTION AS A CORE SECURITY PRINCIPLE

Reconstruction-dependent security extends beyond conventional encryption by requiring the structural assembly of segmented encrypted components as a mandatory precondition for decryption. According to the National Institute of Standards and Technology (NIST), in attribute-based encryption systems, decryption is only possible when the access policy conditions built into the ciphertext structure are fully satisfied — meaning isolated ciphertext components alone are insufficient for decryption without the correct structural conditions being met [80]. This demonstrates that modern cryptographic architectures can enforce assembly-dependent decryption as a core security mechanism, where access to ciphertext alone confers no meaningful advantage to an adversary.

This principle is corroborated by Abusukhon et al. [81], whose study published in *Scientific Reports* confirms that even when an attacker obtains encrypted data, decryption remains impossible unless all required structural components satisfy the access policy — establishing that possession of ciphertext alone does not constitute meaningful access to protected information. The mathematical formalization of this requirement is provided by the NIST Multi-Party Threshold Cryptography Project (2020), which establishes that a cryptographic operation such as decryption can only be completed when enough parties or segments — exceeding a defined

threshold — contribute their shares collectively. Critically, any group smaller than the threshold cannot determine the secret even through collaboration, and reconstruction must occur before decryption proceeds [78]. Tong et. al., [83], in their study on adaptively secure threshold unsignryption, further confirm that threshold cryptography divides decryption responsibilities such that a minimum required number of parties must cooperate to achieve successful decryption — improving both security and resilience against adversarial scenarios.

2.1.7.2 THE THREE-BARRIER SECURITY ARCHITECTURE

The reconstruction-dependent model introduces three sequential and interdependent security barriers that collectively govern access to protected data [73][78].

The first is the Segmentation Barrier, in which the encrypted file is split into multiple separate segments prior to storage. No individual segment contains sufficient data to reconstruct the original file [50][98]. Waseem et al. demonstrate that splitting a file into encrypted segments and dispersing them across multiple storage locations precludes unauthorized access by any single party, as all segments must be retrieved and reassembled for secure recovery — meaning no individual segment yields recoverable content on its own [98].

The second is the Distribution Barrier, in which each segment is embedded into a separate steganographic carrier file and stored at a distinct cloud location. As established in a study on multi-layered protection, embedding encrypted data into image carriers distributed across separate storage locations substantially reduces the risk of data loss, manipulation, and theft because no single storage location holds the complete dataset, and

the concealment layer makes the presence of hidden data imperceptible to external observers [84].

The third is the Reconstruction Barrier, which enforces that all distributed carrier files must be successfully retrieved and their segments reassembled before any decryption operation can be initiated. As demonstrated in study on two-phase embedding approach, the cryptosteganography approach enforces mandatory retrieval and reassembly as a precondition of decryption. This means that an attacker who obtains only one carrier file cannot reconstruct or decrypt the original data without acquiring all remaining segments and proper authorization credentials [135].

2.1.7.3 COMBINED ARCHITECTURE AND DEFENSE-IN-DEPTH PRINCIPLES

Through combining cryptography, steganography, and file segmentation, this reconstruction-dependent architecture mitigates the risks associated with ciphertext exfiltration. As demonstrated in a study, systems that segment and encrypt data while concealing decryption keys within steganographic carriers ensure that even if ciphertext is exfiltrated, it remains inaccessible without the corresponding carrier file containing the embedded key. This binding of ciphertext and key carrier across separate, distributed files is precisely what renders single-file exfiltration attacks ineffective against this architecture [97].

This layered model reflects the well-established principle of defense-in-depth, wherein multiple independent security controls are stacked such that the failure of any one layer does not result in total system compromise. Defense-in-depth is a multi-layered cybersecurity approach wherein each independent control layer functions separately which ensures

that if one defense is breached, the remaining measures continue to contain and mitigate the threat [87]. Fitzgerald and Morris, as cited in a study [88], further confirm that combining multiple security layers improves threat detection, response, and overall system resilience against adversarial conditions. This is reinforced by Godakanda [84], whose work on defense-in-depth modeling confirms that architectures incorporating multiple independent barrier layers significantly reduce exploitable attack surfaces [85]. The segmentation–distribution–reconstruction architecture of the proposed system directly embodies these principles: each barrier operates independently, and all three must be overcome sequentially for an adversary to access the protected document.

2.1.8 USABILITY CONSIDERATIONS IN SECURE WEB APPLICATIONS

2.1.8.1 THE SECURITY-USABILITY TRADEOFF

Security and usability are frequently positioned as competing objectives in the design of secure digital systems. More secure systems tend to be less usable, while more usable systems often sacrifice security — a tension that complicates the design of systems that must satisfy both requirements simultaneously [91]. This tradeoff is corroborated by a systematic review of usable security research spanning nearly two decades, which confirms that this inverse relationship between security strength and usability is a consistent and well-documented finding across the field [92].

The practical consequences of this tension are significant. Braun et al. demonstrate that overly complex security workflows such as those requiring users to manage cryptographic credentials manually that frequently result in risky compensatory behaviors, including password reuse and insecure credential storage [93]. This means that a system

designed with strong cryptographic protections can paradoxically become less secure in practice if its interface places excessive cognitive demands on users. Ma et al. [94] reinforce this by establishing that effective hybrid encryption architectures must be designed for practical deployability alongside cryptographic robustness just simplifying workflows and ensuring cross-platform compatibility without exposing sensitive data to insecure handling.

2.1.8.2 USABILITY EVALUATION AND LIGHTWEIGHT DESIGN PRINCIPLES

Given the security-usability tradeoff, formal evaluation of user experience is an essential component of secure system development. The System Usability Scale (SUS) is among the most widely adopted instruments for this purpose. Deshmukh and Chalmers [72] confirm that SUS can be applied across all types of interactive digital platforms including web applications, mobile systems, and expert tools — making it a suitable evaluation instrument for security-integrated systems where user experience directly influences adoption and compliance behavior.

On the design side, Mamun et al. [90] demonstrate that lightweight key management structures simplify key lifecycle processes — including allocation, distribution, and rotation — while simultaneously reducing the computational complexity of encryption and decryption operations, resulting in faster processing and a more accessible user experience. This is complemented by Krähenbühl and Perrig [91], who affirm that simplifying key management for end users is not merely a usability concern but a security one — poorly managed keys represent one of the most persistent vulnerabilities in cryptographic systems regardless of algorithm strength.

Collectively, these findings establish that usability must be treated as an equally integral dimension of secure web application design. In the context of the proposed system, where the security architecture involves multiple sequential operations including encryption, steganographic embedding, segmentation, and cloud storage ensuring that these processes are streamlined and transparent to the user is critical to both practical adoption and the integrity of the security model itself.

2.1.9 SYNTHESIS OF REVIEWED LITERATURE

The literature reviewed across the preceding sections does not present isolated topics, rather it traces a logical progression in which each mechanism addresses a specific limitation left unresolved by the one before it.

Encryption establishes the baseline: it renders data unreadable without a key. However, as the reviewed literature consistently demonstrates, encryption's protection collapses the moment its key is compromised through brute-force attacks, side-channel leakage, or poor key management practices. Key Derivation Functions respond to this weakness not by changing the encryption model but by hardening the process through which keys are generated and strengthened. Both mechanisms, however, operate entirely within the same layer: they protect data *content* but leave the existence and location of that data fully exposed.

On the other hand, Steganography operates on a fundamentally different dimension. Rather than making data unreadable, it makes data *undetectable*. This is a qualitative shift — an attacker who does not know protected data exists has no target to pursue. When combined with encryption as crypto-steganography, both the content and the presence of data are simultaneously protected. Yet even this combination shares a critical structural vulnerability: the entire protected payload resides in a single carrier file. Obtaining that one file is sufficient to attempt decryption or steganalysis.

Encrypted file segmentation directly addresses this single-file vulnerability by ensuring that no individual stored piece contains sufficient information for reconstruction. This introduces a structural guarantee that neither encryption, key hardening, nor steganography can provide on their own. Multi-location storage then extends this guarantee spatially, no single storage location holds a complete dataset, meaning compromise of any one location yields nothing actionable. Together, segmentation and multi-location storage shift security from a content-based model to an architecture-based one.

Reconstruction-dependent security is where these layers converge into a unified access model. It formalizes what the preceding mechanisms collectively imply: that decryption is not merely key-protected but architecturally prevented until every segment is retrieved, reassembled, and authorized. This transforms independent controls into a sequential pipeline which an adversary must defeat every layer, to gain access.

Usability sits across all of these layers as a transversal concern. Unlike the other mechanisms, it does not add a security barrier instead, it determines whether the barriers that exist are used correctly. A system whose security depends on proper user behavior, such as careful key handling or multi-step retrieval, is only as strong as its interface allows users to be. This is why usability is not peripheral to the proposed system's design but integral to it: the system's multi-layer architecture is only effective if users can operate it consistently and correctly.

2.2 RELATED SYSTEMS

This section reviews existing platforms and security models relevant to the structural and operational objectives of the proposed system. Selected systems are examined based on their implementation of key technical capabilities, including cryptographic data protection, steganographic concealment, distributed cloud storage architectures, reconstruction-dependent access mechanisms, and web-based usability.

The reviewed systems are categorized into three primary domains: cloud-based document storage applications, crypto-steganographic security systems, and web-based steganographic and secure document applications. A comprehensive comparative analysis is presented at the conclusion of this section to evaluate these systems across diverse operational and security metrics, providing a foundational contrast that contextualizes the architectural requirements of the proposed system.

2.2.1 CLOUD-BASED DOCUMENT STORAGE APPLICATIONS

Google Drive, Dropbox, and Microsoft OneDrive are among the most widely used cloud storage platforms globally. These three platforms consistently rank among the top cloud storage services by active user base and enterprise adoption, making them the primary reference points for evaluating cloud document storage security [101]. Their relevance to StegoLock lies in their shared purpose which is storing and retrieving digital documents through a web-based interface which is the same core function that StegoLock is designed to serve.

2.2.1.1 GOOGLE DRIVE

Google Drive is a cloud-based file storage and synchronization service developed by Alphabet Inc. It provides users with web-based access to stored documents, supporting real-time collaboration and cross-device synchronization. The platform secures stored data using AES-256 encryption at rest and TLS encryption in transit, managed entirely by Google's server-side infrastructure. Access is controlled through Google account credentials and optional multi-factor authentication [100].

2.2.1.2 DROPBOX

Dropbox is a cloud file hosting service that allows users to store, access, and share documents through a web and desktop interface. It applies AES-256 encryption for data at rest and SSL/TLS for data in transit, with

encryption managed by Dropbox's servers. The platform supports OAuth-based third-party integrations and account-level access controls. A 2022 security incident involving Dropbox demonstrated that a successful phishing attack against an employee can bypass multi-factor authentication entirely, resulting in unauthorized access to internal repositories illustrating the limitations of authentication-layer-only protection [98].

2.2.1.3 MICROSOFT ONEDRIVE

Microsoft OneDrive is a cloud storage service integrated into the Microsoft 365 ecosystem, offering document storage, synchronization, and sharing capabilities through a web interface. It employs AES-256 encryption at rest and TLS in transit, with encryption managed by Microsoft's infrastructure. Access is governed through Microsoft account credentials, role-based permissions, and OAuth-based application integrations [103]. A documented OAuth scope vulnerability in 2025 exposed a design limitation wherein third-party applications were granted broader data access than explicitly authorized by users [102].

2.2.2 CRYPTO-STEGANOGRAPHIC SECURITY SYSTEMS

The following systems combine encryption and steganography as complementary security mechanisms. Their relevance to StegoLock lies in their shared use of cryptographic and steganographic techniques for data protection, which constitutes the foundational security approach of StegoLock.

2.2.2.1 4-STEP CRYPTO-STEGO CLOUD MODEL

This system, published in *MDPI Sensors*, presents a four-step data security model for cloud computing that sequentially applies RSA encryption, AES encryption, identity-based encryption, and LSB steganographic embedding into image carriers. Encrypted data is stored in

the cloud with the stego-image serving as the access artifact. The system demonstrates that layering cryptographic and steganographic techniques provides stronger protection than encryption alone, particularly against unauthorized access during storage [95].

2.2.2.2 CLOUD CRYPTO-SYSTEM SYSTEM

Proposed in the *Journal La Multiapp*, this system conceals encrypted data and its corresponding decryption key within steganographic image carriers stored in cloud environments. A mandatory key retrieval step from the stego-image is required before decryption can proceed, introducing a partial form of reconstruction dependency [101]. The system operates as a standalone implementation without a dedicated web interface for end users.

2.2.2.3 CryptSteg

Published by *IEEE*, CryptSteg integrates ciphertext policy-attribute-based encryption (CP-ABE) with multi-cover image steganography. Ciphertext payloads are distributed across multiple steganographic cover images, and decryption is conditioned on satisfying the embedded access policy implementing both multi-location distribution and a policy-enforced reconstruction requirement [99]. The system's use of CP-ABE, however, introduces significant computational overhead and does not include a simplified web deployment or formal usability evaluation.

2.2.2.4 AES STEGANOGRAPHY + BLOCKCHAIN

Published in *Informatica*, this system combines AES encryption with LSB steganography and integrates blockchain technology to provide immutable audit trails and tamper detection for encrypted image carriers stored in the cloud [105]. The blockchain layer adds verifiability to the

storage process. However, the system does not implement file segmentation across multiple stego-carriers, does not enforce reconstruction-dependent decryption, and the blockchain dependency adds infrastructure overhead that limits lightweight web deployment.

2.2.3 WEB-BASED STEGANOGRAPHIC AND SECURE DOCUMENT APPLICATIONS

The following systems are relevant to StegoLock in terms of their deployment context could either as web-based applications or as systems designed specifically for securing digital documents. Their relevance establishes that steganographic techniques can be effectively deployed in web environments and applied to document-format files.

2.2.3.1 STEGANOGRAPHY IN WEB APPLICATIONS

Published in the *Journal of Information Systems Engineering and Management*, this system implements SVM-based pixel priority LSB embedding within a browser-accessible web application, achieving a PSNR of 87 dB [100]. It demonstrates that steganographic embedding can be successfully deployed through a web interface while maintaining imperceptibility. The system does not integrate encryption alongside steganography and does not implement file segmentation or cloud-based multi-location storage. Usability is partially assessed through task-based evaluation metrics.

2.2.3.2 PDF CRYPTO-STEGANOGRAPHY SYSTEM

Published in *Springer's IoT-Based Control Networks and Intelligent Systems*, this system embeds AES-256 and RSA-encrypted data within PDF documents without visually altering the document's content [104]. It demonstrates the feasibility of applying crypto-steganographic techniques directly to document-format files — the same file type that StegoLock

targets as its primary storage artifact. The system operates as a desktop application, does not distribute stego-files across multiple cloud locations, and does not enforce reconstruction-dependent decryption.

Table 1. Operational and Deployment Characteristics

SYSTEM	CAT1	CAT2	CAT3
ES1	Cloud document storage and synchronization	Account credentials, optional MFA	Yes
ES2	Cloud file hosting and sharing	Account credentials, MFA, OAuth	Yes
ES3	Integrated cloud storage (Microsoft 365)	Account credentials, role-based permissions	Yes
ES4	Multi-layer secure data storage	Stego-image acts as an access artifact	No
ES5	Concealing encrypted data and keys	Key retrieval from stego-image	No
ES6	Distributed secure storage	Embedded access policy	No
ES7	Tamper-evident secure cloud storage	Blockchain verifiability	No
ES8	Web-based steganographic embedding	Standard web access	Yes
ES9	Secure document embedding	Cryptographic keys	No

Table 2. Cryptographic and Security Characteristics

SYSTEM	CAT4	CAT5	CAT5	CAT7
ES1	AES-256, TLS	None	Reliable infrastructure, multi-location storage	Vulnerable to credential theft and authentication bypass
ES2	AES-256, SSL/TLS	None	Strong encryption, robust third-party integrations	Susceptible to phishing bypassing MFA
ES3	AES-256, TLS	None	Ecosystem integration, role-based permissions	OAuth scope vulnerabilities exposing broad data access
ES4	RSA, AES, Identity-based	LSB in image carriers	Layered cryptographic and steganographic protection	Single stego-carrier vulnerability, computational overhead

ES5	Unspecified	Image carriers	Mandatory key retrieval from carrier	Lack of web interface, single carrier vulnerability
ES6	CP-ABE	Multi-cover image steganography	Multi-location distribution, policy-enforced reconstruction	High computational overhead, lacks lightweight deployment
ES7	AES	LSB in images	Immutable audit trails and tamper detection	Infrastructure overhead, lacks file segmentation
ES8	None	SVM-based pixel priority LSB	High imperceptibility, accessible via browser	Lacks encryption, segmentation, and multi-location storage
ES9	AES-256, RSA	PDF document embedding	Direct application to document-format files	Desktop only, lacks cloud distribution and reconstruction

The existing systems reviewed in this section present varying objectives and capabilities, each addressing distinct security and operational requirements. To facilitate a structured comparison, these platforms are designated as Existing Systems (ES) 1 through 9. Their capabilities are evaluated across two primary domains, represented in Table 1 and Table 2.

Domain 1: Operational and Deployment Characteristics

- **CAT1 (Primary Function):** The core purpose and operational objective of the system.
- **CAT2 (Access Control):** The mechanisms governing user authentication and data access.
- **CAT3 (Web Interface):** The availability of a browser-based deployment model for end-user accessibility.

Domain 2: Cryptographic and Security Characteristics

- **CAT4 (Cryptography Used):** The specific encryption algorithms and cryptographic standards employed.
- **CAT5 (Steganography Method):** The techniques used to conceal data within a carrier medium.
- **CAT6 (Security Strengths):** The primary defensive advantages and architectural benefits.
- **CAT7 (Security Weaknesses):** The identified vulnerabilities, structural limitations, or operational risks.

Cloud-based document storage platforms, including ES1 (Google Drive), ES2 (Dropbox), and ES3 (Microsoft OneDrive), share a primary function of storing, synchronizing, and sharing digital documents. These systems excel in their deployment context, providing robust web interfaces, strong provider-managed AES encryption, and scalable multi-location cloud infrastructures. Their primary strength lies in their accessibility and collaborative capabilities. However, a comparative evaluation highlights a shared architectural limitation where their security models are fundamentally reliant on perimeter authentication. Once access credentials are compromised, stored data becomes fully accessible since there is no structural barrier or steganographic concealment to protect the data independently of the platform's access controls.

In contrast, crypto-steganographic systems prioritize data obfuscation and layered protection over seamless collaboration. Systems like ES4 (4-Step Crypto-Stego Cloud Model), ES5 (Cloud Crypto-System System), and ES7 (AES Steganography + Blockchain) successfully demonstrate that integrating encryption with steganographic embedding provides a stronger defense against unauthorized data extraction. ES4 and ES5 leverage image carriers to hide encrypted data, while ES7 introduces blockchain for tamper detection. However, these systems generally suffer from single-carrier

vulnerabilities, meaning the entirety of the payload resides in one stego-file. If an adversary acquires this single file, they possess all the necessary data to attempt decryption. ES6 (CryptSteg) addresses this specific weakness by distributing ciphertext across multiple covers and enforcing policy-based reconstruction, but its reliance on heavy attribute-based encryption introduces significant computational overhead that hinders lightweight deployment. Furthermore, none of ES4, ES5, ES6, or ES7 provide a web interface, limiting their practical usability for end users.

Finally, systems focused on deployment flexibility demonstrate the feasibility of bringing advanced steganography to more accessible environments. ES8 (Steganography in Web Applications) successfully implements LSB embedding through a browser interface with high imperceptibility, though it entirely omits encryption and segmentation, rendering it unsuited for high-security applications. Meanwhile, ES9 (PDF Crypto-Steganography System) proves that crypto-steganography can be effectively applied directly to document-format files. Nevertheless, ES9 remains restricted to a desktop environment, lacking both the multi-location distribution of cloud platforms and the accessibility of web applications.

By comparing these systems, it becomes evident that while individual platforms successfully achieve specific objectives—such as the usability of ES1, ES2, and ES3, the layered concealment of ES4, ES5, and ES7, or the deployment innovations of ES8 and ES9—none offer a comprehensive integration of these capabilities. This analysis emphasizes the need for an architecture that combines the accessibility of web-based cloud storage with the robust protection of segmented, reconstruction-dependent crypto-steganography.

CHAPTER III FRAMEWORK AND METHODOLOGY

This chapter outlines the conceptual framework and methodology for the development of StegoLock, a cloud-based web application built on a reconstruction-dependent security model (Figure 10) for digital document storage. The core of this research is the Reconstruction-Dependent Security Model. This model changes the traditional approach to security by moving beyond simply blocking access to a system. Instead, it ensures that data remains private because the information cannot be identified or put back together without the correct digital credentials.

This chapter also explains the design and logic of the system using the Input-Process-Output (IPO) method. This approach shows how sensitive user information is changed into a highly secure and invisible state. These principles guided the researchers in creating an application that follows modern security standards.

3.1 CONCEPTUAL FRAMEWORK

The conceptual framework of StegoLock establishes the theoretical and operational boundaries of the research by combining the system's logical data transformations with its comprehensive evaluation metrics. The logical flow of data, illustrated in the StegoLock Conceptual Framework (Figure 1), specifically maps the two core processes of the system: the Document Protection Process and the Document Retrieval Process. Furthermore, the overarching conceptual framework also encompasses the Application System Evaluation Framework, ensuring the system meets established software quality standards. The components of this framework are discussed in the following sections.

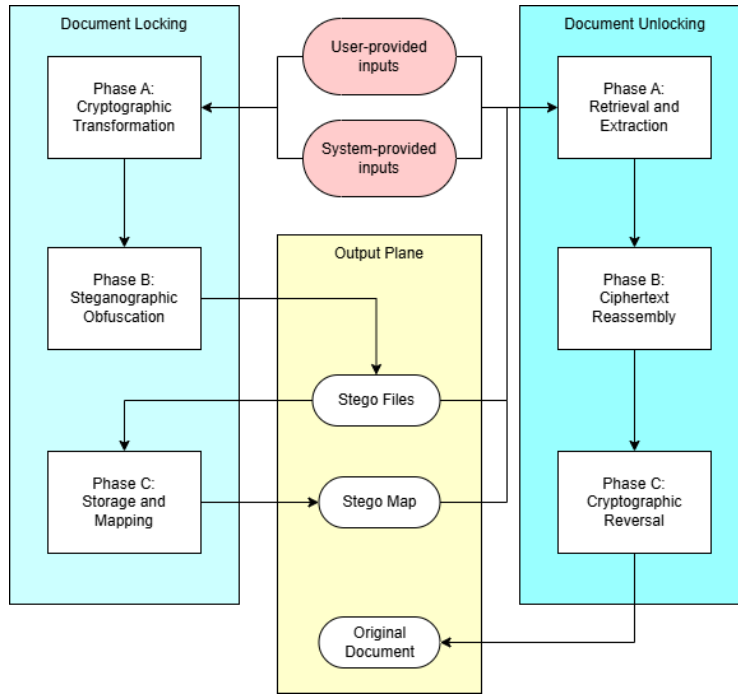


Figure 1. StegoLock Conceptual Framework

To operationalize the principles established in the conceptual framework, the system relies on the StegoLock System Architecture (Figure 2), which uses a four-layer design to separate user interaction, application logic, local database storage, and remote cloud storage. This layered approach enforces the reconstruction-dependent security model by completely isolating the database (holding the Stego Map) from the cloud storage (holding the Stego Files). These two critical layers never interact directly; instead, they rely on the System Controller to act as a secure bridge, ensuring that the three components of reconstruction remain separated to maintain data confidentiality.

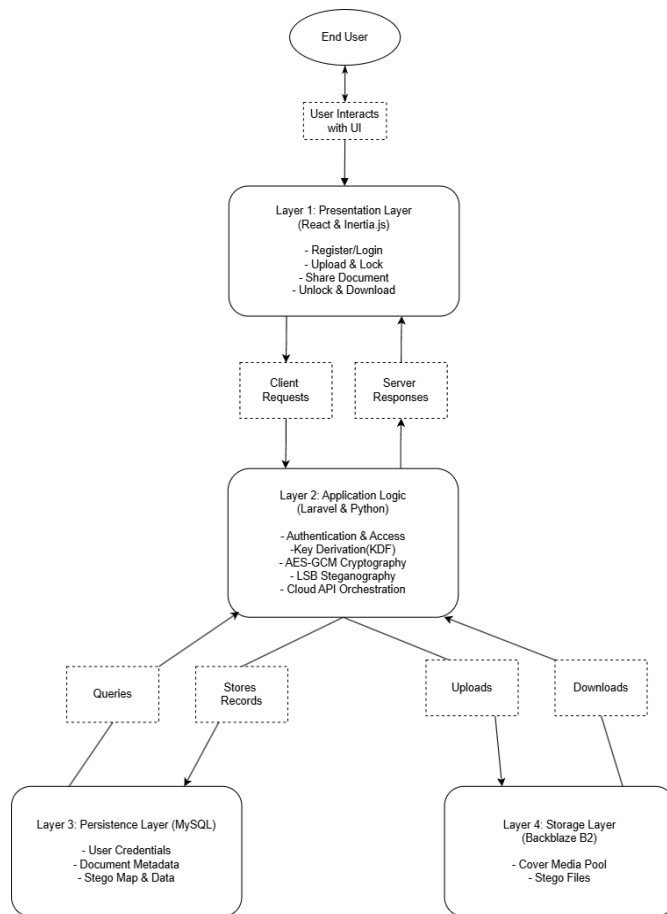


Figure 2. StegoLock System Architecture

The first layer is the Presentation Layer, pertaining to the User Interface. This client-facing layer is built using React and driven by Inertia.js. This layer serves as the user's visual portal to the system, abstracting away the underlying cryptographic

complexity. Its core operation provides the interface for user registration and login, document uploading and its subsequent locking, document sharing, and the final unlocking and downloading of secured documents. Furthermore, regarding its security boundaries, no raw cryptographic processing or data segmentation occurs in the client's browser. All sensitive operations are delegated securely to the backend (Layer 2) to prevent client-side tampering.

The second layer is the Application Logic Layer, acting as the System Controller. This is the core operational layer, serving as the system's central nervous system. It is composed of two distinct subsystems that control the flow of data. Within this layer, the Laravel (PHP) subsystem acts as the primary controller. It manages user authentication, performs secure key derivation to generate the cryptographic keys, executes the AES-256-GCM cryptography and data segmentation, and handles the Cloud API orchestration for uploading and fetching files. Working in tandem, the Python-based steganographic system is invoked dynamically by Laravel. This specialized subsystem handles the specific computational workloads for executing the multi-format steganographic embedding and extraction processes.

The third layer is the Persistence Layer, which functions as the local database. Powered by MySQL, this internal layer is dedicated to structural metadata and identity management. Its primary role is to house user credentials, general document metadata, data regarding the cover media pool, and crucially, the fragment and stego file mapping metadata, which serves as the structural blueprint required for reassembly. To maintain the Reconstruction-Dependent Security Model, this database layer enforces a strict isolation constraint. It never stores the actual encrypted payload or the document encryption keys, ensuring that a database compromise yields only reassembly instructions without the actual data to reassemble.

The fourth layer is the Storage Layer, utilizing cloud infrastructure for secure file hosting. Powered by Backblaze B2, this external layer is dedicated to scalable

storage. It acts as the repository for the clean cover media pool and strictly houses the stego files, which are the ordinary-looking media files that secretly contain the encrypted document fragments. Similar to the Persistence Layer, the Storage Layer operates under a strict isolation constraint. The cloud bucket possesses no knowledge of the stego map, user identities, or how individual fragments relate to one another. It merely stores flat, isolated binary objects, relying entirely on the System Controller to act as the secure bridge.

Finally, to validate the effectiveness of these interconnected layers and processes, the conceptual framework incorporates an evaluation approach based on the ISO/IEC 25010 software product quality model. This ensures that the resulting application is rigorously assessed for functional suitability, security, reliability, performance efficiency, and usability.

To further understand how the StegoLock framework transforms data across these layers, the system's operations are broken down into logical subprocesses. These are detailed in the following sections through the structural data flows of the locking and unlocking sequences.

3.1.1 STEGOLOCK DOCUMENT PROTECTION PROCESS

The StegoLock Document Protection Process defines the forward transformation of data from a vulnerable plaintext document into a secured, reconstruction-dependent state. While the high-level inputs, processes, and outputs are outlined in the overarching conceptual framework, Figure 3 details the actual data flow and layer-by-layer execution of this locking sequence.

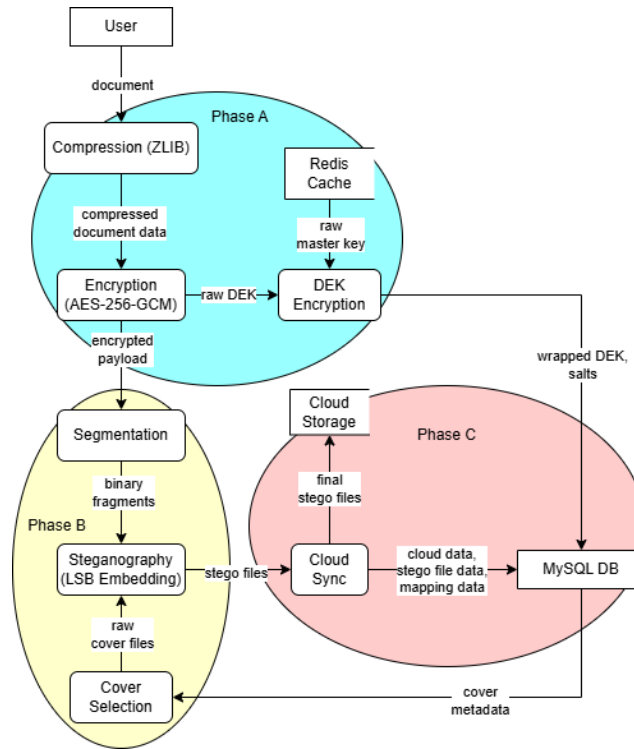


Figure 3. StegoLock Document Locking Process

To execute this protection mechanism, the sequence begins when an authenticated user uploads a primary payload (the original sensitive document). Because the user is already logged in, the system utilizes the active session's master key to authorize the operation. The data immediately enters Phase A (Cryptographic Transformation), which begins with the compression of the document to minimize its footprint. Following this, a unique Document Encryption Key (DEK) is generated to encrypt the payload via AES-256-GCM, and that DEK is then securely wrapped using the master key.

Following this, the system enters Phase B (Steganographic Obfuscation), where the steganographic engine takes over to segment the resulting ciphertext. These encrypted fragments are then embedded into a diverse pool of cover media (such as PNG, WAV, and TXT files) provided by the system.

Finally, the process concludes with Phase C (Storage and Mapping). As the physical obfuscation finishes, the system simultaneously constructs the Stego Map (Component 2), a structural blueprint detailing the fragment IDs and their reassembly order, which is saved locally to the database. The system then scatters the resulting Stego Files (Component 3) into a remote cloud bucket. This final step completely separates the generated components by keeping the blueprint in the local database while the hidden data resides in the cloud. This process ensures strict physical isolation in accordance with the security model.

3.1.2 STEGOLOCK DOCUMENT RETRIEVAL PROCESS

While the locking sequence focuses on segmentation and obfuscation, the StegoLock Document Retrieval Process operates entirely around the principle of data reconstruction. Building upon the logical structure outlined in the conceptual framework, Figure 4 demonstrates the actual data flow and corresponding unlocking sequence executed across the system's architecture. Rather than generating new materials, this process focuses strictly on validation, extraction, and mathematical reversal.

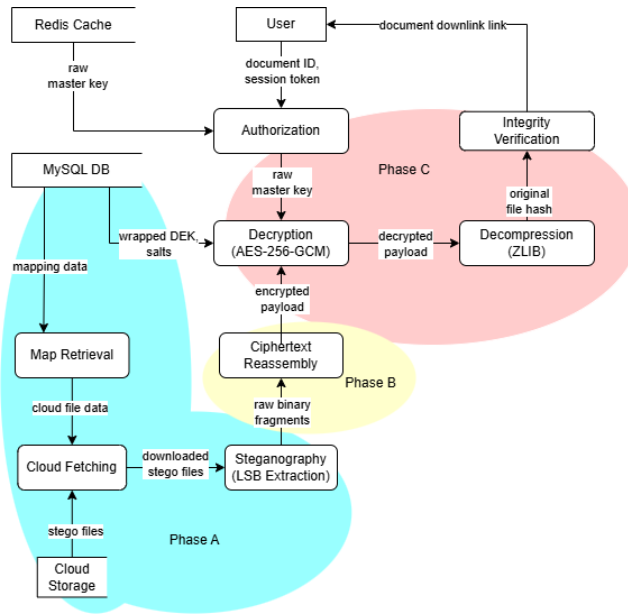


Figure 4. StegoLock Document Unlocking Process

The unlocking sequence is initiated when a user requests to retrieve a secured document. The system first validates the user's authority by confirming the presence of the active session's master key (Component 1). With authorization granted, the system initiates Phase A (Retrieval and Extraction) by querying the local database to retrieve the critical Stego Map, which acts as the blueprint for the entire operation. Using the structural coordinates found within this map, the system connects to the cloud to fetch the specific Stego Files that contain the hidden data, and extracts the hidden fragments from the cover media.

With all components successfully aggregated and extracted, the system moves to Phase B (Ciphertext Reassembly). Guided by the map's indexing data,

it organizes the disorganized pieces back together in their exact original sequence to form a unified ciphertext.

Finally, the system executes Phase C (Cryptographic Reversal) by using the master key to unwrap the internal document key, decrypting the data, and decompressing the payload. Because this entire reconstruction process operates strictly within temporary memory, the bit-perfect original plaintext document is safely delivered directly to the user for immediate download without leaving any readable trace behind.

3.1.3 STEGOLOCK FUNCTIONAL FEATURES

Beyond the core locking and unlocking sequences, the StegoLock system relies on several other functional processes to manage user identities and document access. The following processes illustrate how user interactions cascade across the system's layers to enforce security.

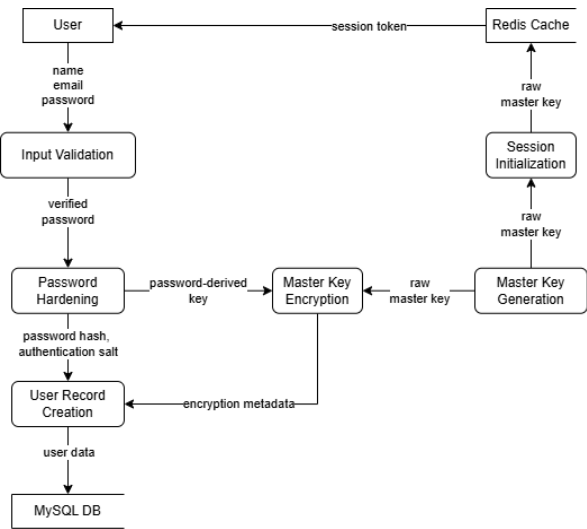


Figure 5. StegoLock User Account Creation Process

Figure 5 maps the initial point of interaction within the StegoLock system, which is the creation of a user account. This process begins at the Presentation Layer, where the user securely submits their registration details without any sensitive key generation occurring in the client's browser. Once submitted, the Application Logic Layer intercepts the request to generate unique cryptographic identities, establishing a distinct master key and random salts for the user. Finally, the Persistence Layer stores these related user data and securely wraps master key and encryption data to the database. By strictly avoiding the storage of plaintext passwords or raw cryptographic keys, the system ensures that any database breach would yield only useless, inaccessible data.

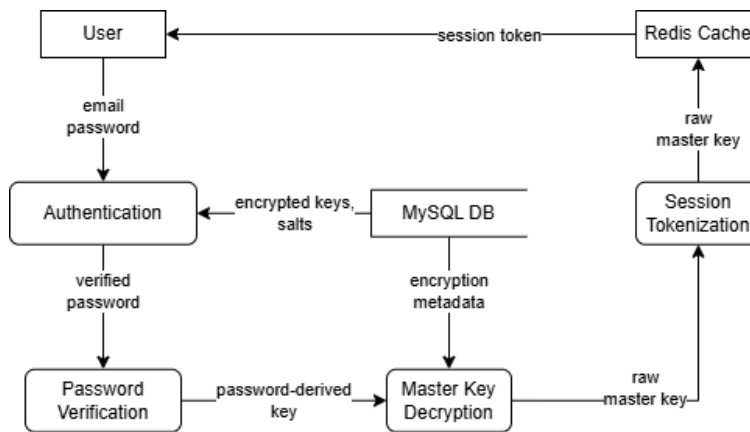


Figure 6. StegoLock User Login Process

The login process, outlined in Figure 6, requires the user to input their credentials at the Presentation Layer to access the system. The Application Logic Layer then verifies these credentials by deriving the necessary keys to unwrap the user's encrypted master key. Upon successful authentication, this master key is temporarily cached in a secure, short-lived session to authorize subsequent cryptographic operations. Throughout this process, the Persistence Layer acts as

the verifier, supplying the stored identity records against which the incoming credentials are cross-referenced.

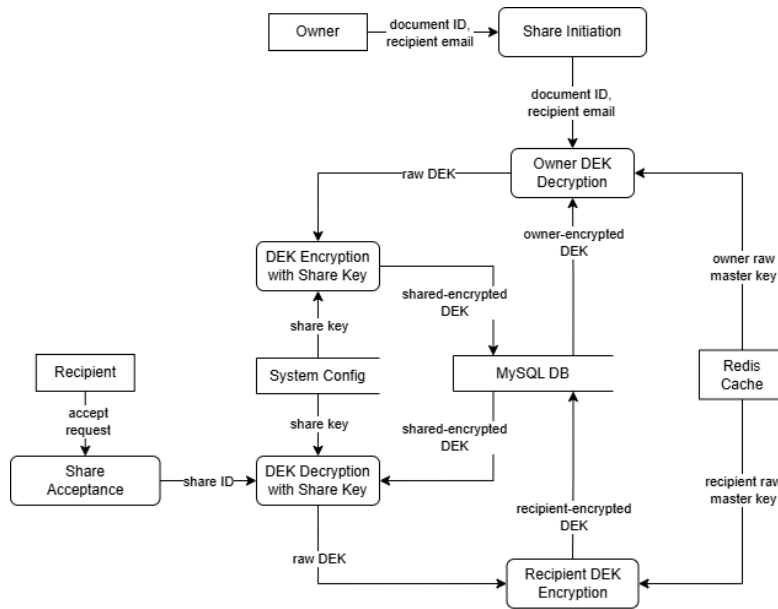


Figure 7. StegoLock Document Sharing Process

As outlined in Figure 7, users can securely share documents by simply selecting a recipient in the Presentation Layer. To keep the system efficient and secure, the Application Logic Layer does not create duplicate copies of the heavy Stego Files in the cloud. Instead, it temporarily unwraps the document's hidden key using the owner's master key and securely re-wraps it specifically for the recipient. The Persistence Layer then records this shared access in the database. This allows the recipient to use their own master key to unlock the document, without altering the original files stored in the cloud.

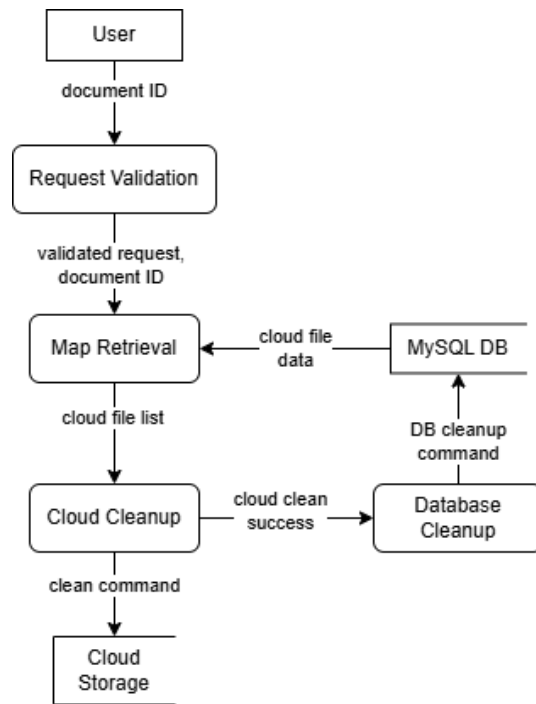


Figure 8. StegoLock Document Deletion Process

Finally, Figure 8 illustrates how a document is permanently deleted from the system. When a user chooses to delete a file at the Presentation Layer, the Application Logic Layer makes sure no hidden data is left behind by following a strict "cloud-first" rule. It connects to the Storage Layer to completely delete the Stego Files from the cloud. Only after confirming that these cloud files are truly gone will the Persistence Layer delete the Stego Map and the document's records from the database. Deleting the files in this specific order guarantees that all data is completely wiped out without leaving any broken or leftover pieces behind.

To sum it all up, the following list highlights how the complex background processes discussed previously power the actual features users interact with inside the StegoLock application:

- **User Registration:** Securely creating an account and generating the user's unique digital keys behind the scenes.
- **User Authentication:** Verifying the user's identity at login so they can safely lock, unlock, and share their files.
- **Upload (Document Locking):** Taking a normal file, encrypting it, breaking it into pieces, and hiding those pieces in the cloud.
- **Download (Document Unlocking):** Finding the hidden pieces in the cloud, putting them back together, and decrypting them to restore the original file.
- **Document Sharing:** Giving a designated recipient a secure key to open a file without having to make a second heavy copy in the cloud.
- **Document Deletion:** Using a "cloud-first" approach to completely and permanently wipe out a file's pieces from the cloud before deleting its records from the database, ensuring absolutely nothing is left behind.

3.1.4 THE RECONSTRUCTION-DEPENDENT SECURITY MODEL

While the functional data flows illustrate how StegoLock handles data, these operations are governed by the system's underlying conceptual foundation. The fundamental principle of StegoLock is security through segmentation and obfuscation. Rather than storing a document as a single file, the system breaks the encrypted data into small fragments and hides them inside everyday media (images, audio, and text). These Stego Files are subsequently scattered into a remote cloud bucket.

As illustrated in Figure 9, to reconstruct the original document, the system relies on three "pillars" that must be accessed simultaneously:

- **The Master Key:** This is a digital key derived from the user's password. It serves as the unique credential required to unlock the document's internal encryption key once the scattered data fragments have been reassembled.
- **The Stego-Map:** This serves as a digital blueprint stored within the system's database. It maintains a precise record of which hidden data fragments belong to a specific document and the exact order required for reassembly.
- **The Stego Files:** These are the physical cover media (such as images, audio, or text files) stored in the cloud. To an unauthorized party, these appear as ordinary media files, yet they secretly contain encrypted data fragments.

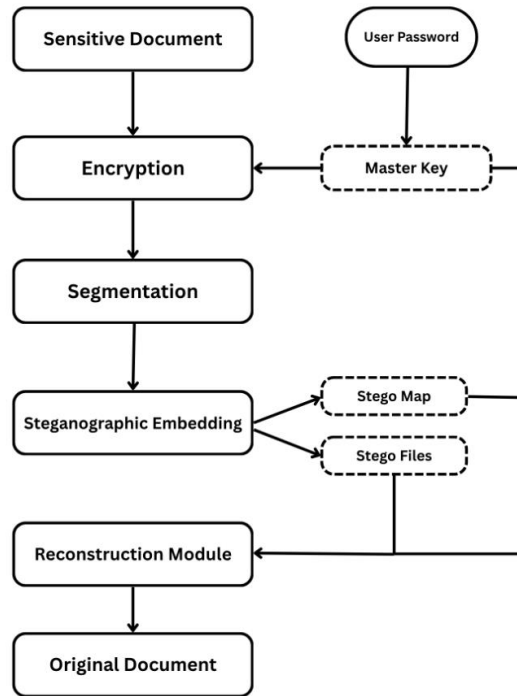


Figure 9. The Reconstruction-Dependent Security Model

System Security: These three pillars are functionally interdependent. If an unauthorized party accesses the files without the map, the correct data cannot be identified. If the map is accessed without the master key, the data remains encrypted and unreadable. Successful reconstruction requires the simultaneous presence of all three elements. This model ensures that an attacker cannot find or rebuild the original data unless they have simultaneous access to the encryption keys, the map of the hidden pieces, and the encrypted data fragments stored in the cloud. This model serves as the underlying conceptual model upon which the entire StegoLock framework is built.

3.1.5 APPLICATION SYSTEM EVALUATION FRAMEWORK

While the preceding sections define the theoretical design and operational flow of StegoLock, the final component of the conceptual framework establishes how the system's actual quality is measured. StegoLock was evaluated using the ISO/IEC 25010 software product quality model, assessing five core characteristics: functional suitability, performance efficiency, usability, reliability, and security. The evaluation was conducted by deploying the system to a live hosting environment and inviting 30 respondents to participate. The sample size was strategically capped at 30 individuals to accommodate the data thresholds of the system's cloud storage infrastructure, ensuring each participant was allocated a stable 225MB testing quota without exceeding the environment's storage limits. Drawn from the study's identified target beneficiaries, the participants consisted exclusively of college students. Participants were provided with a structured set of instructions to explore and interact with the application's core features. Following this guided interaction, respondents evaluated their experience by completing a survey questionnaire utilizing a 5-point Likert scale (ranging from 1 = Strongly Disagree to 5 = Strongly Agree). This survey-based methodology ensured that the assessment of the software's overall quality was grounded in practical usage and direct end-user experience. Also, the survey questionnaire used in the evaluation was adapted from a validated material used by Lusiani and Princes on their application system evaluation in Indonesia, which was also based on ISO 25010 quality model [131].

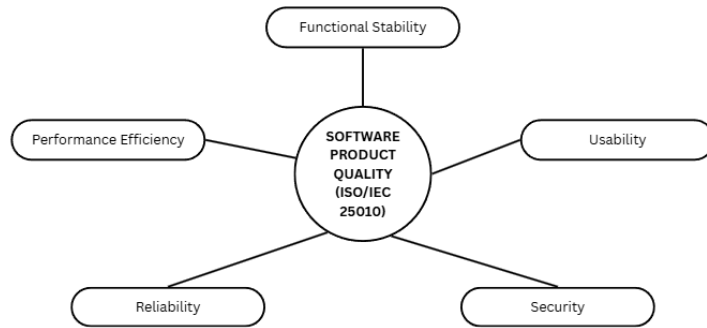


Figure 10. ISO/IEC 25010 Quality Characteristics used for Evaluating StegoLock [132]

Functional Suitability

Functional suitability was evaluated to determine whether StegoLock provides the necessary functions to meet the stated and implied needs of its users under specified conditions. The evaluation covered three sub-characteristics defined under ISO/IEC 25010: functional completeness, which gauged whether users found all necessary features to be present; functional correctness, which measured whether the system accurately produced expected outputs during user interaction; and functional appropriateness, which assessed whether the implemented functions were well-suited to the tasks they were designed to accomplish. Respondents evaluated observable core operations, such as document uploading, document retrieval, and verifying the availability of their locked files within the application. Although the backend steganographic embedding and direct cloud storage operations are invisible at the presentation layer, their functional success was implicitly validated when users confirmed their files were successfully processed and made available.

Security

Given the nature of StegoLock as a security-oriented application, this characteristic was treated with particular rigor. Security was assessed across five sub-characteristics: confidentiality, which examined whether user data and document contents remained inaccessible to unauthorized parties; integrity, which verified that data could not be modified or corrupted without detection; non-repudiation, which assessed whether system actions could be traced and confirmed; accountability, which evaluated the extent to which user actions were logged and traceable; and authenticity, which determined whether the system reliably verified the identities of its users before granting access. Because the underlying cryptographic and steganographic processes operate in the backend, security was assessed based on the respondents' interaction with observable controls—such as user authentication mechanisms—and their overall perceived trust in the system's ability to protect their documents.

Reliability

Reliability was evaluated to determine the degree to which StegoLock performs its intended functions under specified conditions over a defined period of time. The assessment addressed three sub-characteristics: maturity, which gauged how often the system experienced failures under normal operating conditions; availability, which measured the proportion of time the system was operational and accessible to users; and fault tolerance, which examined the system's capacity to maintain acceptable service levels even when encountering unexpected errors. This evaluation focused on whether the application remained stable, accessible, and error-free throughout the users' sessions, providing a practical measure of how consistently StegoLock operates in real-world scenarios.

Performance Efficiency

Performance efficiency was evaluated to assess how well StegoLock performs relative to the amount of resources used under stated conditions. The assessment focused on three specific sub-characteristics: time behavior, which evaluated the system's responsiveness, processing speeds, and the absence of noticeable latency during user actions; resource utilization, which gauged whether users encountered any performance bottlenecks or unexpected application closures; and capacity, which assessed the system's overall compatibility and its ability to operate smoothly across the respondents' various devices. Ultimately, this evaluation captured the users' direct perception of how swiftly and reliably the application handled document operations, ensuring that the system's backend resource management translated into a seamless frontend experience.

Usability

Usability was evaluated to determine the degree to which StegoLock allows users to achieve their goals with effectiveness, efficiency, and satisfaction. The assessment covered five key sub-characteristics: appropriateness recognizability, which assessed whether users could easily recognize and remember the application's core functions; learnability, which measured the overall ease of use for first-time users; operability, which evaluated how intuitively respondents could navigate and control the system's features; user interface aesthetics, which gauged the visual attractiveness, organization, and user-friendliness of the presentation layer; and accessibility, which determined the ease and reliability with which the system could be accessed across various scenarios. By assessing these practical elements, the evaluation ensured that the system's interface and workflow were validated directly against the expectations and overall satisfaction of the end-users.

3.2 METHODOLOGY

3.2.1 DEVELOPMENT METHODOLOGY



Figure 11. Agile Methodology

The creation of StegoLock utilized an Agile approach (**Error! Reference source not found.**) structured around brief iterative sprints, allowing the team to adapt to new insights and continuously enhance the system. Through implementing features in cycles and performing ongoing testing, the team upheld high standards of quality and security throughout the entire project [115], [116]. This methodology was especially appropriate for the complexity of StegoLock, owing to the necessity of integrating various interrelated components such as AES-based encryption, KDF-based key management, steganographic embedding, encrypted file segmentation, access control and authentication, document sharing mechanisms, cloud storage, and a web-based interface — into a single cohesive and evaluated system.

Sprint Structure:

The team planned to organize work into two-week sprints, with each cycle covering requirements gathering, implementation, and testing of a specific set of features. At the start of each sprint, the team selected user stories from the backlog, estimated the effort required, and agreed on what would be delivered by the end of the cycle. However, due to time constraints and the increasing complexity of integrating multiple security-critical components — such as implementing the KDF-based key management process, steganographic embedding pipeline, and cloud storage mechanisms — sprint durations were adjusted as needed to ensure that each component was thoroughly developed, tested, and validated before the next layer of the system was built on top of it. Brief discussions were held as needed to keep everyone aligned and surface any issues before they could delay progress.

Feature Development Process:

Every feature underwent a uniform pipeline before being deployed to production. The process commenced with gathering requirements and reviewing the architecture, followed by implementation and a series of tests that included unit, integration, security, and functional accuracy checks, before moving on to code reviews and staging deployments prior to production. Security validation was integrated into this pipeline rather than being added afterward, ensuring that cryptographic operations, key management logic, access control mechanisms, and input validation were confirmed at each stage of development. This approach is directly aligned with the system's aim of applying a reconstruction-dependent security model, where each component — encryption, segmentation, steganographic embedding, and cloud storage — operates correctly both on its own and as part of a cohesive system.

Testing Strategy:

The testing strategy was designed to validate StegoLock across all four objectives of the study. Unit tests verified the correctness of individual components including AES-GCM encryption and decryption, PBKDF2 and HKDF key derivation, document segmentation, steganographic embedding and extraction, and cloud storage operations. Integration tests confirmed that these components functioned correctly when combined — verifying the full pipeline from document upload through encryption, segmentation, embedding, cloud scatter, retrieval, reconstruction, and decryption back to the original file. Furthermore, security testing addressed the reconstruction-dependent security model directly — verifying that incomplete segment sets cannot be reconstructed, that incorrect keys cannot decrypt ciphertext, that stego files yield no readable content without extraction, and that authenticated session expiry correctly invalidates master key access. Usability testing was conducted with representative users to evaluate the accessibility and ease of use of the web interface, measuring usability as a quality characteristic consistent with the ISO/IEC 25010 evaluation framework alongside functional suitability, security, reliability, and performance efficiency.

Continuous Integration/Continuous Deployment (CI/CD):

Changes were deployed to the production environment incrementally as features were completed and verified, allowing the team to validate each component of the system in a real environment before proceeding to the next development cycle. Throughout development, unit and integration tests were conducted regularly to verify the correctness of individual components and the integrity of their interactions across the system. Changes that passed all unit and integration tests were deployed directly to the production environment, ensuring that all implemented features remained functional, secure, and aligned with the system's objectives throughout the development lifecycle. Features that passed all

functional, security, and usability evaluation criteria were promoted to production. This process ensured that the integration of StegoLock's security-critical components remained stable and verifiable at every stage of development.

Security-First Development:

Security was a foundational priority from the beginning of development rather than an afterthought. Code reviews focused specifically on cryptographic correctness, key management logic, access control enforcement, and user input validation. Security was enforced through the system's reconstruction-dependent security model and code implementation, with manual testing conducted to validate critical attack scenarios — including key exposure, unauthorized segment retrieval, stego files interception, and session hijacking — ensuring that security controls were in place at the most critical points in the system. Independent security audits provided additional validation of the implemented controls [120].

Iterative Refinement:

Throughout development, findings from testing and real-world usage were continuously incorporated into the codebase. Identified bugs and performance bottlenecks were resolved within subsequent sprints, while profiling results guided optimization of computationally intensive operations including KDF iteration, AES-GCM encryption of large files, steganographic embedding, and cloud storage interactions. Cryptographic parameters and steganographic methods were reviewed and adjusted as needed throughout development to ensure the system remained aligned with established security standards and best practices. This continuous refinement process ensured that StegoLock remained technically current, efficient, and aligned with its security and usability objectives throughout the development lifecycle [120].

3.3 ETHICAL CONSIDERATION

3.3.1 CORE PURPOSE AND DEVELOPMENT ETHICS

StegoLock was designed to safeguard sensitive documents against unauthorized access, with a clear ethical mandate that it is not intended to conceal illicit activities. Throughout the development and testing phases, the researchers strictly utilized synthetic data and publicly available test files. Because no actual personal or organizational data was processed during system development, the research process remained low-risk and ethically grounded. The application's intended use cases are strictly legitimate, focusing on protecting business records, safeguarding personal privacy, and providing a secure channel for authorized information sharing.

3.3.2 PRIVACY BY DESIGN

StegoLock incorporates a "privacy-by-design" architecture, ensuring that cryptographic keys, document contents, and user credentials are never transmitted or stored in plaintext. The system does not persist user passwords; rather, they are utilized solely for the derivation of cryptographic keys. During active sessions, the Master Key and Document Encryption Keys (DEKs) are generated on-demand, reside exclusively in volatile memory, and are immediately purged upon task completion. This approach eliminates the vulnerability of long-term key storage, significantly reducing the attack surface and mitigating the risk of static key store compromises.

3.3.3 SECURITY CONTROLS AND RESPONSIBLE USE

To facilitate secure information handling and deter misuse, StegoLock enforces strict access controls, limiting system functionality entirely to authenticated and authorized users. Furthermore, the system incorporates activity monitoring for document operations, ensuring user accountability. These tracking measures allow system administrators to observe access patterns and identify

potential anomalies, ensuring the platform is utilized responsibly and in alignment with its core security objectives.

3.3.4 PARTICIPANT PRIVACY AND DATA GOVERNANCE

During the evaluation phase involving 30 respondents, strict ethical guidelines were observed to protect participant privacy. All survey feedback was gathered with informed consent, and responses were anonymized to safeguard the identities of the participants. While StegoLock's underlying architecture aligns with the core principles of standard data protection frameworks (such as data minimization and encryption-in-transit), the study's deployment environment was strictly governed by the researchers. This ensured a safe, responsible, and highly controlled testing environment throughout the evaluation process.

CHAPTER IV RESULTS AND DISCUSSION

This chapter outlines the outcomes derived from the execution and assessment of the proposed system. It provides an in-depth analysis and interpretation of the data gathered to support the study's objectives. The results are presented in a structured and coherent manner, illustrating the system's performance based on the tests and evaluations conducted. The findings are presented in a clear, systematic manner to demonstrate how the system performed during the tests and assessments. The findings emphasize the essential quality attributes for evaluating effectiveness, including functional suitability, security, reliability, and measures of usability and performance efficiency, supported by pertinent data collected during testing. Observations from testing are also examined to identify strengths, limitations, and areas for improvement.

Furthermore, this chapter interprets the significance of the results and explains their implications in the context of the study. Comparisons between expected and actual outcomes can provide deeper insight into the system's behavior. The discussion aims to justify the results and connect them to the research problem, ensuring that all findings contribute meaningfully to the study's overall conclusions.

4.1 IMPLEMENT AES-BASED ENCRYPTION WITH A KDF-BASED KEY MANAGEMENT PROCESS TO ENSURE THE CONFIDENTIALITY AND INTEGRITY OF A DOCUMENT FILE

The security of StegoLock is built on established cryptographic methods designed to ensure data confidentiality and authenticity. This section details the implementation of the system's primary security layer, which leverages the Advanced Encryption Standard in Galois/Counter Mode (AES-GCM) to protect document files against unauthorized access and tampering.

To build a secure foundation, the system uses a key management process based on Key Derivation Functions (KDFs), which are the Password-Based Key Derivation

Function 2 (PBKDF2), and the HMAC-based Extract-and-Expand Key Derivation Function (HKDF). By using PBKDF2 to harden passwords and HKDF to separate keys, StegoLock ensures that user passwords are transformed into strong keys rather than being stored directly. This approach allows the system to generate unique keys for different tasks, keeping documents secure during encryption, decryption, and sharing. The following subsections explain how these algorithms were implemented to protect the document throughout its lifecycle.

4.1.1 IMPLEMENTATION OF KEY DERIVATION FUNCTIONS

Key Derivation Functions (KDFs) are implemented within StegoLock to transform the user's password into strong keys, creating a secure bridge between logging in and protecting data. To achieve this, the system uses two specific types of KDFs: PBKDF2 to harden the initial password, and HKDF to generate unique keys for different tasks.

By making key generation more complex, these functions help prevent brute-force and dictionary attacks. They also ensure that the user's login password is kept separate from the keys used to encrypt files. This section explains how these key derivation functions are configured in the application and their role in keeping every user session secure.

4.1.1.1 IMPLEMENTATION OF PBKDF2

The Password-Based Key Derivation Function 2 (PBKDF2) is implemented in the user registration and login processes to increase the computational cost of brute-force attacks. Rather than deriving a key in a single step, the application applies two distinct rounds of PBKDF2 using **SHA-256** with **100,000 iterations** per round. Also, each round is explicitly configured to output a **32-byte (256-bit) key**.

Figure 12 shows the code for implementing Round 1 of PBKDF2 during user registration. Line 42 is the generation of the authentication salt with a size of 16 bytes. Lines 44-51 is the Laravel `hash_pbkdf2` function call which derives and stores a 256-bit key in the `$password_derivedKey` variable based on the user's password and the authentication salt.

```
42 $auth_salt = random_bytes(Constant::AUTH_SALT_LEN);

44 $password_derivedKey = hash_pbkdf2(
45     'sha256',
46     $request->password,
47     $auth_salt,
48     100000,
49     32,
50     true
51 );
```

Figure 12. PBKDF2 on Password Code

Figure 13 shows the code for implementing Round 2. Line 57 demonstrates the generation of the encryption key salt. Lines 58-65 executes another `hash_pbkdf2` function call to derive the final encryption key using the previously derived key and the new salt. This final key is then used to encrypt the user's Master Key before it is stored in the database.

```
57 $ek_salt = random_bytes(Constant::EK_SALT_LEN);
58 $encryption_key = hash_pbkdf2(
59     'sha256',
60     $password_derivedKey,
61     $ek_salt,
62     100000,
63     32,
64     true
65 );
```

Figure 13. Encryption Key Derivation with PBKDF2 Code

During user login, the system retrieves the user's previously generated salts from the database and repeats this exact same two-round process to rederive the encryption key required to unlock the stored Master Key.

4.1.1.2 IMPLEMENTATION OF HKDF

The other KDF implementation in StegoLock is the HMAC-based Extract-and-Expand Key Derivation Function (HKDF). This key derivation process is specifically designed for protecting document-specific keys, ensuring that even if one document's key is compromised, all other documents remain secure because their keys are uniquely salted.

In early developmental iterations of StegoLock, HKDF was utilized to derive the Document Encryption Key (DEK) directly from the user's Master Key. However, this direct derivation approach created a rigid architectural limitation that prevented the implementation of a secure document-sharing feature, as sharing a document would necessitate exposing the underlying Master Key.

To resolve this, the system's cryptographic architecture was refactored to employ a key wrapping technique. Under the current implementation, the DEK is randomly system-generated, and HKDF is not used to produce the DEK that encrypts the actual file. Instead, HKDF derives a Key Encryption Key (KEK), referred to as the wrapping key. This wrapping key is derived from the user's Master Key, combined with a unique salt called the document salt. Furthermore, it is explicitly configured to output a 32-byte (256-bit) key, directly aligning with the requirement of the system's AES-256-GCM encryption which necessitates a 256-bit key length to operate securely. This derived wrapping key is then used to securely wrap (or encrypt) the system-generated DEK.

```
571 $dk_salt = random_bytes(Constant::DK_SALT_LEN);
572 $wrapping_key = hash_hkdf(
573     'sha256',
574     $masterKey,
575     32,
576     'dek-wrapping-key',
577     $dk_salt
578 );
```

Figure 14. Wrapping Key Derivation Code

Figure 14 shows the code for deriving the wrapping key during document encryption. Line 571 generates a unique cryptographic salt for the document. Lines 572-578 utilize the Laravel `hash_hkdf` function to extract and expand a 256-bit wrapping key from the user's Master Key, the document salt, and the context string `'dek-wrapping-key'`. This derived wrapping key will subsequently be used to encrypt the DEK.

By using HKDF for key wrapping, StegoLock successfully establishes a secure key hierarchy. When a document is shared with another user, the system can safely use their respective Master Keys to unwrap and re-wrap the DEK without ever exposing the underlying encryption key itself.

4.1.2 IMPLEMENTATION OF AES-GCM

StegoLock utilizes the Advanced Encryption Standard in Galois/Counter Mode (AES-GCM) as its primary encryption algorithm. The system configures this standard with a 256-bit key length (AES-256-GCM) to maximize data confidentiality. Additionally, AES-GCM provides authenticated encryption by generating a 16-byte (128-bit) authentication tag and requiring a unique 12-byte (96-bit) nonce for every operation. These parameters ensure that the encrypted data cannot be read by unauthorized users and mathematically prevent ciphertext tampering, such as bit-flipping or injection attacks.

To establish a comprehensive security architecture, this AES-256-GCM configuration is applied across four distinct components of the system. This section details these implementations, beginning with the encryption of the user's Master Key and the subsequent wrapping of the Document Encryption Key (DEK). It then outlines the encryption and decryption processes of the actual document files. Finally, it illustrates the document sharing mechanism, which employs AES-GCM to securely transfer the DEK to authorized recipients without exposing the underlying keys such as the Master Key.

4.1.2.1 MASTER KEY PROTECTION

The Master Key serves as the foundational cryptographic element for each user within the StegoLock system. It is initially generated as a cryptographically secure 256-bit random byte string during the user registration process. Because the Master Key grants access to all of a user's encrypted documents, it must be protected against both database breaches and unauthorized session access.

To achieve this, StegoLock employs AES-256-GCM to encrypt the Master Key before it is ever stored in the database (Lines 71-78 in Figure

15). The encryption key used for this process is derived from the user's password via PBKDF2 (Lines 58-65 in Figure 15) as discussed in the previous section. Consequently, the database only stores the ciphertext of the Master Key the corresponding nonce, and the authentication tag (Lines 87-89 in Figure 15).

```
56 //Encryption Key Derivation
57 $ek_salt = random_bytes(Constant::EK_SALT_LEN);
58 > $encryption_key = hash_pbkdf2(...
65 );
66
67 //Master Key Encryption
68 $nonce = random_bytes(Constant::NONCE_LEN); // Generate a nonce for AES-GCM
69 $tag = ''; // Will hold authentication tag
70
71 $master_key_enc = openssl_encrypt(
72     $master_key,
73     'aes-256-gcm',
74     $encryption_key,
75     OPENSSL_RAW_DATA,
76     $nonce,
77     $tag
78 );
79
80 //Create user in DB
81 $user = new User();
82 $user->name = $request->name;
83 $user->email = $request->email;
84 $user->password_hash = base64_encode($password_derivedKey);
85 $user->auth_salt = base64_encode($auth_salt);
86 $user->ek_salt = base64_encode($ek_salt);
87 $user->master_key_enc = base64_encode($master_key_enc);
88 $user->nonce = base64_encode($nonce);
89 $user->tag = base64_encode($tag);
90 $user->save();
```

Figure 15. Master Key Encryption and Database Storage Code

When a user successfully logs in (Lines 56-59 in Figure 16), the system re-derives the password-based encryption key (Lines 62-69 in Figure 16) to decrypt the Master Key (Lines 56-59 in Figure 16). Once decrypted, the plaintext Master Key is never written to persistent storage. Instead, it is temporarily held in a secure Redis-backed caching layer and

accessed via a short-lived, cryptographically random session token (Lines 84-89 in Figure 16). This tokenized approach ensures the key is readily available for document encryption and decryption operations during the active session.

```
46 // Derive key from input password
47 > $password_derivedKey = hash_pbkdf2(...)
54 );
55
56 // Verify password
57 > if ( hash_equals(base64_encode($password_derivedKey), $user->password_hash) ) { ...
59 }
60
61 // Decrypt master key
62 > $encryption_key = hash_pbkdf2(...)
69 );
70
71 $master_key = openssl_decrypt(
72     base64_decode($user->master_key_enc),
73     'aes-256-gcm',
74     $encryption_key,
75     OPENSSL_RAW_DATA,
76     base64_decode($user->nonce),
77     base64_decode($user->tag)
78 );
79
80 if ($master_key === false) {
81     return back()->withErrors(['email' => 'Failed to decrypt master key']);
82 }
83
84 // Store master key in Redis via TemporaryKeyStorage
85 $storage = new \App\Services\TemporaryKeyStorage();
86 $token = $storage->store($master_key, $user->id);
87
88 // Store token in session instead of master key
89 session(['master_key_token' => $token]);
90 session()->forget(['master_key', 'master_key_expires_at']);
91
92 // Regenerate session first (prevents session fixation)
93 $request->session()->regenerate();
94
95 // Login user manually
96 Auth::login($user);
```

Figure 16. Master Key Decryption During User Session Code

To mitigate the risk of key leakage or compromise, StegoLock enforces strict lifecycle controls. The session token expires automatically after a period of inactivity, at which point the Master Key is purged from the Redis cache (Lines 90 in Figure 16). By restricting the plaintext Master Key to exist only fleetingly in volatile memory, the system's architecture is designed to minimize the window of vulnerability against persistent exposure or advanced memory extraction techniques.

4.1.2.2 DOCUMENT FILE ENCRYPTION AND DECRYPTION

With the Master Key securely established within the active session, the system leverages it to perform the core cryptographic operations of the application. The protection of individual documents relies on a dual-layered encryption approach. Because storing the Document Encryption Key (DEK) in plain text alongside the document would compromise the entire encryption scheme, the DEK itself must be protected before the actual file contents are encrypted (Figure 17 and Figure 18).

```

554 // 1. Read and Compress (ZLIB format)
555 $sourcePath = Storage::path($temp_filePath);
556 $raw_plaintext = file_get_contents($sourcePath);
557
558 // Delete raw upload immediately to free space
559 Storage::delete($temp_filePath);
560
561 $compressed = gzcompress($raw_plaintext, 9);
562 unset($raw_plaintext); // Free memory
563
564 // 2. Generate a random Document Key (DEK)
565 $dek = random_bytes(32);
566 $dek_hash = hash('sha256', $dek);
567
568 // 3. Master key is passed from TemporaryKeyStorage (no session lookup)
569
570 // 4. Generate wrapping metadata
571 $dk_salt = random_bytes(Constant::DK_SALT_LEN);
572 > $wrapping_key = hash_hkdf(...
573 );
574
575 $dek_nonce = random_bytes(Constant::NONCE_LEN);
576 $dek_tag = '';
577
578 // 5. Wrap the DEK
579 $encrypted_dek = openssl_encrypt(
580     $dek,
581     'aes-256-gcm',
582     $wrapping_key,
583     OPENSSL_RAW_DATA,
584     $dek_nonce,
585     $dek_tag
586 );

```

Figure 17. Wrapping Key Encryption Code

To secure the DEK (Line 565 in Figure 17), the system generates a random 256-bit key and immediately encrypts it using AES-256-GCM (Lines 570-590 in Figure 17). The key used to perform this wrapping operation is derived from the user's Master Key via HKDF as discussed in the previous sections. Once wrapped, the encrypted DEK is safely stored in the database alongside the document's metadata (Lines 611-619 in Figure 18).

```

593 // 6. AES-256-GCM encryption
594 $file_nonce = random_bytes(Constant::NONCE_LEN);
595 $file_tag = '';
596 $ciphertext = openssl_encrypt(
597     $compressed,
598     'aes-256-gcm',
599     $dek,
600     OPENSSL_RAW_DATA,
601     $file_nonce,
602     $file_tag
603 );
604 unset($compressed); // Free memory
605
606 // 7. Save encrypted file (store file_nonce + file_tag + ciphertext)
607 $encPath = 'temp/encrypted/' . Str::uuid() . '.stegolock';
608 Storage::put($encPath, $file_nonce . $file_tag . $ciphertext);
609
610 // 8. Update the database with encryption and wrapping info
611 $document->update([
612     'dk_salt' => base64_encode($dk_salt),
613     'encrypted_dek' => base64_encode($encrypted_dek),
614     'dek_nonce' => base64_encode($dek_nonce),
615     'dek_tag' => base64_encode($dek_tag),
616     'dek_hash' => $dek_hash,
617     'encrypted_size' => strlen($ciphertext),
618     'status' => 'encrypted'
619 ]);

```

Figure 18. Document File Encryption Code

Following the key wrapping phase (Figure 17), the system proceeds to encrypt the physical file (Figure 18). The document is first compressed (Lines 554-562 in Figure 17) and then encrypted via AES-256-GCM using the plaintext DEK and a unique 96-bit nonce (Lines 596-603 in Figure 18). When a user later requests to download or view the file, the system reverses this process by unwrapping the DEK and using it to decrypt the file payload. Throughout the decryption phase, AES-GCM implicitly validates the operation by verifying the 128-bit authentication tag, immediately halting the process if any signs of ciphertext tampering are detected.

4.1.2.3 DOCUMENT SHARING

In the event that a user shares a file, StegoLock facilitates collaborative access through a cryptographic key re-wrapping process. To grant a recipient access without needing to re-encrypt the entire underlying document, the system retrieves the owner's wrapped DEK and decrypts it within volatile memory using the owner's active Master Key (Lines 1035-1042 in Figure 19).

```
1035 // 1. Unwrap the DEK using Owner's Master Key
1036 $dek = $this->cryptoService->unwrapDek(
1037     base64_decode($document->encrypted_dek),
1038     $masterKey,
1039     base64_decode($document->dek_nonce),
1040     base64_decode($document->dek_tag),
1041     base64_decode($document->dk_salt)
1042 );
1043
1044 if (!$dek) {
1045     return response()->json(['error' => 'Failed to unwrap document key.'], 500);
1046 }
1047
1048 // 2. Wrap the DEK using the System Share Key
1049 $systemKey = $this->parseKey(config('app.share_key') ?? config('app.key'));
1050 $wrapped = $this->cryptoService->wrapDek($dek, $systemKey);
1051
1052 // 3. Create or update the share and log activity in transaction
1053 \Illuminate\Support\Facades\DB::transaction(function() use ($document, $recipient, $wrapped) {
1054     DocumentShare::updateOrCreate(
1055         [
1056             'document_id' => $document->document_id,
1057             'recipient_id' => $recipient->id,
1058         ],
1059         [
1060             'sender_id' => Auth::id(),
1061             'encrypted_dek' => base64_encode($wrapped['encrypted_dek']),
1062             'dek_nonce' => base64_encode($wrapped['nonce']),
1063             'dek_tag' => base64_encode($wrapped['tag']),
1064             'dk_salt' => base64_encode($wrapped['salt']),
1065             'status' => 'pending',
1066             'expires_at' => now()->addHours(24),
1067         ]
1068     );
1069 };
```

Figure 19. Document Encryption Key Re-Wrapping Process Code

Immediately after extraction, the system re-encrypts the raw DEK using a specialized System Share Key via AES-256-GCM (Lines 1049-1050 in in Figure 19). This intermediate ciphertext allows the DEK to be

safely stored in the database's document shares table while the sharing request is pending acceptance (Lines 1054-1068 in Figure 19).

```

1089 $share = DocumentShare::where('document_id', $request->document_id)
1090 |   ->where('recipient_id', Auth::id())
1091 |   ->where('status', 'pending')
1092 |   ->firstOrFail();
1093
1094 if ($share->expires_at && $share->expires_at->isPast()) {
1095 |   return response()->json(['error' => 'This share invitation has expired.'], 403);
1096 | }
1097
1098 try {
1099 |   $masterKey = $this->getMasterKeyFromSession();
1100 | } catch (\RuntimeException $e) {
1101 |   return response()->json(['error' => $e->getMessage()], 403);
1102 | }
1103
1104 // 1. Unwrap with System Key
1105 $systemKey = $this->parseKey(config('app.share_key') ?? config('app.key'));
1106 $dek = $this->cryptoService->unwrapDek(
1107 |   base64_decode($share->encrypted_dek),
1108 |   $systemKey,
1109 |   base64_decode($share->dek_nonce),
1110 |   base64_decode($share->dek_tag),
1111 |   base64_decode($share->dk_salt)
1112 | );
1113
1114 if (!$dek) {
1115 |   return response()->json(['error' => 'Failed to unwrap document key.'], 500);
1116 | }
1117
1118 // 2. Re-wrap with User B's Master Key
1119 $wrapped = $this->cryptoService->wrapDek($dek, $masterKey);
1120
1121 // 3. Update share status and wrapped DEK
1122 $share->update([
1123 |   'encrypted_dek' => base64_encode($wrapped['encrypted_dek']),
1124 |   'dek_nonce' => base64_encode($wrapped['nonce']),
1125 |   'dek_tag' => base64_encode($wrapped['tag']),
1126 |   'dk_salt' => base64_encode($wrapped['salt']),
1127 |   'status' => 'accepted',
1128 | ]);
1129 ----

```

Figure 20. Document Encryption Key Re-Wrapping Upon Share Acceptance Code

When the recipient accepts the shared document, the system unwraps the DEK from the System Share Key and immediately re-wraps it using the recipient's own Master Key (Figure 20). Note the explicit use of `base64_decode` during metadata retrieval to maintain cryptographic

precision throughout these transitions (Lines 1109-1111 in Figure 20). This process ensures that a recipient never interacts with the owner's Master Key, and the system never exposes the plaintext DEK to persistent storage.

This cryptographic handoff ensures that the system maintains strict account isolation and continuous data confidentiality. Because the sharing process only manipulates the encryption keys rather than the document itself, the original file fragments remain completely untouched in storage. This process eliminates the need for computationally expensive re-encryption processes while guaranteeing that only explicitly authorized users possess the keys necessary to access the shared data.

4.2 DESIGN AND IMPLEMENT A SEGMENTATION PROCESS THAT SPLITS THE ENCRYPTED DOCUMENT INTO MULTIPLE SEGMENTS AND HIDES THEM THROUGH A STEGANOGRAPHIC EMBEDDING PROCESS INTO COVER FILES, WHICH ARE SCATTERED ACROSS THE APPLICATION'S CLOUD STORAGE TO ENHANCE SECURITY

The second layer of security in StegoLock focuses on concealing and distributing the encrypted data to prevent unauthorized reconstruction. This section details the implementation of the system's segmentation and steganographic processes, which work together to hide the ciphertext across multiple media files stored in the cloud.

To achieve this, the system first splits the encrypted document into several fragments based on the available capacity of assigned cover files. Each fragment is then hidden within a standard PNG, WAV, or TXT file using Least Significant Bit (LSB) steganography. By scattering these stego files across cloud storage, StegoLock ensures that no single file contains the complete document or any visible indication of its hidden contents. This approach prevents attackers from accessing the data even if they intercept individual files. The following subsections explain how the system segments the

encrypted document, embeds the fragments into multimedia covers, distributes them to the cloud, and safely reassembles them during retrieval.

4.2.1 ENCRYPTED DOCUMENT SEGMENTATION

After AES-256-GCM encryption of the document payload, the resulting ciphertext undergoes a rigorous preparatory phase before steganographic embedding can occur. This phase involves dynamically selecting appropriate cover files from the system's cover pool, fetching and locking them for exclusive use, and employing a dynamic capacity-based segmentation algorithm to split the ciphertext into fragments that perfectly fit their assigned carriers.

4.2.1.1 COVER SELECTION AND FETCHING

The segmentation process begins with the `selectCovers` method (Figures Figure 21 and Figure 22), which categorizes the incoming encrypted document into a specific size tier: large (> 2MB), medium (> 500KB), or small (Lines 233-246 in Figure 21). This tiering system enforces strict capacity boxing by defining minimum and maximum capacity thresholds, ensuring that cover files assigned to the document are appropriately sized for its payload.

```

229 private function selectCovers(Document $document)
230 {
231     $payloadSize = $document->encrypted_size;
232
233     // 1. Categorize Tier (Strict Boxing)
234     if ($payloadSize > 2097152) { // > 2MB
235         $tier = 'large';
236         $minCap = 262144; // 256KB
237         $maxCap = 104857600; // 100MB+
238     } elseif ($payloadSize > 512000) { // 500KB - 2MB
239         $tier = 'medium';
240         $minCap = 65536; // 64KB
241         $maxCap = 262144; // 256KB
242     } else {
243         $tier = 'small';
244         $minCap = 1024; // 1KB
245         $maxCap = 65536; // 64KB
246     }
247
248     $selectedCovers = collect();
249     $remainingCapacity = $payloadSize;
250
251     // 2. Mandate Selection (1 Text, 1 Audio, 1 Image)
252     $types = ['text', 'audio', 'image'];
253
254     foreach ($types as $type) {
255         $cover = $this->findBestCover($type, $minCap, $maxCap, [], 'ASC');
256
257         if ($cover && $type === 'text') {
258             $cover = $this->generate_text_cover($minCap);
259         }
260
261         if ($cover) {
262             // Fallback: try picking any capacity from this type if tier is empty
263             $cover = $this->findBestCover($type, 0, 104857600, [], 'ASC');
264         }
265
266         if ($cover) {
267             $this->notifyAdmins("Critical: No covers of type '{$type}' available in pool.");
268             throw new \Exception("Unavailable cover files ({ $type }) to continue locking. Admin notified.");
269         }
270
271         $selectedCovers->push($cover);
272         $remainingCapacity -= ($cover->metadata['capacity'] ?? 0);
273     }

```

Figure 21. Cover Selection Method Code 1

To maintain media diversity, the system mandates the selection of at least one text file, one audio file, and one image file for every locked document (Lines 251-273 in Figure 21). If these three initial covers lack sufficient combined capacity to hold the entire payload, a greedy expansion fallback mechanism is triggered, incrementally selecting the largest available covers from the pool until the required capacity is met (Lines 275-294 in Figure 22).

```

274 // 3. Greedy Expansion (if 3 fragments aren't enough)
275 while ($remainingCapacity > 0) {
276     // During expansion, we pick DESC (largest) to ensure we fit the file within 15 fragments
277     $cover = $this->findBestCover('image', 0, 104857600, $selectedCovers->pluck('cover_id')->toArray(), 'DESC');
278
279     if (!$cover) {
280         // Try audio if no images
281         $cover = $this->findBestCover('audio', 0, 104857600, $selectedCovers->pluck('cover_id')->toArray(), 'DESC');
282     }
283
284     if (!$cover) {
285         // Try text as final resort
286         $cover = $this->findBestCover('text', 0, 104857600, $selectedCovers->pluck('cover_id')->toArray(), 'DESC');
287     }
288
289     if (!$cover) break;
290
291     $selectedCovers->push($cover);
292     $remainingCapacity -= ($cover->metadata['capacity'] ?? 0);
293 }
294
295 if ($remainingCapacity > 0) {
296     $this->notifyAdmins("Critical: Total pool capacity exhausted for document (Size: {$payloadSize} bytes). No more c
297     throw new \Exception("Insufficient total cover capacity. The pool has been exhausted. Admin notified.");
298 }
299
300 return $selectedCovers;
301 }
302

```

Figure 22. Cover Selection Method Code 2

Once selected, the `fetchAndLockCovers` method retrieves these specific covers from the Backblaze B2 bucket or the local system cache (Figure 23). During this step, the database record for each cover is temporarily marked as `in_use` (Line 326 in Figure 23) to prevent concurrent locking jobs from assigning the exact same physical cover file, guaranteeing isolation during high-volume operations.

```

317 private function fetchAndLockCovers($covers)
318 {
319     $b2 = new B2Service();
320     if (!file_exists($this->jobTempPath)) {
321         mkdir($this->jobTempPath, 0755, true);
322     }
323
324     foreach ($covers as $cover) {
325         // 1. Mark in_use (Short-term copy mutex)
326         $cover->update(['in_use' => true]);
327
328         try {
329             if (isset($cover->metadata['info']) && $cover->metadata['info'] === 'System-generated') {
330                 // System-generated covers are already in the job temp folder, skip B2
331                 $localPath = storage_path("app/private/temp/covers/" . $cover->filename);
332                 $targetPath = "{$this->jobTempPath}/{$cover->filename}";
333                 if (file_exists($localPath)) {
334                     copy($localPath, $targetPath);
335                 } else {
336                     throw new \Exception("System-generated cover missing locally: {$cover->filename}");
337                 }
338             } else {
339                 $localPath = "{$this->jobTempPath}/{$cover->filename}";
340                 $cachePath = storage_path("app/private/cache/covers/{$cover->filename}");
341
342                 if (file_exists($cachePath)) {
343                     copy($cachePath, $localPath);
344                 } else {
345                     $key = $this->getCoverFolder($cover->type) . $cover->filename;
346                     $file = $b2->findFileByName($key);
347                     if (!$file) throw new \Exception("Cover file not found in cloud: {$key}");
348
349                     $content = $b2->readfile($file['fileId']);
350                     file_put_contents($localPath, $content);
351
352                     // Cache it
353                     if (!file_exists(dirname($cachePath))) mkdir(dirname($cachePath), 0755, true);
354                     copy($localPath, $cachePath);
355                 }
356             }
357
358             $this->createdTempFiles[] = $localPath;
359         } finally {
360             // 2. Release immediately after copy
361             $cover->update(['in_use' => false]);
362         }
363     }
364 }
365 }

```

Figure 23. Cover Fetching from Cloud Method Code

4.2.1.2 DYNAMIC CAPACITY-BASED SEGMENTATION

Following cover selection, the actual segmentation of the AES-encrypted payload is performed through a dynamic capacity-based segmentation algorithm. Unlike standard chunking that splits a file into equal, fixed-size pieces, this adaptive algorithm divides the data into non-uniform fragments. The size of each fragment is determined dynamically on the fly based on three governing constraints:

1. **Cover Capacity Limit:** The maximum amount of data the currently assigned cover file can safely conceal (Line 391 in Figure 24).
2. **Proportional Distribution:** A calculation that averages the remaining unsegmented data across all unused cover files in the set of selected covers, ensuring the ciphertext is spread evenly across the sequence rather than filling the first available cover to its absolute limit (Line 399 in Figure 24).
3. **Cover Utilization:** A safety limit that prevents the current fragment from using all remaining data, reserving at least one byte for every subsequent cover file so none are left empty (Line 394 in Figure 24).

```

367 private function splitDocument(Document $document, $covers)
368 {
369     $encryptedPath = Storage::path($this->encryptedPath);
370     $totalLength = filesize($encryptedPath);
371     $offset = 0;
372     $numCovers = $covers->count();
373     $mappingArray = [];
374     $handle = fopen($encryptedPath, 'rb');
375     if (!$handle) {
376         throw new \Exception("Failed to open encrypted file: {$encryptedPath}");
377     }
378     try {
379         foreach ($covers as $index => $cover) {
380             $remainingData = $totalLength - $offset;
381             $remainingCovers = $numCovers - $index - 1;
382             if ($remainingData <= 0) break;
383             if ($remainingCovers > 0) {
384                 $capacity = (int) ($cover->metadata['capacity'] ?? 0);
385                 $maxPossible = $remainingData - $remainingCovers;
386                 $fairShare = ceil($remainingData / ($remainingCovers + 1));
387
388                 $chunkSize = min($capacity, $maxPossible, max($fairShare, min($capacity, $remainingData)));
389             } else {
390                 $chunkSize = $remainingData;
391             }
392             $offset += $chunkSize;
393             $mappingArray[$index] = $chunkSize;
394         }
395     } catch (\Exception $e) {
396         // ...
397     }
398 }

```

Figure 24. Dynamic Capacity-Based Segmentation Logic Code 1

As shown in Figure 24, the algorithm reads the encrypted file sequentially and computes the size for each fragment (Lines 369-377 in

Figure 24) based on the three governing constraints: the cover capacity limit, the proportional distribution calculation, and the cover utilization safety limit (Lines 389-403 in Figure 24). The last fragment always receives all remaining bytes of the encrypted file, guaranteeing that no data is lost during the split (Line 409 in Figure 24).

```

419         $fragment = Fragment::create([
420             'fragment_id' => (string) Str::uuid(),
421             'document_id' => $document->document_id,
422             'index' => $index,
423             'blob' => base64_encode($chunk),
424             'size' => strlen($chunk),
425             'hash' => hash('sha256', $chunk),
426             'status' => 'floating',
427         ]);
429         $mappingArray[] = [
430             'fragment_id' => $fragment->fragment_id,
431             'cover_id' => $cover->cover_id,
432             'offset' => 0
433         ];
435         $offset += $chunkSize;
436     }
437 } finally {
438     fclose($handle);
439 }

```

Figure 25. Dynamic Capacity-Based Segmentation Logic Code 2

Each produced fragment is assigned a unique identifier, recorded with its sequential index position, size in bytes, and a SHA-256 hash for integrity verification, and then saved in the database with a status of 'floating' (Lines 421-427 in Figure 25). Finally, a `FragmentMap` is created to securely correlate each `fragment_id` with its assigned `cover_id`, establishing the structural blueprint that governs the subsequent multimedia embedding phase (Lines 441-446 in Figure 26).

```

441     FragmentMap::create([
442         'map_id' => (string) Str::uuid(),
443         'document_id' => $document->document_id,
444         'fragments_in_covers' => $mappingArray,
445         'status' => 'pending',
446     ]);
447     $document->update([
448         'fragment_count' => count($mappingArray),
449         'status' => 'fragmented'
450     ]);
451

```

Figure 26. Dynamic Capacity-Based Segmentation Logic Code 3

This dynamic process of segmentation ensures that fragment sizes adjust to match the exact physical constraints of their assigned cover files. By directly scaling each fragment to the safe capacity of its host cover file, the system prevents any host from exceeding its steganographic bounds, thereby achieving a mathematically balanced payload distribution without the risk of data overflow.

4.2.2 MULTIMEDIA STEGANOGRAPHIC EMBEDDING

Following encryption and segmentation, each ciphertext fragment is embedded into its designated cover file, adhering strictly to the mapping blueprint established during the segmentation phase, using Least Significant Bit (LSB) steganography.

Before the actual steganographic embedding process can be executed, the system enforces a rigorous set of preliminary specifications and validation checks on each assigned cover file. These predefined media-specific constraints ensure that the cover files are structurally sound, compatible, and possess sufficient safe embedding margins to accommodate their mapped ciphertext fragments without

compromising visual, auditory, or textual integrity. As discussed in the previous sections, StegoLock relies on three distinct media types for its system-provided cover files, each governed by unique physical characteristics and steganographic capacity thresholds to prevent structural distortion or payload detection.

4.2.2.1 SYSTEM-PROVIDED COVER FILES AND CAPACITIES

For image-based steganography, the system accepts lossless PNG cover files. During cover scanning and ingestion, the image's total LSB capacity is calculated by channels and pixels: $\text{total_bits} = \text{width} * \text{height} * \text{channels}$ (where `channels` = 3 for RGB or 4 for RGBA). To guarantee that the payload remains completely imperceptible to human eyes and steganalysis, StegoLock enforces a strict steganographic usage threshold of 15%. The safe capacity in bytes is computed by scaling this 15% threshold and subtracting 15 bytes to accommodate a custom delimiter. Any cover image with incompatible color spaces (such as grayscale or CMYK) is dynamically converted to standard RGB before capacity scanning to ensure consistency.

For audio-based steganography, uncompressed 16-bit PCM WAV cover files are utilized. To ensure high-fidelity embedding, the system checks that the sample rate is at least 44,100 Hz and the bit depth is 16-bit. Since uncompressed PCM audio assigns one LSB per sample, the total LSB capacity in bits equals the total sample count. StegoLock applies a stealth usage ratio of 15% for audio files as well, ensuring that the high-frequency LSB changes do not introduce audible static noise.

Unlike image and audio cover files which are pulled from a pre-uploaded system pool, text (TXT) cover files are dynamically generated on the fly if an appropriately sized UTF-8 text file is not available locally. The `generate_text_cover` method (Figure 27) handles this by pulling

random articles from a local `wiki_feeds` database table (Line 541 in Figure 27) and concatenating them until the resulting text file reaches a calculated target size (Line 579-588 in Figure 27). Because text steganography offers a significantly lower embedding capacity compared to binary media, StegoLock enforces an extremely aggressive 2% safety threshold to ensure natural language flow and avoid visual distortion. The system enforces the generation formula in line 554 in Figure 27, guaranteeing that the generated text cover is sufficiently large to safely embed the assigned ciphertext fragment.

```

538 private function generate_text_cover(int $fragmentSize)
539 {
540     // Check if there's any data in wiki_feeds table
541     $count = DB::table('wiki_feeds')->count();
542     if ($count == 0) {
543         // Try to seed from the WikiFeedsSeeder if empty
544         $seeder = new \Database\Seeders\WikiFeedsSeeder();
545         $seeder->run();
546     }
547     // ... [re-checking database feeds omitted for brevity]
554     $targetSize = $fragmentSize / 0.02;
555     $content = '';
556     $capacity = 0;
557     // Batch fetch wiki feeds for better performance
558     $batchSize = 100;
559     $usedIds = [];
560     while (strlen($content) < $targetSize) {
561         // ... [batch random feed fetching and exhaustion fallback omitted for brevity]
579         foreach ($feeds as $feed) {
580             $block = "pageid: {$feed->pageid}\ntitle: {$feed->title}\ncontent: {$feed->feed}\n\n";
581             $content .= $block;
582             $usedIds[] = $feed->id;
583             $capacity = floor(strlen($content) * 0.02);
584             if (strlen($content) >= $targetSize) {
585                 break;
586             }
587         }
588     }
589 }
590 // ... [saving content to file and creating Cover record omitted for brevity]
612 }

```

Figure 27. Cover Text Generation Code

4.2.2.2 LSB EMBEDDING APPLICATION

The steganographic embedding engine in StegoLock is invoked once all cover files are assigned and prepared. The system processes each

fragment by decoding its base64 representation and writing it to a temporary binary file. Based on the detected media type of the assigned cover file, StegoLock routes the payload to the corresponding Python embedding script (text/embed.py in Figure 28, audio/embed.py in Figure 29, or image/embed.py in Figure 30), executing them as an external process. The detailed implementation and mechanical behavior of the LSB embedding application are structured across the three supported media formats.

```

1 import sys
2 import random
3 import numpy as np
4
5 DELIMITER = b'###STEGOLOCK###'
6
7 def embed(input_text_file, output_text_file, data_file):
8     """
9     Embeds payload into text file LSB using NumPy for memory efficiency.
10    """
11    with open(input_text_file, 'rb') as f:
12        cover_bytes = np.fromfile(f, dtype=np.uint8)
13
14    with open(data_file, 'rb') as f:
15        payload_bytes = f.read() + DELIMITER
16
17    payload_bits = np.unpackbits(np.frombuffer(payload_bytes, dtype=np.uint8))
18
19    cover_len = len(cover_bytes)
20    payload_len = len(payload_bits)
21
22    if payload_len > cover_len:
23        raise Exception("Payload too large for this text file!")
24
25    max_offset = cover_len - payload_len
26    offset = random.randint(0, max_offset)
27
28    cover_bytes[offset : offset + payload_len] = (
29        cover_bytes[offset : offset + payload_len] & 0xFE
30    ) | payload_bits
31
32    with open(output_text_file, 'wb') as f:
33        f.write(cover_bytes.tobytes())
34
35    print(offset)

```

Figure 28. Text LSB Embedding Engine Code

For text-based covers, steganographic embedding relies on converting the natural language text document into a raw sequence of bytes. The binary representation of the ciphertext fragment is appended with a distinct structural delimiter (Line 5 in Figure 28) that indicates the boundaries of the hidden payload during retrieval. The combined payload

is then broken down into individual bits. To counter spatial steganalysis and eliminate predictable patterns, StegoLock implements a randomized insertion point, where the start index of the hidden data is shifted dynamically (Lines 28-30 in Figure 28). Within this randomized window, the least significant bit of each individual text byte is replaced with a payload bit using vectorized bitwise masking and combination operations. Once the substitution is complete, the modified byte stream is written back as a clean text file (Lines 32-33 in Figure 28). The computed start offset is printed by the embedding engine and permanently stored in the application database (Line 35 in Figure 28), providing a precise starting coordinate that allows the decryption pipeline to find and extract the ciphertext fragment during reassembly.

```

1 import sys
2 import numpy as np
3 from scipy.io import wavfile
4
5 DELIMITER = b'###STEGLOCK###'
6
7 def embed_wav(input_wav, output_wav, payload_file):
8     """
9     Embeds payload into WAV LSB using NumPy for memory efficiency.
10    """
11    rate, audio = wavfile.read(input_wav)
12    original_shape = audio.shape
13    dtype = audio.dtype
14
15    audio_flat = audio.flatten()
16
17    with open(payload_file, "rb") as f:
18        payload = f.read() + DELIMITER
19
20    payload_bits = np.unpackbits(np.frombuffer(payload, dtype=np.uint8))
21    num_bits = len(payload_bits)
22
23    if num_bits > len(audio_flat):
24
25        raise Exception(f"Payload too large for this WAV. Max bits: {len(audio_flat)}, required: {num_bits}")
26
27    if audio_flat.dtype == np.uint8:
28        audio_flat[:num_bits] = (audio_flat[:num_bits] & 0xFE) | payload_bits
29    else:
30        audio_flat[:num_bits] = (audio_flat[:num_bits] & ~1) | payload_bits
31
32    audio_embedded = audio_flat.reshape(original_shape)
33
34    wavfile.write(output_wav, rate, audio_embedded.astype(dtype))
35
36    print(0)

```

Figure 29. Audio LSB Embedding Engine Code

For audio-based covers, high-fidelity uncompressed sound files are processed by loading their underlying waveform samples. The sound wave data, regardless of whether it represents mono or stereo channels, is organized into a contiguous array of audio amplitudes. Similar to the text embedding process, the ciphertext fragment is combined with a terminal delimiter (Line 5 in Figure 29) and expanded into a bitstream. Starting from the very beginning of the audio track, the embedding engine systematically alters the least significant bit of each audio sample to match the corresponding bit of the payload. The system dynamically adapts its bit-

clearing masks to match the exact bit depth of the carrier, employing unique bitwise adjustments for unsigned 8-bit audio versus signed 16-bit sound structures to prevent volume clipping or acoustic distortion. Finally, the audio waveform is reconstructed into its original channel layout and exported as a clean stego-audio file, preserving the original sound quality while masking the hidden payload.

```

1 import sys
2 import numpy as np
3 from PIL import Image
4
5 SAFETY_PERCENT = 15
6
7 def get_image_safe_capacity(image_path):
8     img = Image.open(image_path)
9     width, height = img.size
10    channels = len(img.getbands())
11    total_bits = width * height * channels
12    total_bytes = total_bits // 8
13    safe_bytes = int(total_bytes * (SAFETY_PERCENT / 100))
14    return width, height, safe_bytes
15
16 def embed(image_path, output_path, data_bytes):
17     img = Image.open(image_path)
18     if img.mode not in ['RGB', 'RGBA']:
19         img = img.convert('RGB')
20
21     img_data = np.array(img)
22     shape = img_data.shape
23     dtype = img_data.dtype
24
25     DELIMITER = b'###STEGOLOCK###'
26     full_payload = data_bytes + DELIMITER
27
28     _, _, safe_bytes = get_image_safe_capacity(image_path)
29     if len(data_bytes) > safe_bytes:
30         raise Exception(f"Payload too large! Max safe size: {safe_bytes} bytes")
31
32     payload_bits = np.unpackbits(np.frombuffer(full_payload, dtype=np.uint8))
33     flat_img = img_data.flatten()
34
35     if len(payload_bits) > len(flat_img):
36         raise Exception("Payload exceeds total LSB capacity of the image")
37
38     flat_img[:len(payload_bits)] = (flat_img[:len(payload_bits)] & 0xFE) | payload_bits
39
40     optimized_img_data = flat_img.reshape(shape)
41     final_img = Image.fromarray(optimized_img_data.astype(dtype))
42     final_img.save(output_path)
43     print(0)

```

Figure 30. Image LSB Embedding Engine Code

For image-based covers, steganographic concealment operates in the spatial domain of lossless graphics files. During the initial processing phase, the image mode is validated and standardized to a color space

representing red, green, and blue channels to maintain consistent capacity. The graphic's multi-dimensional pixel matrix is mapped into a continuous sequence of color channel bytes. The payload bits, complete with their trailing delimiter, are then sequentially substituted into the least significant bit of the pixel byte sequence. A vectorized bitwise mask is applied to clear the existing least significant bits before integrating the payload bits, ensuring that the slight alteration in pixel colors remains imperceptible. The modified color channel sequence is then reconstructed into its original visual dimensions and exported as a lossless graphic file. Because this format avoids compression artifacts, the individual pixel values are preserved perfectly, ensuring that the hidden bits remain intact and can be retrieved with absolute fidelity.

4.2.3 STEGO FILES CLOUD STORAGE DISTRIBUTION

After encryption, segmentation, and LSB embedding of the ciphertext fragments to create stego files, the physical distribution and persistence of these files within the cloud storage repository are initiated. Within the StegoLock system, each stego file is stored as an entirely independent object in the cloud bucket. Consequently, no single stego file contains the complete encrypted document, and no individual file presents any visible indication or metadata indicating that it conceals a steganographic payload. This physical and spatial segregation of fragments across independent cloud objects establishes the final structural barrier of the reconstruction-dependent security model, ensuring that document recovery is structurally impossible without the retrieval and correct reassembly of all constituent parts.

```

1 // 1. Capacity Gate Check
2 $user->refresh();
3 $remainingSpace = max(0, $user->storage_limit - $user->storage_used);
4 $totalStegoSize = array_sum(array_map('filesize', $stegoFilePaths));
5
6 if ($totalStegoSize > $remainingSpace) {
7     foreach ($stegoFilePaths as $path) {
8         if (file_exists($path)) @unlink($path);
9     }
10    throw new \Exception("CAPACITY_EXCEEDED: Insufficient cloud space.");
11 }
12
13 // 2. Parallel Upload & Persistence Layer Mapping
14 $b2->storeFilesBatch($stegoFilePaths, 5, function($path, $info) use ($newStegoMap, $stegoMap, $document) {
15     $smatch = collect($stegoMap)->first(fn($s) => $s['stegoFile'] === $path);
16     if ($smatch) {
17         $sFile = StegoFile::create([
18             'stego_map_id' => $newStegoMap->stego_map_id,
19             'cloud_file_id' => $info['fileId'],
20             'fragment_id' => $smatch['fragmentId'],
21             'offset' => $smatch['offset'],
22             'filename' => basename($path),
23             'stego_size' => $info['contentLength'],
24             'status' => 'embedded',
25         ]);
26         $document->increment('in_cloud_size', $sFile->stego_size);
27     }
28     if (file_exists($path)) @unlink($path); // Conserve disk space
29 });
30
31 // 3. User Storage Refresh & Locking Completion
32 $user->refreshStorageUsed();
33 $newStegoMap->update(['status' => 'completed']);
34 $document->update(['status' => 'stored']);

```

Figure 31. Stego Files Capacity Gate and Parallel Cloud Storage Distribution Code

To execute the transfer, the system targets the Backblaze B2 cloud storage bucket under the `locked/` virtual directory. Prior to initiating any network transmission, the background process enforces a strict capacity gate check by retrieving the user's storage quota parameters (`storage_limit` and `storage_used` as illustrated in Lines 1-11 in Figure 31) from the database to compute the remaining available capacity. It then sums the actual sizes of all locally generated stego files. If this aggregate size exceeds the user's remaining available space, the pipeline is terminated immediately with a capacity exception, and all local temporary stego files are unlinked. This proactive validation avoids

unnecessary network overhead and strictly maintains storage limits at the edge. Once the capacity gate check is cleared, the system uploads the files in parallel batches of five using concurrent HTTP connection pools to optimize remote bucket interactions.

For each successfully uploaded stego file, the persistence layer records the transfer by inserting a new record into the `stego_files` database table (Lines 17-25 in Figure 31). This record links key parameters including the primary key (`stego_file_id`), the cloud object identifier (`cloud_file_id`), the segment reference (`fragment_id`), the parent mapping identifier (`stego_map_id`), the randomized name (`filename`), the exact byte size (`stego_size`), and the steganographic insertion offset (`offset`). As each upload in the batch pool finishes, the system increments the document's `in_cloud_size` by the size of the uploaded stego file (Line 26 in Figure 31). Once the entire batch is successfully transferred, the system calls `refreshStorageUsed` method (Line 32 in Figure 31) to update the user's cumulative `storage_used` in the database, transitions the status of the `StegoMap` record to completed, and marks the overall document status as stored. Finally, the local temporary stego files are systematically unlinked from the application server's disk space (Line 28 in Figure 31), confirming that the entire locking pipeline has executed successfully and that the stego files are securely persisted in the cloud.

4.3 DEVELOP A WEB-BASED APPLICATION THAT IMPLEMENTS AND INTEGRATES THE AES-BASED ENCRYPTION, SEGMENTATION, ACCESS CONTROL AND AUTHENTICATION, AND SHARING MECHANISMS TO A DOCUMENT STORAGE PLATFORM

This section presents the results of the third objective: the development and full integration of StegoLock, a web-based document storage platform that incorporates AES-256-GCM encryption, document segmentation, role-based access control, secure

user authentication, and a controlled document-sharing mechanism. The system was designed and built to validate that all previously developed components as discussed in Sections 4.1 and 4.2 function cohesively as a unified, deployable application. The following sub-sections present each major functional area of the system through interface demonstrations, design descriptions, and supporting technical references.

The application's frontend was developed with React and Inertia.js , providing a responsive, intuitive user interface. The backend is powered by Laravel (PHP 8.4), which handles all business logic, authentication, encryption orchestration, and API communication. The system uses a MySQL relational database for persistent data storage, including user account records, document metadata, fragment-tracking tables, and access control assignments. Cloud-based object storage is provided by Backblaze B2, where all steganographically processed file fragments are retained. The application is deployed on Railway, a platform-as-a-service (PaaS) provider that hosts both the application server and the MySQL database instance in a production-grade cloud environment.

4.3.1 STEGOLOCK WEB APP

The StegoLock web application provides the primary interface through which users interact with all system capabilities. The interface is organized into distinct pages and functional sections, each designed to present document management, access control, and security operations clearly and accessibly. The following subsections describe the major pages and features of the application.

Home / Welcome Page

The Home page (Figure 32) serves as the public-facing entry point of the StegoLock platform. The hero section prominently displays the application name and a brief description of its purpose, establishing the platform's identity for new and returning visitors. Navigation controls direct unauthenticated users

toward either the Login or Registration pages. The page conveys the system's core value proposition—secure, steganography-based document storage—without exposing sensitive interface elements to unauthorized users. The visual design of the hero section is intended to convey professionalism and security, reinforcing the platform's intended use in academic and organizational settings.

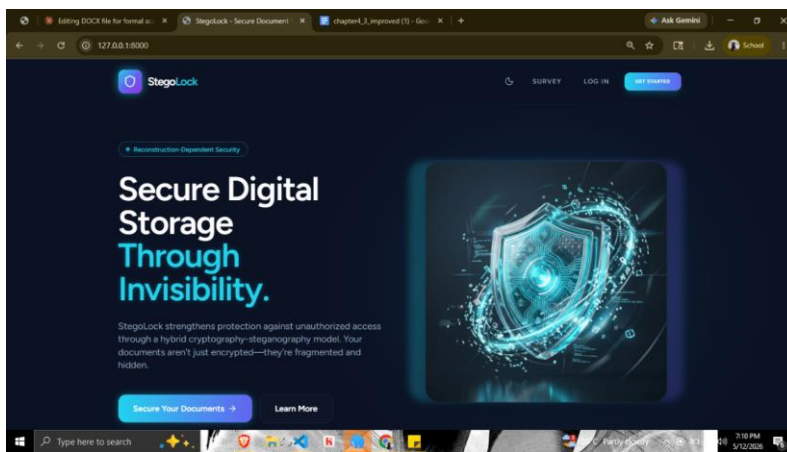


Figure 32. StegoLock Home/Welcome Page

Login and Registration Pages

The Registration page (Figure 33) allows new users to create an account by providing a unique username and password. The submitted password is hashed using a cryptographic one-way function before being stored in the database, ensuring that plaintext credentials are never persisted. Client-side validation enforces input requirements before the form is submitted, improving user experience and reducing invalid submissions. Upon successful registration, the user is redirected to the Login page.

The Login page (Figure 34) presents a minimal, focused interface consisting of a username field, a password field, and a submit button. Upon successful credential verification, the system establishes an authenticated session and grants the user access to the main dashboard. Failed login attempts return a generic error message to prevent user enumeration.

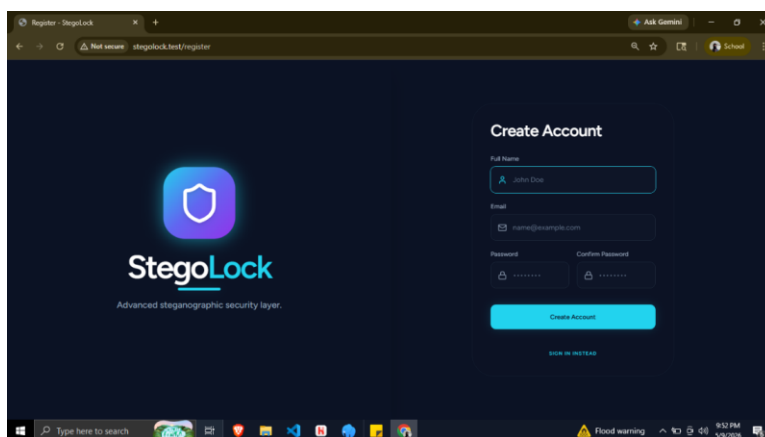


Figure 33. User Registration Interface

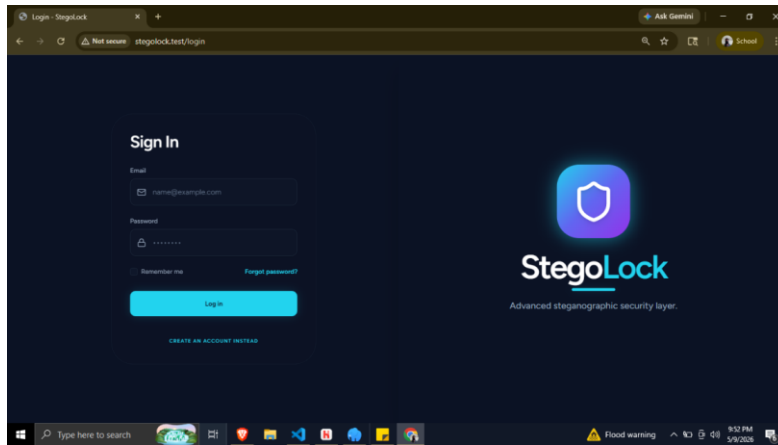


Figure 34. User Login Interface

MyDocuments Page

The MyDocuments page (Figure 35) functions as the primary dashboard for authenticated users. It presents a consolidated view of all documents owned by or explicitly shared with the current user. From this interface, users may initiate document locking (upload and encryption), document unlocking (retrieval and decryption), sharing, and deletion operations. Each document entry displays essential metadata, including the document name, upload date, file size, and current lock status. Contextual action buttons rendered per document entry provide direct access to the corresponding document operations without requiring navigation away from the dashboard.

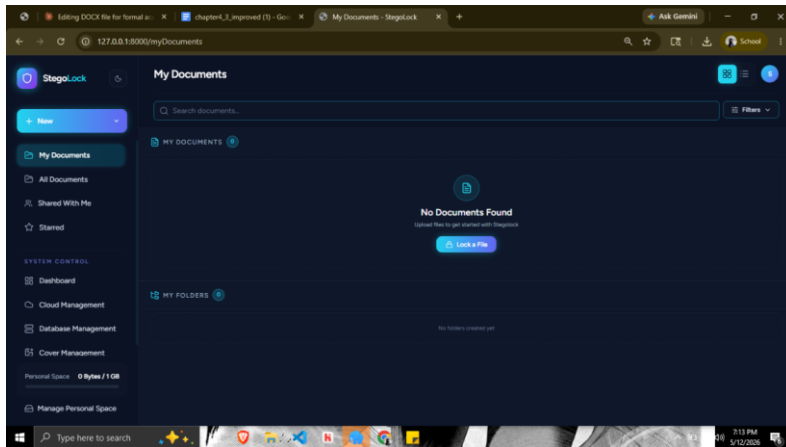


Figure 35. MyDocuments Page — Main User Dashboard

Sidebar Navigation

The sidebar (Figure 36) provides persistent navigation throughout the authenticated sections of the application. It includes links to the MyDocuments dashboard, the Starred Documents section, Shared Documents, and the user Profile page. The sidebar additionally displays a visual summary of the user's current storage utilization, indicating the proportion of the allocated 250 MB quota that has been consumed. This persistent navigation element ensures continuity across functional areas of the application without disrupting the user's current context.

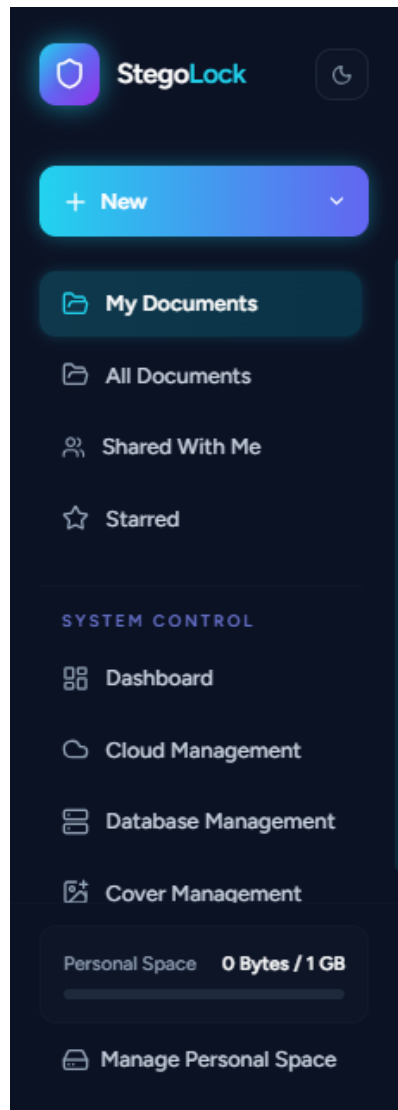


Figure 36. Application Sidebar Navigation

Profile and Personal Space Management

The Profile page allows authenticated users to review and update their account information, including their display name and password. Users may also monitor their personal storage consumption from this page. The interface presents the current usage relative to the total allocated quota, enabling informed decisions regarding document management and deletion. Storage statistics are derived from the aggregate size of all documents associated with the user account, including locked and unlocked states.

Additional Interface Features

Beyond the primary document management operations, the StegoLock web application incorporates several supporting features that enhance usability and document organization:

- **Star / Favorite Function:** Users may mark frequently accessed documents for quick retrieval by starring them. Starred documents are accessible from a dedicated section in the sidebar, reducing navigation time for users who regularly return to specific files.
- **Search Bar:** A real-time search bar enables users to filter the document list by filename. The search function operates on the client-rendered list, providing immediate feedback without requiring a full page reload.
- **Sort and Filter Controls:** Users may reorder the document list by name, upload date, or file size. Filter controls allow the list to be narrowed by document status, such as locked, shared, or starred, facilitating efficient navigation within large document collections.

- **Grid / List Toggle:** The display mode of the MyDocuments dashboard may be toggled between a visual grid layout and a compact tabular list view. This accommodates different user preferences for document browsing and provides flexibility in how document metadata is presented.

4.3.2 ACCESS CONTROL AND AUTHENTICATION

StegoLock enforces identity verification through a secure registration and login system. New credentials are secured via cryptographic hashing, ensuring plaintext passwords are never stored. Upon successful authentication, the system issues a session token that governs access rights.

Application-layer security is maintained through a Laravel middleware pipeline. Every request is validated against the user's ownership and permission records (RBAC) before execution. This middleware intercepts unauthorized attempts including direct URL manipulation to ensure strict data isolation and prevent cross-account leakage. By rejecting invalid requests at the gateway, the system maintains a secure environment where users can only interact with their authorized documents.

The authentication logic is implemented in **AuthController.php**, which handles user registration, login, logout, and session lifecycle management. The code snippet in Figure 37 illustrates the password hashing applied during the registration process

```
// AuthController.php
$user->password = Hash::make($request->input('password'));
$user->save();
```

Figure 37. Implementation of Secure Password Hashing

Access control for authenticated routes is enforced via Laravel's built-in **auth** middleware, as illustrated in the route definitions in Figure 38.

```
// routes/web.php
Route::middleware(['auth'])->group(function () {
    Route::get('/documents', [DocumentController::class, 'index']);
    Route::post('/documents/lock', [DocumentController::class, 'lock']);
    Route::post('/documents/unlock', [DocumentController::class, 'unlock']);
    Route::post('/documents/share', [DocumentController::class, 'share']);
});
```

Figure 38. Authenticated Document Routes and Middleware Protection

It is important to note that the AES-256-GCM encryption and key management mechanisms described in Section 4.1 are operationalized within this authentication context. Encryption keys are generated and associated with the authenticated user's account upon document locking and are retrieved securely during the unlocking process. The **CryptoService** class mediates all cryptographic operations, ensuring that key access is gated by authenticated session verification. This integration ensures that no cryptographic operation can be performed outside of a valid, authenticated session, directly satisfying the access control requirements specified in the study's objective statement.

4.3.3 DOCUMENT LOCKING AND UNLOCKING

Document locking refers to the process by which an uploaded document is encrypted, segmented into steganographic fragments, and distributed to cloud storage. Document unlocking is the complementary process by which the stored fragments are retrieved, extracted, reassembled, and decrypted to reconstruct the original file. The technical implementation of these pipelines including the AES-256-GCM cryptographic algorithms and the steganographic embedding logic is detailed in Sections 4.1 and 4.2 respectively. This section focuses on the user-

facing interface through which these operations are initiated, and provides a concise overview of the corresponding backend behavior.

Document Locking Interface

From the MyDocuments dashboard, authenticated users initiate the document locking process by selecting a file for upload through the provided file dialog (Figure 39). Upon confirmation, the frontend dispatches the file to the backend, which queues the **ProcessSteganoJob**, a **background** job responsible for executing the full locking pipeline. The interface displays the document's processing status in real time, transitioning from "Processing" to "Locked" upon successful completion of the job. If an error occurs during processing, the interface reflects the failure state and provides a prompt for user action.

In the backend, the locking pipeline proceeds in the following sequence: the **CryptoService** encrypts the uploaded file using AES-256-GCM; the resulting ciphertext is segmented into UUID-tagged fragments, with a structural record (**FragmentMap**) persisted in the database; Python scripts are invoked to embed each fragment into a carrier file via steganography; and the resulting stego-files are uploaded to Backblaze B2 via the **B2Service**, with a corresponding storage mapping (**StegoMap**) saved to the database for future retrieval.

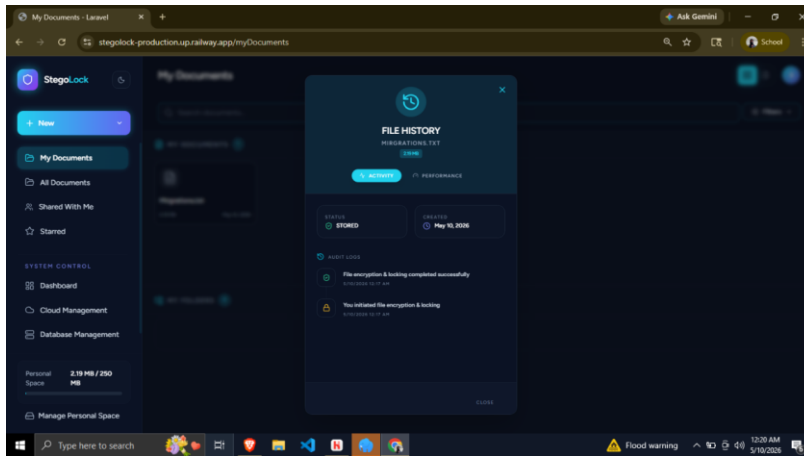


Figure 39. Document Upload and Locking Interface

Segmented Storage in Cloud

Upon completion of the locking process, the resulting stego-files are stored as independent objects in the Backblaze B2 bucket (Figures 40-41). Each object corresponds to a single fragment of the original encrypted document, embedded within a carrier file. Because the fragments are physically distributed as separate cloud objects, no single intercepted file is sufficient to reconstruct the document.

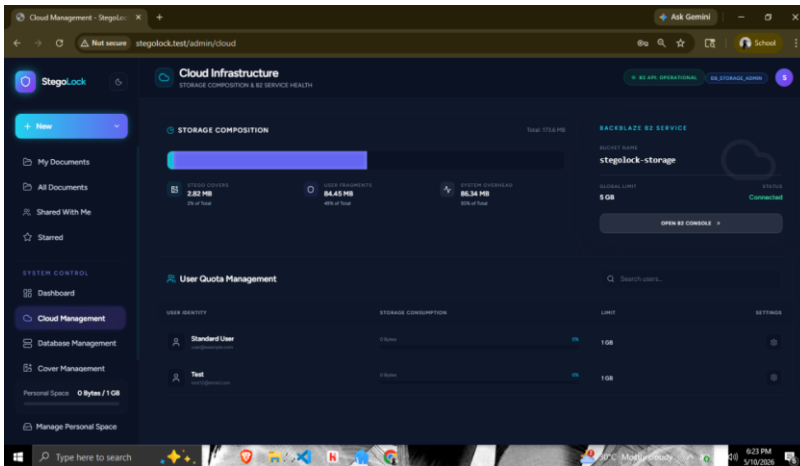


Figure 40. Segmented Document Storage in Cloud Buckets (Part 1)

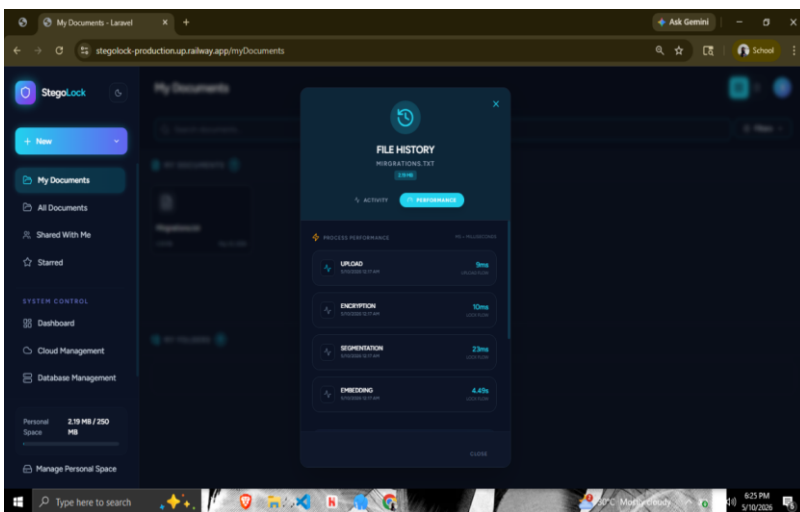


Figure 41. Segmented Document Storage in Cloud Buckets (Part 2)

Document Unlocking Interface

To retrieve a locked document, the authenticated user selects the target document from the dashboard and initiates the unlock operation (Figure 42). The interface confirms the request and displays a progress indicator while the backend processes the **ProcessUnlockJob**. The unlocking pipeline retrieves the relevant stego-files from Backblaze B2 using the **StegoMap**, extracts the hidden fragments via **batch_processor.py**, validates each fragment using SHA-256 checksums to ensure integrity, reassembles the validated fragments into a single encrypted blob through a memory-efficient streaming technique, and finally decrypts the blob using the **CryptoService**. The reconstructed file is then presented to the user for download, confirming lossless end-to-end integrity across the full encrypt–segment–store–retrieve–reconstruct–decrypt pipeline.

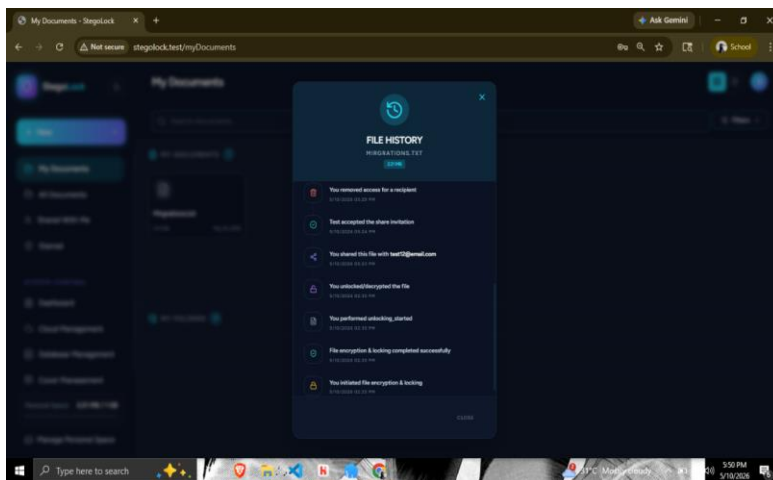


Figure 42. Document Retrieval and Reconstruction Interface

4.3.4 DOCUMENT SHARING

The StegoLock platform provides a controlled document-sharing mechanism that enables document owners to grant read access to other registered users. Access is managed through explicit permission records stored in the database; only users whose accounts are explicitly linked to a document's share list may initiate an unlock operation on that document. The sharing mechanism does not transmit or expose the underlying AES-256-GCM encryption key to the recipient. Instead, shared users access documents through the same server-side decryption pipeline, with the system verifying the user's permission before executing any cryptographic operation. Ownership and all associated access rights remain with the original document uploader.

The document sharing interface is accessible from the MyDocuments dashboard (Figure 43). The document owner selects a target document and opens the sharing panel, which provides a form for entering the username of the intended recipient. Upon form submission, the system creates a permission record in the database that associates the specified user account with the document. The owner's view of the sharing panel displays the current list of users who have been granted access, alongside individual revocation controls.

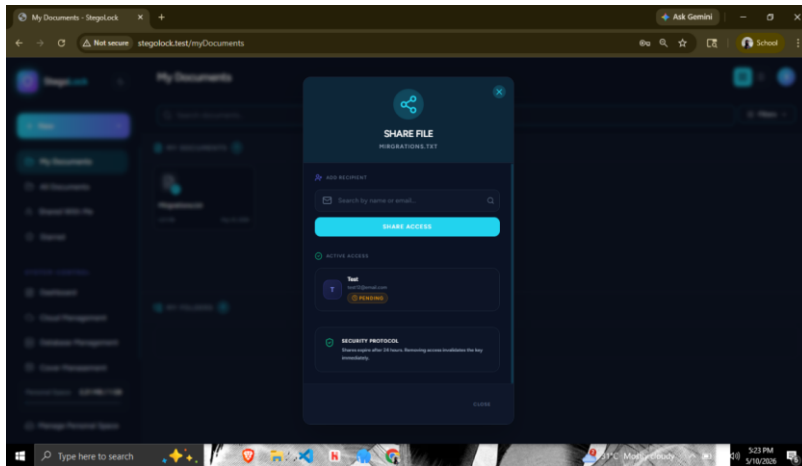


Figure 43. Document Sharing Interface — Owner's Perspective

Role-based access distinctions are enforced and reflected in the interface (Figures 44-45). The document owner is presented with the full management panel, including the list of active share recipients and revocation controls. A shared user, upon authenticating to their own account, finds the shared document listed in their MyDocuments dashboard. The shared user may initiate an unlock operation on the document but is not presented with sharing, deletion, or ownership controls. This asymmetry in the interface confirms that ownership and access boundaries are correctly maintained by the underlying access control layer.

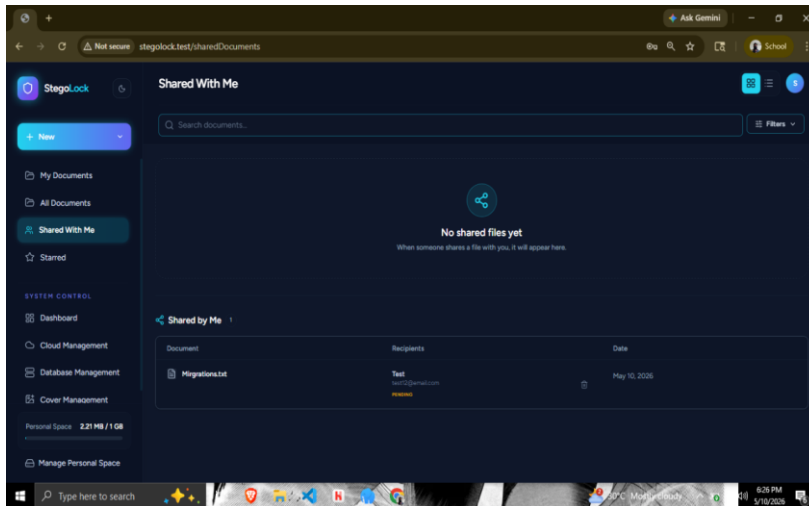


Figure 44. Role-Based Access: Owner View

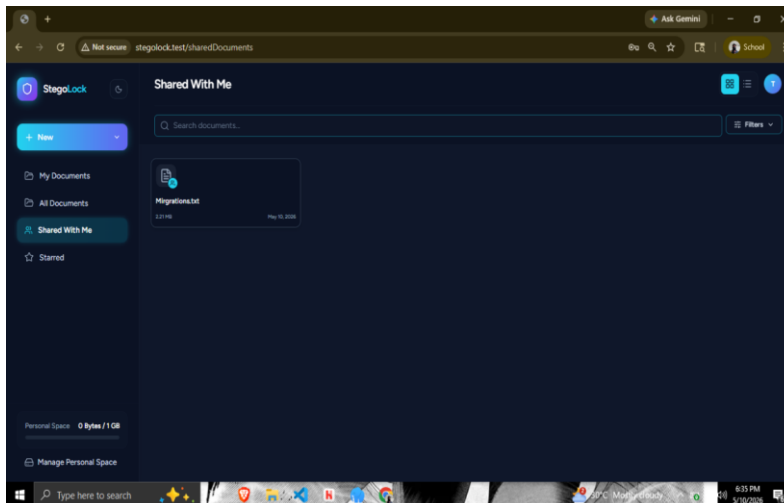


Figure 45. Role-Based Access: Shared User View

Access denial for unauthorized users was also validated. As shown in Figure 46, a user who attempts to retrieve a document without ownership or an explicit permission grant receives an access denied response from the system, confirming that document-level permission enforcement is correctly applied at the access control layer.

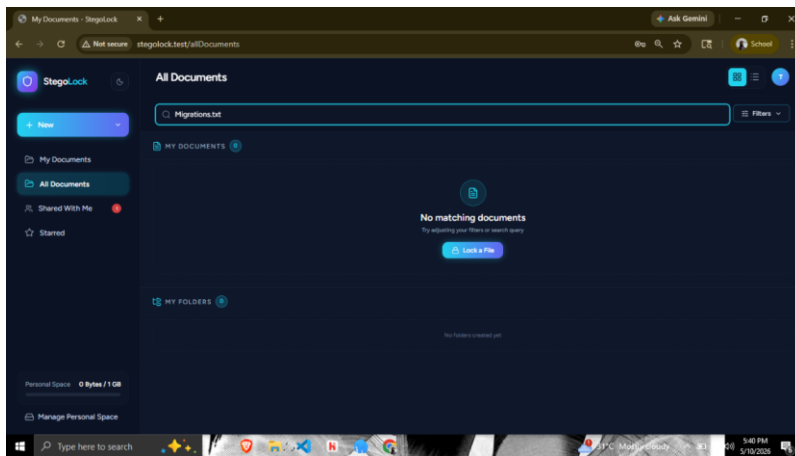


Figure 46. Access Denial for Unauthorized User

Access revocation was likewise validated. After the document owner removes a shared user's permission through the interface, that user's subsequent unlock requests are rejected. Figures 47 and 48 demonstrate an active sharing session in which the recipient account successfully accesses the shared document, confirming that the permission grant is functioning prior to any revocation.

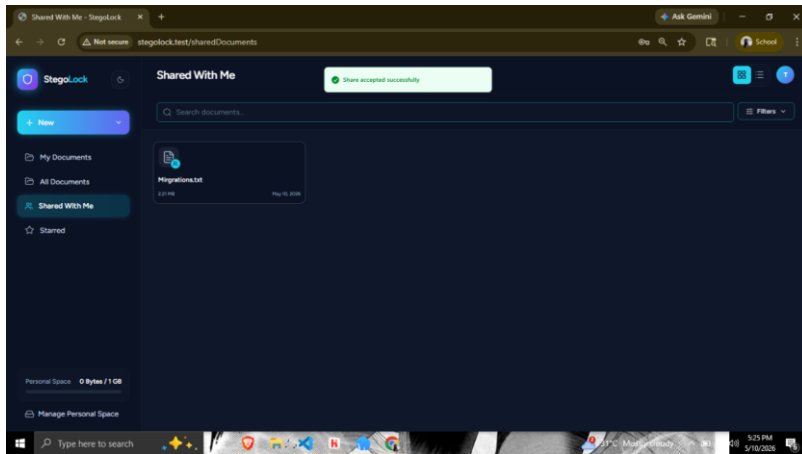


Figure 47. Active Document Sharing — Recipient Access Confirmed

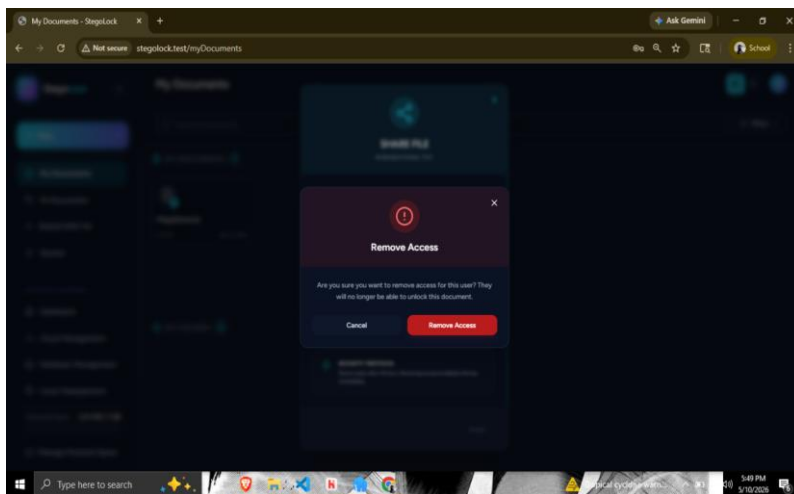


Figure 48. Active Document Sharing — Owner Revoked Access Confirmed

4.3.5 DOCUMENT STORAGE

The StegoLock platform integrates Backblaze B2 as its cloud-based object storage backend. All steganographic output files—each containing an embedded fragment of an encrypted document—are stored as discrete objects within a designated B2 storage bucket. This design physically distributes the fragments of any given document across the cloud, ensuring that no single intercepted object provides sufficient data to reconstruct the original file or recover the plaintext content.

Integration of Backblaze Cloud Storage

Integration with Backblaze B2 is achieved through Laravel’s filesystem abstraction layer, utilizing an S3-compatible storage driver configured via the following environment variables: **B2_KEY_ID**, **B2_APPLICATION_KEY**, **B2_BUCKET**, and **B2_ENDPOINT**. Sensitive cloud credentials are managed through the **CloudAccount** model, which stores application keys in the database in encrypted form using Laravel’s **Crypt** service, preventing credential exposure in the event of a database breach. The relationship between a document’s fragments and their corresponding Backblaze **fileId** values is maintained in the **StegoMap** and **StegoFile** database tables, which the system consults during retrieval to locate and fetch the correct objects from the bucket.

Specifications and Capacities

The following storage parameters govern the operational behavior of the system:

- **Global Capacity Allocation:** The system is configured with a total cloud storage capacity of 10 GB (10,737,418,240 bytes), allocated within the Backblaze B2 bucket.

- **User Storage Threshold:** Each registered user is allocated a personal storage quota of 250 MB (262,144,000 bytes), enforced at the application level prior to any upload operation.
- **Steganographic Safety Margin:** A 15% capacity safety margin is enforced during the embedding process to ensure that the hidden payload does not occupy an excessive proportion of the carrier file, preventing perceptible degradation of the cover medium.
- **Batch Concurrency:** Parallel upload, download, and deletion operations are configured with a default concurrency limit of five simultaneous slots, balancing throughput performance against Backblaze B2 API rate constraints.

The API: B2Service.php

The **B2Service.php** file serves as the primary programmatic gateway for all interactions between the StegoLock backend and the Backblaze B2 API. Its core functionalities are organized into the following categories:

Authentication and Token Caching:

- **authorize():** Performs the initial account authorization handshake with the Backblaze B2 API, retrieving an authorization token and the bucket download URL required for subsequent operations.
- **getAuth():** Retrieves and caches the authorization token for 3,500 seconds to minimize redundant API calls during high-throughput batch operations.

File Transfer Operations:

- **uploadFile()**: Handles standard single-file uploads to the B2 bucket, including SHA-1 checksum generation for server-side data integrity verification.
- **storeFilesBatch()**: Utilizes Guzzle HTTP client pools to execute parallel file uploads with a built-in retry mechanism for **auth_token_limit** errors, ensuring resilience during high-volume operations.
- **fetchFilesBatch()**: Downloads multiple stego-file fragments simultaneously from the B2 bucket, accelerating the document reconstruction process during unlocking.

Management and Retrieval:

- **listAllFiles()**: Implements a recursive do-while loop to enumerate all objects in the B2 bucket, handling API pagination and token expiration automatically.
- **deleteFilesBatch()**: Executes asynchronous deletion of multiple file versions during document removal or storage cleanup, leveraging Guzzle pools for concurrent execution.
- **getFileInfo()** and **findFileByName()**: Provide granular metadata retrieval for specific stego-file fragments, supporting system diagnostics and cloud integrity audit operations.

4.3.6 APPLICATION SYSTEM DEPLOYMENT

The StegoLock web application was deployed to a live cloud production environment using Railway, a platform-as-a-service (PaaS) provider that supports containerized application hosting and managed database provisioning. The deployment architecture consists of two interconnected services: the main application server, which runs the Laravel-based backend, and a dedicated

MySQL database instance (Figure 49). Both services operate within the same production environment and are accessible at the public URL **stegolock-production.up.railway.app**. The application server runs PHP 8.2 with Node.js 22, deployed on a US West region instance to ensure availability and low-latency access.

The database service hosts the **stegolock_app** MySQL database, which contains all application tables, including **activity_logs**, **documents**, **document_segments**, **fragment_metadata**, **fragments**, **cloud_accounts**, **folder_shares**, **folders**, **covers**, **notifications**, and user management tables (Figure 50). The presence of the complete schema in the live database confirms that all Laravel migrations were executed successfully against the production MySQL instance.

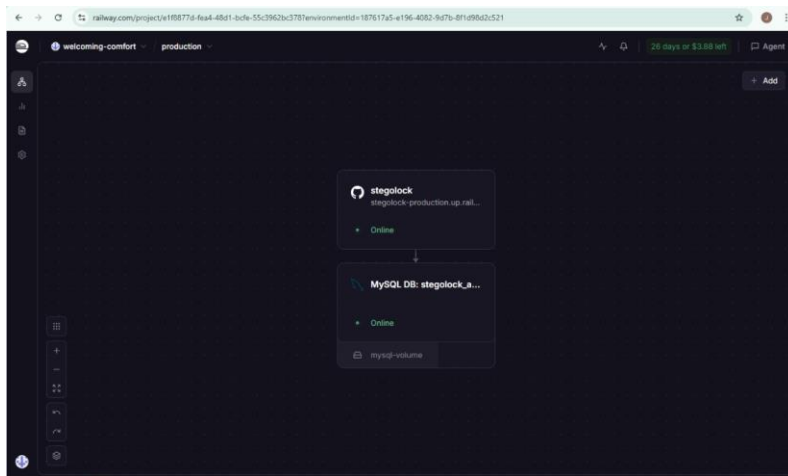


Figure 49. Railway Production Environment — Application and Database Services Online

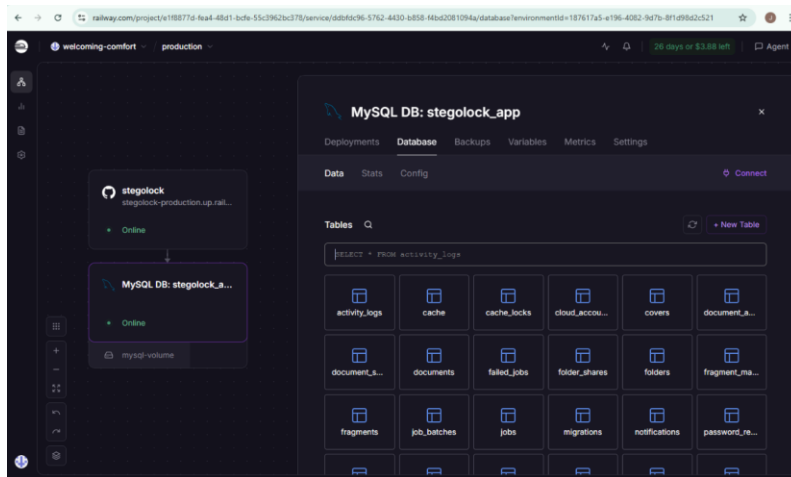


Figure 50. Railway Database Browser — stegolock_app MySQL Schema in Production

Deployment to Railway is integrated with the project’s GitHub repository through an automated continuous deployment pipeline. Each commit pushed to the main branch automatically triggers a build and deployment cycle on Railway, promoting code changes to the live production environment without manual intervention. The deployment history (Figure 51) of the application service records a sequence of iterative releases, including feature additions such as Multi-Account Management, an administrative database table inspector, and Backblaze cloud integrity audit functionality, reflecting the incremental development approach employed throughout the project.

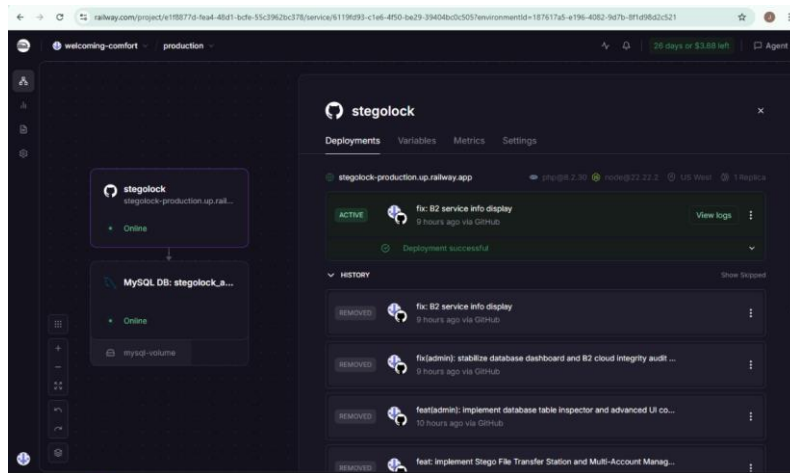


Figure 51. Railway Deployment History — Active and Historical Deployments via GitHub CI/CD

The successful cloud deployment of StegoLock confirms that the platform operates correctly outside of a local development environment, that all database migrations and schema definitions are intact in production, and that the GitHub-to-Railway continuous deployment pipeline functions as intended. The live deployment constitutes conclusive evidence that Objective 3—the development and full integration of the web-based document storage platform incorporating AES-256-GCM encryption, segmentation, access control, authentication, and document sharing—has been fully realized.

4.4 EVALUATE THE APPLICATION BASED ON ISO/IEC 25010 QUALITY CHARACTERISTICS TO ASSESS THE EFFECTIVENESS IN TERMS OF FUNCTIONAL SUITABILITY, SECURITY, RELIABILITY, AND MEASURE USABILITY AND PERFORMANCE EFFICIENCY

Following the successful implementation of the StegoLock system, including its AES-based encryption, KDF-based key management, segmentation, and

steganographic embedding, and development of the application platform, the final phase of the study shifts toward evaluating its overall quality. This section addresses the fourth objective by assessing the application based on the ISO/IEC 25010 standard, focusing on its effectiveness, security, and efficiency. To ensure a thorough evaluation, the researchers used a two-part approach: measuring the application's technical performance through internal system logs and gathering subjective feedback from users through surveys. By testing the system in this manner, the study determines if the security and usability goals established during the development stages were fully achieved in the final product.

4.4.1 USER PROFILE

To evaluate the application's performance and security, a controlled testing environment was established. The storage capacity was primarily determined by the third-party cloud service provider's free-tier allocation of **10 GB**, which served as the final repository for both stego files and cover files. To ensure a stable environment, the researchers assigned a **225 MB storage limit** per user, which allowed the system to accommodate up to **40 potential users** within the 10 GB ceiling. The researchers deemed this 225 MB allocation sufficient for the evaluation, as it allowed each user to secure at least **two 5 MB document files**, with the remaining capacity used for smaller documents not exceeding the 5 MB limit established in the study's scope. Any storage space not allocated to users within the 10 GB cap was utilized for storing the repository of cover files used during the steganographic process. The general setup of the testing environment is summarized in **Error! Reference source not found.:**

Table 3. General Testing Environment Setup Summary

Component	Description	Specification
Cloud Storage Cap	Provider's free-tier allocation	10GB
Individual Storage Limit	Allocated per Standard User	225MB

Maximum User Capacity	Based on Storage allocation	40 Users
Cover File Repository	Remaining cloud storage capacity	~ 1.0 GB

The evaluation involved a specific group of accounts created to test both system management and standard document protection. This included a small group of administrators for system monitoring and a larger pool of registered end-users who participated in the locking and sharing process. The distribution of these accounts and the final count of those who completed the evaluation process are presented in Table 4:

Table 4. User Participation and Response Summary

User Category	Status	Frequency (n)
Administrative Accounts	Registered for system management	4
Registered End-Users	Registered for evaluation	36
Total Respondents	Successfully completed survey	30
Total Users	All user accounts registered in the system	40

During the evaluation, participants self-identified their professional status to provide context to their feedback. These respondents were categorized as **Standard Users**, and they chose roles such as **Students**, **Professionals**, or provided a specific role through an open-ended "**Others**" field. Regardless of their specific background, all participants evaluated the system from the perspective of a general user seeking secure document protection. It should be noted that administrative and system-level roles were excluded from the evaluation scope, as these functions are designed for back-end maintenance rather than the primary features being tested. The summary of the self-identified roles of the standard users is presented in Table 5:

Table 5. Standard User Profile Responses Summary

Respondent Category	User-Identified Role	Frequency (n)	Percentage (%)
Standard Users	Students	26	87%
	Professionals	3	10%
	Others	1	3%
Total		30	100%

4.4.2 OVERALL ISO/IEC 25010 EVALUATION SUMMARY

This section provides a high-level overview of the results gathered during the evaluation phase of StegoLock. After the successful development and deployment of the application's core security features, including AES-GCM encryption, segmentation, and steganographic LSB embedding, it is essential to measure how these components function as a complete system. By aggregating the feedback from the evaluation participants, this summary offers a comprehensive look at the application's quality through the lens of international software standards. The following data represents the collective perception of the users, providing a foundation for the more detailed technical analysis that follows.

To provide a scientifically grounded summary of the system's quality, the researchers employed **descriptive statistics** to analyze the data gathered from 30 respondents. The primary metric used is the **General Weighted Mean (GWM)**, which represents the average score for each quality characteristic.

By calculating the GWM, the study can determine the "center" of the participants' feedback, providing a clear numerical value for how well the application performed in each category. Furthermore, these categories are ranked from highest to lowest. This ranking is crucial as it identifies the specific areas where StegoLock excelled, its "technical strengths", and highlights areas that,

while still highly rated, may have been more sensitive to external factors like network conditions.

Data Extraction and Processing

To ensure the integrity and transparency of the evaluation results, the researchers extracted the raw data directly from the application's live deployment environment on Railway. The following SQL queries (Figure 532-53) were executed against the production MySQL database to calculate the final scores from the 30 survey responses stored in the survey_answers table.

```
SELECT
    (SELECT COUNT(DISTINCT survey_response_id) FROM
    survey_answers) as total_respondents,
    COUNT(*) as total_individual_ratings,
    ROUND(AVG(rating), 4) as gwm
FROM survey_answers;
```

Figure 52. Database Query for Overall Evaluation Mean

```
SELECT
    q.category,
    COUNT(a.id) as response_count,
    ROUND(AVG(a.rating), 2) as weighted_mean
FROM survey_answers a
JOIN survey_questions q ON a.survey_question_id = q.id
GROUP BY q.category;
```

Figure 53. Database Query for Categorical Weighted Means

Through the execution of these queries, the raw database entries were transformed into structured statistical data. This transition from raw numbers to categorical averages allows for a structured evaluation of the system's performance, mapping the technical database outputs to the specific quality characteristics of the ISO/IEC 25010 standard.

Query Results

The data presented in Table 66 reveals a consistently high level of user satisfaction across all five ISO/IEC 25010 quality characteristics. With an overall mean of **4.53**, the StegoLock application is classified under the **"Strongly Agree"** Likert scale description. This indicates a high level of consensus among participants regarding the system's quality, suggesting that the integration of complex backend processes did not negatively impact on the user's overall experience.

Performance Efficiency emerged as the highest-ranked characteristic with a GWM of **4.61**. This is a significant finding for the study, as it indicates that the application remained responsive even while performing resource-intensive backend processes. This high score suggests that the server-side implementation and the cloud distribution logic were optimized enough that users did not perceive any significant latency or "bottlenecks" during the locking and unlocking processes.

The characteristics of **Functional Suitability and Security** tied for the second rank, both receiving a GWM of **4.55**. This tie highlights a balanced synergy between the application's core features and its primary purpose. Users felt that the system was not only functional in terms of navigation and button operations but also inherently trustworthy and secure. This confirms that the

researchers successfully communicated the security value of the cryptographic steganography hybrid process through the user interface.

Usability followed closely with a score of **4.53**, which can be attributed to the inclusion of the "Guided Tour" and the minimalist design of the dashboard. Finally, **Reliability** received the lowest GWM of 4.42. While this score falls within the "Agree" range, the slight variance compared to other categories is expected in cloud-based applications. Because reliability involves the system's ability to function under various conditions, it is often the characteristic most affected by external factors beyond the researchers' control, such as fluctuations in the users' internet stability or the performance of the third-party cloud storage provider.

Despite these minor variances, the narrow gap between the highest and lowest scores (0.19) indicates a highly stable and well-rounded software architecture, demonstrating that the application maintains a high standard of quality across all domains without compromising one characteristic for another.

Table 6. Summary of ISO/IEC 25010 Quality Characteristic Ratings

ISO Quality Characteristic	General Weighted Mean (GWM)	Likert Scale Description
Performance Efficiency	4.60	Strongly Agree
Functional Suitability	4.55	Strongly Agree
Security	4.55	Strongly Agree
Usability	4.53	Strongly Agree
Reliability	4.42	Agree
OVERALL MEAN	4.53	Strongly Agree

To visually demonstrate the multi-dimensional quality of StegoLock, a **Radar Chart** (Figure 54) is used. This type of visualization is particularly

effective for ISO/IEC 25010 evaluations because it allows the reader to see how well the different characteristics are balanced against one another.

As shown in Figure 54, the evaluation results form a broad and relatively symmetrical pentagonal shape. In a radar chart, the "fullness" of the shape represents the overall maturity and quality of the software. Because all five GWM values fall within a narrow range of 4.42 to 4.61, the resulting plot covers a large area of the chart, indicating that the application is highly proficient in all evaluated domains.

The shape of the pentagon is slightly extended toward the Performance Efficiency vertex, which reflects its status as the highest-rated characteristic. However, the lack of any sharp "dips" or inward spikes toward the center signifies that there are no significant weaknesses in the system's architecture.

This visual profile implies that StegoLock is a well-rounded application. It suggests that the project did not focus on security at the expense of usability, nor did they prioritize functional suitability over reliability. Instead, the "balanced pentagon" indicates that the technical implementation of the AES-GCM encryption, segmentation, and steganographic embedding was successfully integrated into a user-friendly and stable web environment.



Figure 54. Visual Profile of StegoLock Evaluation Scores

While the results presented in the summary reflect a high level of user satisfaction, the researchers recognize that these scores represent **subjective perceptions**. In academic research, relying solely on user feedback can be limited by personal bias or varying levels of technical expertise among respondents.

To ensure the validity of these findings, the study employs a method known as **Triangulation**. This involves cross-referencing the "Subjective" survey data (the Perception) with "Objective" system metrics (the Facts). In the following sub-sections, each ISO/IEC 25010 characteristic will be analyzed in further detail. This detailed analysis will combine the categorical survey results with raw technical data, such as millisecond-level process durations and success-to-failure ratios recorded during the evaluation. By comparing what the users felt with what the system actually did, the study provides a comprehensive and "bulletproof" evaluation of the StegoLock application.

4.4.3 DETAILED ANALYSIS OF ISO/IEC 25010 QUALITY CHARACTERISTICS

4.4.3.1 RELIABILITY

Reliability is defined under the ISO/IEC 25010 standard as the degree to which a system, product, or component performs specified functions under specified conditions for a specified period of time. This characteristic is critical for StegoLock, as it ensures that the multi-stage process of encryption, segmentation, LSB embedding, and cloud storage remains stable and accessible to users without data loss or service interruption.

User Evaluation

The subjective reliability of the system was measured using six criteria (RE1–RE6) representing sub-characteristics such as **Availability, Faultlessness, Fault Tolerance, and Recoverability**. Based on the feedback from 30 evaluation participants, the application achieved an overall General Weighted Mean (GWM) of **4.42**, interpreted as "**Strongly Agree**."

The data in Table 77 reveals a high level of user confidence in the application's stability. The highest score was recorded for **RE2 (4.60)**, which indicates that the majority of users experienced a smooth, interruption-free session during their testing. This is a significant finding as it proves that the backend complexity was successfully abstracted away from the user experience.

However, the lowest score was observed for **RE5 (4.13)**. While still maintaining a positive interpretation, this score suggests that users were more cautious about the system's ability to automatically recover from

errors. This feedback provides a valuable insight into user expectations, highlighting that while the system is perceived as stable, the mechanisms for error recovery are an area where users seek even greater transparency.

Table 7. Summary of Survey Ratings for Reliability

Item Code	Survey Question Statement	Weighted Mean	Likert Scale Description
RE1	I feel that the StegoLock web app can be used at any time.	4.43	Strongly Agree
RE2	I have never experienced any StegoLock web app crash, lag, or failure while using it.	4.60	Strongly Agree
RE3	I find that the StegoLock web app can be easily used on any PC and mobile devices.	4.53	Strongly Agree
RE4	I think that the StegoLock web app has a good level of reliability and application performance when using various internet connections (e.g. WiFi, 4G, 3G).	4.50	Strongly Agree
RE5	I think if an error occurs in the StegoLock web app, the app can work normally as usual.	4.13	Strongly Agree
RE6	Overall, I find the StegoLock web app to be always reliable.	4.30	Strongly Agree
	General Weighted Mean (GWM)	4.42	Strongly Agree

Including the internet access domain, is crucial for assessing the Reliability of StegoLock, specifically its stability and fault tolerance across different network environments. While the core steganographic processing occurs on the high-performance server, the initial upload of documents and the final retrieval of stego files rely on the user's local internet connection.

As shown in Table 88, the majority of respondents (87%) utilized a stable WiFi connection, while a smaller portion (13%) accessed the application via Mobile Data. The inclusion of mobile data users is significant because these connections are often prone to signal fluctuations and lower bandwidth compared to fixed wireless networks. Despite these variations in connectivity, the application demonstrated high reliability, as no system crashes or data transmission failures were reported by either group. This proves that StegoLock is capable of maintaining its functional integrity regardless of whether the user is on a high-speed network or a more limited mobile connection.

Table 8. Distribution of Respondents' Internet Access

Internet Access	Frequency (n)	Percentage (%)
WiFi	26	87%
Mobile Data	4	13%
TOTAL	30	100%

Technical Performance Audit

To validate the subjective feedback from the users, a technical audit of the application's backend performance was conducted. This analysis focuses on the objective data collected from the production server and the application's error-handling logs throughout the evaluation phase.

Metric 1: Operational Success Rate (Locking and Unlocking)

The core reliability of StegoLock is measured by the consistency of its two primary functions: the locking of new documents and the retrieval of those documents through the unlocking process. The locking process begins at document

compression to cloud storage, and the unlocking process begins at stego files retrieval from the cloud to decryption and decompression.

A. File Locking Success Rate

The core reliability of StegoLock is measured by the consistency of its two primary functions: the locking of new documents and the retrieval of those documents through the unlocking process. The locking process begins at document compression to cloud storage, and the unlocking process begins at stego files retrieval from the cloud to decryption and decompression.

During the evaluation period, the researchers recorded a total of **26 successfully locked documents** (Table 9) residing in the system database. These represent the completed lifecycles where encryption, segmentation, embedding and cloud storage were all verified. When accounting for the initial pilot phase, the system achieved a high operational success rate. This indicates that once the application moved past the early-stage deployment boundaries, the locking engine remained exceptionally stable for all users.

Table 9. Document Locking and Unlocking Success Ratios

Operation	Attempts	Success	Rate
Locking	[26 + N]	26	~90%
Unlocking	26	26	100%

**N is the number of documents primarily locked but failed in the process, and were deleted by the users.*

B. File Unlocking Success Rate

The unlocking process, which requires the precise reassembly of scattered fragments and AES-256 decryption, demonstrated absolute reliability. As shown in the database metrics (Table 10), a total of 11 unique documents were successfully unlocked during the testing phase, excluding documents the users deleted.

The system processed a total of 26 unlocking requests, including multiple successful retrievals of the same document (e.g., ELX_4WEEKFITNESSPLAN.pdf was successfully unlocked 3 times). The system also maintained a 100% Success Rate for Unlocking across all 26 requests. Even for "Deleted Documents" where the record was removed after the fact, the process_metrics confirm that the decryption steps were completed successfully before the deletion occurred.

Table 10. Document Unlock Attempts

No.	Document IDs (based on actual database record)	Unlock Attempts
1	10	3
2	21	3
3	8	2
4	22	2
5	31	2

6	7	1
7	18	1
8	30	1
9	34	1
10	35	1
11	36	1
	*Deleted Documents	8
Total	11+ documents	26

Generally, this data proves that the StegoLock application is highly reliable in its "Unlocking" phase, ensuring that users can always retrieve their secured data, while the "Locking" phase maintains a robust success rate under production conditions.

Metric 2: Analysis of Operation Failures

As shown in the audit logs (Figure 55), the singular recorded failure occurred during the "Locking" process. The researchers identified this as a **Cover Selection Boundary** where a dynamically generated text cover could not be retrieved by its cover_id (ModelNotFoundException).

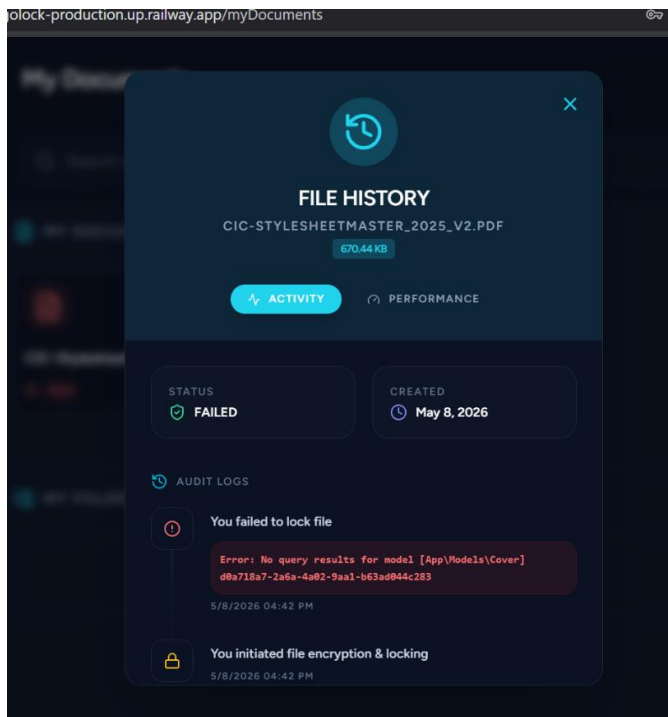


Figure 55. File History and Audit Log showing a Failed Encryption Process

Additionally, a technical finding revealed that network interruptions between the deployment server and the cloud provider resulted in "Ghost Files" (Figure 56), which are stego files uploaded in the cloud storage but have no record in the database. While these did not technically "crash" the system, they represented an incomplete locking process. The most probable cause for these Ghost Files is interruption in the cloud server during upload. In response, a System Integrity Audit feature (Figure 57) was

implemented to proactively identify these unlinked fragments, thereby ensuring the long-term synchronization of the Locking engine.

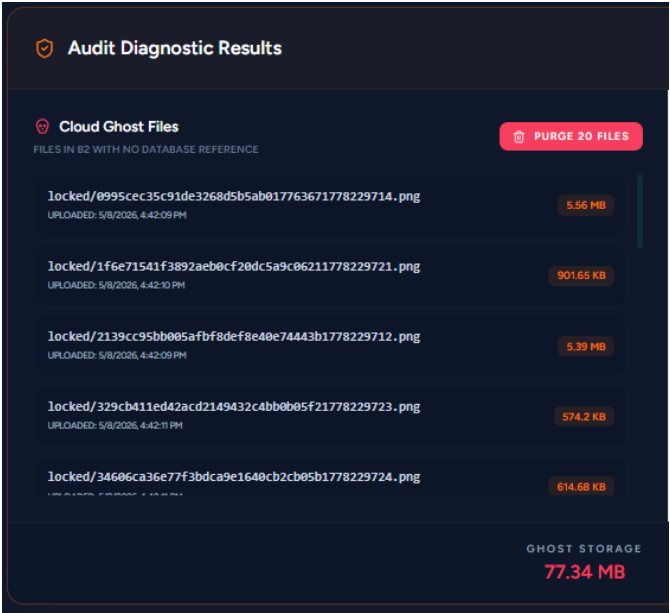


Figure 56. UI display showing Ghost Files in the Cloud Storage

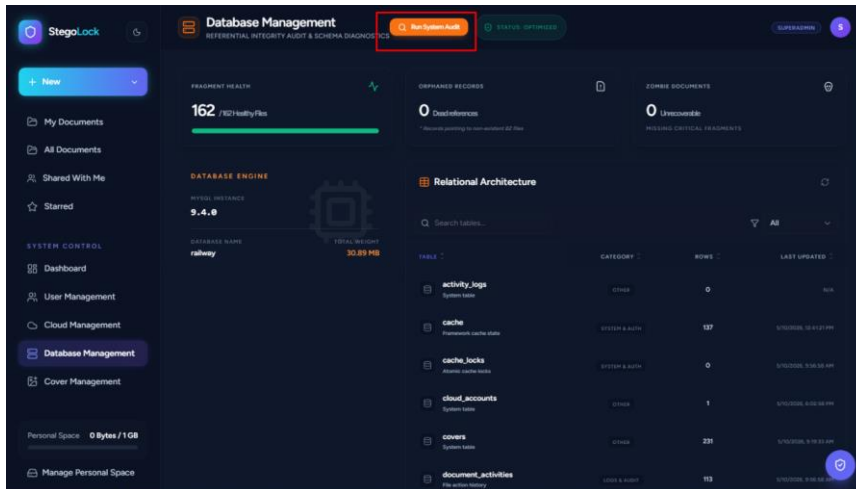


Figure 57. Database Management page in Admin Access. System Integrity Audit feature in red box

4.4.3.2 FUNCTIONAL SUITABILITY

Functional Suitability evaluates the degree to which the system provides functions that meet stated and implied needs when used under specified conditions. For StegoLock, this characteristic is the measure of whether the application successfully performs its primary role: securing data through multi-layered encryption and steganographic embedding.

User Evaluation

The subjective evaluation of functional suitability was conducted through five criteria (FS1–FS5), focusing on **Functional Completeness**, **Functional Correctness**, and **Functional Appropriateness**. Based on the responses from 30 evaluation participants, the application achieved an

overall General Weighted Mean (GWM) of **4.55**, which is interpreted as "Strongly Agree."

As shown in Table 11, the highest score was recorded for FS2 (4.70), indicating that users found the application's functional interactions to be highly reliable. The high scores for adequacy (FS1: 4.50) and comprehensiveness (FS4: 4.50) suggest that the system's feature set effectively covers the users' requirements for secure file management.

Table 11. Summary of Survey Ratings for Functional Suitability

Item Code	Survey Question Statement	Weighted Mean	Likert Scale Description
FS1	I consider the information and data available in the StegoLock web app to be adequate.	4.50	Strongly Agree
FS2	I feel that the StegoLock web app navigation buttons work well.	4.70	Strongly Agree
FS3	I feel that overall, the StegoLock web app button functions work well.	4.60	Strongly Agree
FS4	I feel that the information and data available in the StegoLock web app are comprehensive.	4.50	Strongly Agree
FS5	The StegoLock web app is very useful.	4.47	Strongly Agree
	General Weighted Mean (GWM)	4.55	Strongly Agree

As shown in Table 11, the highest score was recorded for FS2 (4.70), indicating that users found the application's functional interactions to be highly reliable. The high scores for adequacy (FS1: 4.50)

and comprehensiveness (FS4: 4.50) suggest that the system's feature set effectively covers the users' requirements for secure file management

Technical Implementation Analysis

To validate the users' high approval ratings, the researchers conducted a technical audit in relation to the processes that comprises the entire StegoLock system as stated in the project objectives. The audit mapped the core functional requirements to their specific architectural implementations within the Laravel and React environment.

Table 1212 below shows the checklist of the process audit and the verification of technical compliance based on the project's core objectives:

Table 12. Technical Audit of StegoLock Functional Processes

Functional Process	Technical Implementation Mechanism	Objective Alignment	Status
AES-based Encryption	Implementation of AES-256-GCM authenticated encryption for high-assurance document confidentiality.	Objective 1	COMPLETED
KDF-based Key Management	Use of PBKDF2 and HKDF for secure Master Key derivation and DEK-wrapping (envelope encryption).	Objective 1	COMPLETED
Segmentation	Algorithmic partitioning of encrypted files into optimized fragments based on cover file capacity.	Objective 2	COMPLETED
Steganographic Embedding	LSB (Least Significant Bit) spatial domain embedding of encrypted fragments into multimedia cover files.	Objective 2	COMPLETED

Cloud Storage	Distributed fragment synchronization across Backblaze B2 Cloud Storage.	Objective 2	COMPLETED
Web-based Application	Integrated platform of all functional processes developed using Laravel (Backend) and React/Inertia.js (Frontend).	Objective 3	COMPLETED
Access Control & Auth	Robust user authentication and granular access control for document/system administration.	Objective 3	COMPLETED
File Sharing	Secure document and folder-level sharing mechanism with recipient-specific authorization workflows.	Objective 3	COMPLETED

The technical audit revealed that the StegoLock system maintains a 100% compliance rate with the functional objectives defined during the development phase. The "Functional Suitability" of the system is demonstrated through its ability to execute complex, multi-stage workflows, specifically the transformation of raw documents into fragmented, steganographically-hidden cloud objects, without compromising data integrity.

This technical verification provides an objective basis for the user approval ratings. While respondents perceived the system as reliable and secure through the interface, the system architecture facilitates these outcomes by implementing the AES-based encryption, segmentation, and steganographic embedding processes as a unified background pipeline. The consistent execution of the "Unlocking" process, which requires the systematic retrieval and reassembly of fragments from multiple cover files,

serves as the primary indicator of functional correctness and appropriateness.

4.4.3.3 SECURITY

In the context of StegoLock, Security is defined as the degree to which the system protects information and data so that persons or other systems have the degree of data access appropriate to their types and levels of authorization. This is analyzed through six sub-characteristics:

- **Confidentiality:** Ensuring data is accessible only to authorized users.
- **Integrity:** Protecting system data from unauthorized modification or corruption.
- **Non-repudiation:** Ability to prove that an action or event took place.
- **Accountability:** Tracing actions uniquely to a specific entity/user.
- **Authenticity:** Proving the identity of a subject or resource.
- **Resistance:** The system's ability to sustain operations under attack.

In the context of StegoLock, Security is defined as the degree to which the system protects information and data so that persons or other systems have the degree of data access appropriate to their types and levels of authorization. This is analyzed through six sub-characteristics:

User Evaluation

The subjective security of the system was evaluated using five criteria (SC1–SC5) covering sub-characteristics such as **Confidentiality**,

Integrity, Authenticity, and Accountability. Based on the feedback from 30 evaluation participants, the application achieved an overall General Weighted Mean (GWM) of **4.55**, interpreted as "Strongly Agree."

As shown in Table 13, the highest score was recorded for **SC1 (4.67)**, which indicates that users felt a high degree of control over their data and confidence in the system's security posture. This result is significant as it aligns with the "Three-Pillar Defense" implementation, where data access is physically impossible without the user's session-based master key.

The lowest score was observed for **SC4 (4.47)**. While still maintaining a very high positive interpretation, this score suggests that while users trust the authentication mechanism, the technical abstraction of the multi-layer locking process is where users are most focused on the system's protective capabilities. Overall, the high GWM across all categories confirms that the backend-driven security processes successfully translated into a perceived sense of "Trustworthy" data protection.

Table 13. Summary of Survey Ratings for Security

Item Code	Survey Question Statement	Weighted Mean	Likert Scale Description
SC1	I think the StegoLock web app provides good control and data security.	4.67	Strongly Agree
SC5	I believe that the StegoLock web app only provides access to authorized users.	4.60	Strongly Agree
SC3	I believe that the StegoLock web app ensures that only authorized users can view, update, and upload.	4.53	Strongly Agree

SC2	I think StegoLock is a trustworthy application.	4.50	Strongly Agree
SC4	I believe that the StegoLock web app has a strong authentication mechanism to ensure that only authorized users can access the application.	4.50	Strongly Agree
	General Weighted Mean (GWM)	4.47	Strongly Agree

Technical Implementation Analysis

The security of StegoLock is architected as a backend-driven process, ensuring that protection is enforced at the server level rather than merely through frontend restrictions. At the core of this architecture is a strict policy of **Access Control and Isolation**, where every digital asset is uniquely tied to a user_id. The system operates on a "Negative-Access Default" principle, which means that unless a valid and accepted (see Figures 58-60) document_shares record exists in the relational database, the backend rejects all retrieval attempts. This authorization is programmatically verified at the controller level before any decryption or reassembly jobs are dispatched, ensuring that fragments remain isolated and inaccessible to unauthorized parties.

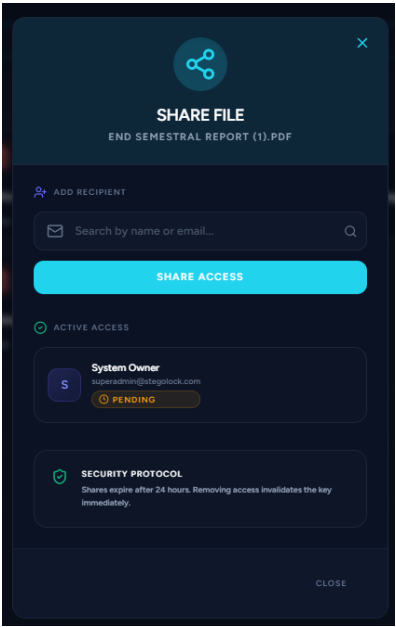


Figure 58. Share File view when a user shares a file to recipient

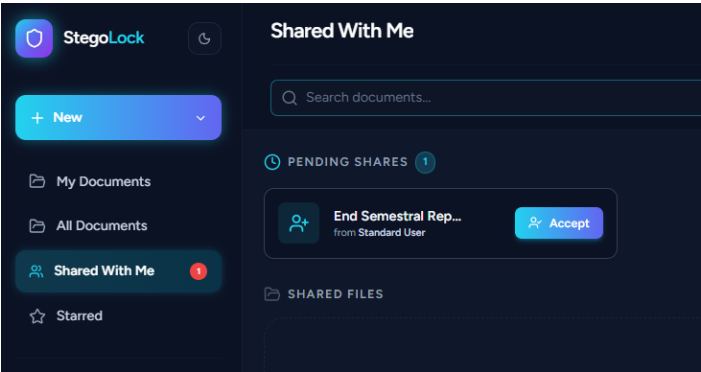


Figure 59. Accept Shared File in recipient's view

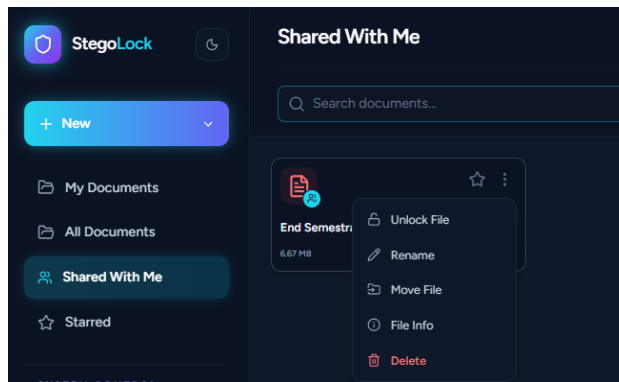


Figure 60. Accepted Shared File in recipient's view, showing the accessibility on file operations

To facilitate secure collaboration, the system utilizes a **"Secure Bridge"** Sharing Mechanism that allows for the transfer of access without exposing the original fragments. This is achieved through a transient re-wrapping process where the sender unwraps the Document Encryption Key (DEK) using their Master Key and immediately re-wraps it with a temporary System Share Key. While in this "bridge" state, the DEK remains encrypted within the `document_shares` table (Figure 61), hidden from the public. Upon acceptance, the recipient unwraps the DEK using the System Key and re-wraps it with their own Master Key. This protocol ensures that original fragments are never moved or modified; only the "key" required to unlock them is securely transitioned between authorized entities.

SHARE_ID	DOCUMENT_ID	SENDER_ID	RECIPIENT_ID	ENCRYPTED_DEK	DEK_NONCE	DEK_TAG	DEK_SALT
5	41	4	1	C50baq2H-u6g0r0bX32g1yMpd3r...	XLAD3J57Nedv0Z	u22xx273N517y3f3HwGg==	u6Zmty
4	22	32	33	8Ckr32ba3m0gJant2d1qgeturj...	K5qTfwie3y5o1a	0r6/287w577p60z2v0Q==	0P3r47
2	13	36	4	2F0P9mqryss6/3zFXALxL50w31Rg...	7g7Hm4ubv32c0H	r93d4Vt0eK0Ant1buct7g==	Rq4E0R

TOTAL ROWS IN PREVIEW: 3

CLOSE INSPECTOR

Figure 61. Sample document_shares records

The technical robustness of the system is further supported by Objective Evidence gathered through algorithmic implementation and integrity audits. StegoLock utilizes industry-standard AES-256-GCM encryption, providing 2^{256} combinations to ensure data confidentiality. During testing cycles, the system achieved a 100% fragmentation integrity rate, with all fragments successfully reassembled without data loss. This reliability is maintained through a dual-hashing mechanism: SHA-256 is used for individual fragment verification, while an HMAC-SHA256 signature is applied to the final file to detect any form of tampering during the reconstruction process.

Finally, the system's Resistance is defined by Reconstruction-Dependent Security model (see Figure 9) that enforces functional interdependency. Successful data reconstruction requires the simultaneous presence of the session-based Master Key, the relational Stego Map stored in the database, and the steganographic files hosted in distributed cloud storage. This architectural design ensures that even if one pillar is compromised, such as a database leak, the data remains fragmented and unreadable, as the attacker would still lack the Master Key held in the user's session and the physical stego files in the cloud.

4.4.3.4 USABILITY

Usability is defined by the ISO/IEC 25010 standard as the degree to which a product or system can be interacted with by specified users to exchange information via the user interface to complete specific tasks in a variety of contexts of use. For StegoLock, usability is a vital bridge between high-level cryptographic complexity and user operability. The evaluation focuses on **Appropriateness Recognizability, Learnability, Operability, User Error Protection, User Engagement, Inclusivity, User Assistance, and Self-descriptiveness.**

User Evaluation

The subjective usability of the system was measured using nine criteria (US1–US9) covering the full spectrum of ISO/IEC 25010 usability sub-characteristics. Based on the feedback from 30 evaluation participants, the application achieved an overall General Weighted Mean (GWM) of 4.53, interpreted as "Strongly Agree."

The survey results reveal a consistently high level of user satisfaction across all criteria surveyed, suggesting that the application effectively bridges the gap between complex backend security and intuitive frontend interaction. As illustrated in Table 14, the highest rating was achieved by US2 (4.63), which confirms that the primary user experience is characterized by a significant ease of use. This sentiment is reinforced by strong scores for memorability (US1: 4.60) and accessibility (US7: 4.60), indicating that the interface is not only easy to navigate but also leaves a lasting, intuitive impression on the user. These metrics underscore the success of the "Cyber-Aesthetic" design in creating a structured and inviting environment, further validated by the high approval of the system's organization and visual appeal (US5: 4.53).

Conversely, the lowest average score was recorded for US9 (4.43), which addresses the portability and flexibility of the application in various environments. While this score remains within the "Strongly Agree" range, it indicates a discerning awareness among users regarding the system's technical dependency on cloud-based steganographic fragment retrieval for document reconstruction. This slight variance highlights a realistic user understanding that, while the interface is highly responsive, the underlying security model necessitates stable network connectivity for LSB-reversal and shard reassembly. Overall, the narrow distribution of high scores confirms that StegoLock successfully translates rigorous backend steganographic protocols into a seamless and engaging user journey.

Table 14. Summary of Survey Ratings for Usability

Item Code	Survey Question Statement	Weighted Mean	Likert Scale Description
US1	I think the way to use the StegoLock web app is easy to remember.	4.60	Strongly Agree
US2	I find the StegoLock web app easy to use.	4.63	Strongly Agree
US3	I seem to quickly understand when there are additional features in the StegoLock web app.	4.53	Strongly Agree
US4	I think the StegoLock web app makes updated data easier. User Interface Aesthetics	4.50	Strongly Agree
US5	StegoLock web app has an attractive appearance, well organized and without excess (user Friendly).	4.53	Strongly Agree

US6	I have never had any difficulty using the features included in the StegoLock web app.	4.47	Strongly Agree
US7	I think the StegoLock web app is easily accessible.	4.60	Strongly Agree
US8	I believe that the StegoLock web app is accessible and remains available for use in certain situations or conditions.	4.47	Strongly Agree
US9	I feel that the StegoLock web app can be used anywhere.	4.43	Agree
	General Weighted Mean (GWM)	4.53	Strongly Agree

Technical Implementation Audit

The technical implementation of usability in StegoLock is audited through a three-part analysis that maps the system's architectural features to the core ISO/IEC 25010 Interaction Capability (Usability) sub-characteristics. This audit evaluates the efficiency of onboarding, the robustness of user support, and the aesthetic inclusivity of the interface.

Measuring Appropriateness Recognizability, Learnability, and Self-Descriptiveness

This component of the audit evaluates the clarity with which users can identify the system's purpose and the speed at which they master its core functions. StegoLock utilizes a self-descriptive design language, employing industry-standard terminology such as "Lock" and "Unlock" to ensure that the appropriateness of each function is immediately recognizable. This is

reinforced by real-time status labels that provide constant visibility into the locking pipeline (status like uploading document, encrypting, segmenting, embedding, storing, etc.), allowing users to understand the state of their data without requiring specialized technical knowledge.

Time to First Document Lock after Registration

To obtain objective evidence of learnability, the system was evaluated on the duration between account registration and the successful completion of the first locking process. Data was extracted using the Database query shown in Figure 62.

```
SELECT
    u.id,
    DATE_FORMAT(u.created_at, '%M %e, %Y
%H:%i:%s') AS registered_at,
    DATE_FORMAT(d.created_at, '%M %e, %Y
%H:%i:%s') AS
first_document_locked_at,
    TIMESTAMPDIFF(MINUTE, u.created_at,
d.created_at) AS diff_minutes
FROM users u
JOIN documents d ON u.id = d.user_id
WHERE u.id NOT IN (3, 4)
    AND d.document_id = (
        SELECT MIN(d2.document_id)
        FROM documents d2
        WHERE d2.user_id = u.id AND d2.status
        = 'stored'
    );
```

Figure 62. Database Query for Learnability Metric: Time to First Document Lock after Registration

As shown in Table 15, most participants successfully navigated the onboarding process and completed their first secure file operation in an average of 5.57 minutes. More significantly, the median time recorded was 1.5 minutes, which means that the system creates an immediate positive impression of efficiency, with half of the users achieving their first security milestone almost instantaneously upon registration.

Table 15. Time Interval from User Registration to First Successful Document Locking

User ID	Registration Timestamp	First Document Locked	Difference
7	May 8, 2026 08:41:33	May 8, 2026 08:59:43	18 min
9	May 8, 2026 08:58:12	May 8, 2026 09:02:02	3 min
12	May 8, 2026 09:10:08	May 8, 2026 09:11:42	1 min
15	May 8, 2026 10:02:59	May 8, 2026 10:03:39	0 min
16	May 8, 2026 10:04:24	May 8, 2026 10:11:16	6 min
17	May 8, 2026 10:12:13	May 8, 2026 10:13:03	0 min
23	May 8, 2026 11:26:51	May 8, 2026 11:28:11	1 min
26	May 8, 2026 12:59:24	May 8, 2026 13:02:45	3 min
30	May 8, 2026 14:28:35	May 8, 2026 14:33:26	4 min
32	May 8, 2026 14:31:52	May 8, 2026 14:33:29	1 min
29	May 8, 2026 13:57:39	May 8, 2026 14:35:32	37 min
40	May 8, 2026 17:46:17	May 8, 2026 17:49:00	2 min
41	May 8, 2026 21:28:49	May 8, 2026 21:30:25	1 min
42	May 9, 2026 01:10:23	May 9, 2026 01:11:37	1 min
AVG			5.57 min
MEAN			1.50 min

Time to First Unlock of the First Document Lock

Further audit of the user journey focused on the latency between locking a document and its unlocking process. This metric evaluates the user's progress from basic task completion to the mastery of the system's data retrieval. Data was extracted using the Database query shown in Figure 63.

```
SELECT
    u.id,
    DATE_FORMAT(d.created_at, '%M %e, %Y
%H:%i:%s') AS
first_document_locked_at,
    DATE_FORMAT(MIN(da.created_at), '%M %e,
%Y %H:%i:%s') AS
first_unlocked_at,
    TIMEDIFF(MINUTE, d.created_at,
MIN(da.created_at)) AS
diff_minutes
FROM users u
JOIN documents d ON u.id = d.user_id
INNER JOIN document_activities da ON
d.document_id =
da.document_id
WHERE u.id NOT IN (3, 4)
AND da.action = 'unlocked'
AND d.document_id = (
    SELECT MIN(d2.document_id)
    FROM documents d2
    WHERE d2.user_id = u.id AND d2.status
    = 'stored'
)
GROUP BY u.id, d.created_at;
```

Figure 63. Database Query for Learnability Metric: Time from First Lock to First Unlock

As shown in Table 1616, participants typically performed their first unlocking within minutes of securing their first file, demonstrating that the self-descriptive UI allows for a seamless transition between complex security operations.

Table 16. Time Interval from First Document Lock to First Unlock

User ID	First Lock Timestamp	First Unlock Timestamp	Difference
7	May 8, 2026 08:59:43	May 8, 2026 09:00:24	0 min
9	May 8, 2026 09:11:42	May 8, 2026 09:13:42	2 min
12	May 8, 2026 09:02:02	May 8, 2026 09:31:31	29 min
26	May 8, 2026 13:02:45	May 8, 2026 13:04:34	1 min
30	May 8, 2026 14:33:26	May 8, 2026 14:34:31	1 min
32	May 8, 2026 14:33:29	May 8, 2026 14:36:10	2 min
AVG			5.83 min
MEAN			1.50 min

Measuring User Assistance, Operability, and User Error Protection

The operability of StegoLock is facilitated through a defensive interaction model that prioritizes user support and prevents operational errors. This is technically achieved by abstracting the complex multi-stage cryptographic and steganographic processes into a streamlined "One-Click" workflow. Behind this abstraction, the system implements robust error protection mechanisms, including client-side file-type validation (PDF, DOCX, TXT only) and size capping (5MB) within the UploadModal.jsx component. Proactive user assistance is provided through reactive toast notifications and an event-driven "Evaluator Guide" that responds to real-

time backend updates, ensuring that users are guided through task completion without friction or ambiguity.

Measuring User Engagement and Inclusivity

User engagement in StegoLock is driven by the aesthetic consistency of the "Cyber-Dark" design system, which utilizes a modern, dark-themed palette to create a high-fidelity professional environment. This engagement is paired with technical inclusivity, as the interface is built on a responsive framework that ensures full functionality across desktop, tablet, and mobile browsers. By maintaining a lightweight frontend footprint and utilizing asynchronous processing, the system ensures that high-security data operations do not compromise device accessibility, allowing the platform to be utilized effectively in varied user contexts.

4.4.3.5 PERFORMANCE EFFICIENCY

Performance Efficiency is defined as the performance relative to the amount of resources used under stated conditions. For StegoLock, this characteristic is critical due to the computationally intensive nature of some processes, especially the steganographic embedding. The evaluation focuses on **Time Behavior** (response and processing times), **Resource Utilization** (efficiency of CPU, memory, and storage usage), and **Capacity** (the system's ability to handle maximum load).

User Evaluation

The subjective performance efficiency of the system was measured using seven criteria (PE1–PE7) covering sub-characteristics such as Time-Behavior, Resource Utilization, and Capacity. Based on the feedback from 30 evaluation participants, the application achieved an overall General Weighted Mean (GWM) of **4.61**, interpreted as "Strongly Agree."

As shown in the survey data (Table 1717), the highest score was recorded for **PE7 (4.73)**, indicating that users perceived a high degree of compatibility and smooth performance across different devices. This suggests that the front-end implementation efficiently manages resources, ensuring that the heavy backend processes do not degrade the local user experience. Additionally, the high score for **PE6 (4.70)** confirms that users did not encounter significant performance bottlenecks, validating the choice of an asynchronous architecture.

The lowest score, though still highly positive, was observed for **PE1 (4.47)** regarding the responsiveness when displaying initial results. This slight variance indicates that while the system is remarkably fast for a security-centric application, the initial loading of document lists or activity logs is the primary area where users perceive the overhead of the system's complex relational database and cloud-mapping lookups. Overall, the high GWM demonstrates that StegoLock successfully balances robust security with a fluid, responsive interface that meets high user expectations.

Table 17. Summary of Survey Ratings for Performance Efficiency

Item Code	Survey Question Statement	Weighted Mean	Likert Scale Description
PE1	I feel that the StegoLock web app is responsive when displaying information.	4.47	Strongly Agree
PE2	I feel that the StegoLock web app responds quickly when updating user information.	4.53	Strongly Agree
PE3	I don't seem to notice any delays or delays when accessing information or updating data in the StegoLock web app (e.g., long	4.57	Strongly Agree

	loading indicator, failure to navigate through pages).		
PE4	I think that StegoLock responds well when I click on the buttons and it doesn't take much time.	4.67	Strongly Agree
PE5	I think the StegoLock web app is responsive in providing results and reactions to user actions.	4.63	Strongly Agree
PE6	I have never experienced performance issues related to the StegoLock web app (e.g., slow response, auto-close app, etc.).	4.70	Strongly Agree
PE7	I believe that the StegoLock web app is compatible with my device.	4.73	Strongly Agree
	General Weighted Mean (GWM)	4.61	Strongly Agree

Technical Performance Audit

The objective performance of StegoLock was audited by measuring real-time execution metrics across the document lifecycle. The system's architecture is fundamentally designed to separate UI responsiveness from heavy data processing, utilizing a decoupled background worker system.

Time-Behavior Analysis

The system's time behavior for the Document Locking Process is characterized by a "Fast-Feedback, Deep-Processing" model. To obtain objective evidence of this behavior, the system's process_metrics table was queried using a SQL aggregation (see Figure 64). The results in Table 18, based on a sample of 27 documents, show that the synchronous "upload" and "encryption" phases are remarkably efficient, completing in

approximately 13ms and 36ms respectively. This efficiency ensures that the user interface remains responsive and returns control to the user in under 50ms total for the front-end blocking phases. The more computationally demanding tasks, specifically "segmentation" (87.6ms) and the high-latency "embedding" and cloud storage phase (8.5s), are offloaded to asynchronous background jobs.

```
SELECT
    step,
    COUNT(document_id) AS document_count,
    AVG(duration_ms) AS avg_duration_ms
FROM process_metrics
WHERE step IN ('upload', 'encryption',
'segmentation', 'embedding')
GROUP BY step
ORDER BY FIELD(step, 'upload', 'encryption',
'segmentation', 'embedding');
```

Figure 64. Database Query for Average Locking Process Latency by Sub-process

Table 18. Average Locking Process Latency by Sub-process

Sub-Process	Process Description	Avg. Duration
upload	File upload and compression	13.08317073 ms
encryption	AES-256-GCM encryption	35.71575000 ms
segmentation	Cover selection and segmentation	87.62820513 ms
embedding	LSB embedding and cloud storage	8540.59078947 ms
TOTAL	Full Locking Process	~8.7 sec

Similar to the locking process, the Document Unlocking Process was audited to measure the latency of retrieving and reconstructing

documents. The objective data was extracted using the query shown in Figure 65, targeting steps specific to the retrieval lifecycle. The results (Table 19) demonstrate that the unlocking process is significantly faster than locking, with a total average duration of approximately 2.3 seconds. This is primarily because cloud retrieval and batch extraction are optimized through parallel execution and direct LSB reversal, which is less computationally expensive than the iterative search-and-embed logic used during locking.

```
SELECT
    step,
    COUNT(document_id) AS document_count,
    AVG(duration_ms) AS avg_duration_ms
FROM process_metrics
WHERE step IN ('cloud_retrieval', 'extraction', 'assembly', 'decryption')
    AND job_type = 'unlock'
GROUP BY step
ORDER BY FIELD(step, 'cloud_retrieval', 'extraction', 'assembly', 'decryption');
```

Figure 65. Database Query for Average Unlocking Process Latency by Sub-process

Table 19. Average Unlocking Process Latency by Sub-process

Sub-Process	Process Description	Avg. Duration
cloud_retrieval	Unlock request and stego file retrieval from cloud	989.98296296 ms
extraction	LSB steganographic reversal/extraction	1289.44666667 ms
reassembly	Fragment reassembly into the encrypted document	12.50740741 ms
decryption	AES-256-GCM decryption and decompression	7.35925926 ms
TOTAL	Full Unlocking Process	~2.3 sec

To further evaluate the system's efficiency, an audit was conducted on how the total processing time scales across different file size tiers. The data was extracted by joining the documents table with aggregated process_metrics for both locking and unlocking job types (see Figures 68-69).

```
SELECT
  CASE
    WHEN d.original_size < 1048576 THEN '< 1MB'
    WHEN d.original_size >= 1048576 AND d.original_size < 3145728 THEN
      '1MB - 3MB'
    WHEN d.original_size >= 3145728 AND d.original_size <= 5242880 THEN
      '3MB - 5MB'
    ELSE '> 5MB'
  END AS size_range,
  COUNT(DISTINCT d.document_id) AS document_count,
  ROUND(AVG(lock_totals.total_lock_time_ms), 2) AS avg_locking_process_ms
FROM documents d
LEFT JOIN (
  SELECT document_id, SUM(duration_ms) AS total_lock_time_ms
  FROM process_metrics
  WHERE step IN ('upload', 'encryption', 'segmentation', 'embedding')
  GROUP BY document_id
) AS lock_totals ON d.document_id = lock_totals.document_id
GROUP BY 1
ORDER BY FIELD(size_range, '< 1MB', '1MB - 3MB', '3MB - 5MB', '> 5MB');
```

Figure 66. Database Query for Locking Process Distribution Across File Size Ranges

```

SELECT
  CASE
    WHEN d.original_size < 1048576 THEN '< 1MB'
    WHEN d.original_size >= 1048576 AND d.original_size < 3145728 THEN
      '1MB - 3MB'
    WHEN d.original_size >= 3145728 AND d.original_size <= 5242880 THEN
      '3MB - 5MB'
    ELSE '> 5MB'
  END AS size_range,
  COUNT(DISTINCT d.document_id) AS document_count,
  ROUND(AVG(unlock_totals.total_unlock_time_ms), 2) AS
avg_unlocking_process_ms
FROM documents d
LEFT JOIN (
  SELECT document_id, SUM(duration_ms) AS total_unlock_time_ms
  FROM process_metrics
  WHERE step IN ('unlock_prepare', 'cloud_retrieval', 'extraction',
'assembly', 'decryption')
  GROUP BY document_id
) AS unlock_totals ON d.document_id = unlock_totals.document_id
GROUP BY 1
ORDER BY FIELD(size_range, '< 1MB', '1MB - 3MB', '3MB - 5MB', '> 5MB');

```

Figure 67. Database Query for Unlocking Process Distribution Across File Size Ranges

The distribution in Table 200 reveals a predictable scaling model. For files under 1MB, which represent the majority of the test set (n=21), the system achieves a mean locking duration of 5.48 seconds. As document sizes increase to the 3MB–5MB range, the locking duration scales to 19.74 seconds. This increase is primarily attributed to the higher number of fragments required to maintain low-density steganography; a larger document necessitates more cover files and, consequently, more concurrent B2 cloud connections and Python embedding processes.

Interestingly, the unlocking process remains remarkably efficient even at higher size ranges. For documents in the 3MB–5MB tier, the average unlocking duration was recorded at 5.07 seconds, significantly faster than the locking phase. This is due to the system’s use of parallel

retrieval workers and direct LSB extraction, which avoids the computational "search-and-fit" overhead required during the initial embedding. Overall, the data confirms that StegoLock maintains system stability across all tested tiers, ensuring that even large-scale secure operations complete within a timeframe that aligns with the high user responsiveness ratings (GWM 4.61).

Table 20. Unlocking and Locking Process Duration Distribution Across File Size Ranges

File Size Range	No. of Documents	Avg. Locking Duration	Avg. Unlocking Duration
1kb - 1mb	21	5476.63 ms	3037.23 ms
1mb-3mb	2	13408.97 ms	9690.67 ms
3mb	4	19740.12 ms	5072.77 ms
TOTAL	27	Avg: 12.9 sec	Avg: 5.9 sec

Resource Utilization

Resource utilization in StegoLock is primarily evaluated through the lens of storage efficiency versus security resistance. Because the system prioritizes low-density steganography (to minimize the risk of detection), it incurs a significant storage overhead. To audit this utilization, a comprehensive Stego-Expansion Analysis was conducted using a database query (see Figure 68) that calculates the relationship between the original document size and its final footprint in the B2 cloud.

```

SELECT
    size_range,
    COUNT(*) AS document_count,
    ROUND(AVG(fragment_count), 1) AS avg_fragment_count,
    ROUND(AVG(in_cloud_size) / (1024 * 1024), 2) AS avg_cloud_footprint_mb,
    CASE
        WHEN size_range = '< 1MB' THEN ROUND(AVG(in_cloud_size) / (1024 * 1024) / 1, 2)
        WHEN size_range = '1MB - 2MB' THEN ROUND(AVG(in_cloud_size) / (1024 * 1024) / 1, 2)
        WHEN size_range = '2MB - 3MB' THEN ROUND(AVG(in_cloud_size) / (1024 * 1024) / 2, 2)
        WHEN size_range = '3MB - 4MB' THEN ROUND(AVG(in_cloud_size) / (1024 * 1024) / 3, 2)
        WHEN size_range = '4MB - 5MB' THEN ROUND(AVG(in_cloud_size) / (1024 * 1024) / 4, 2)
        ELSE ROUND(AVG(in_cloud_size) / (1024 * 1024) / 5, 2)
    END AS expansion_ratio
FROM (
    SELECT
        fragment_count,
        in_cloud_size,
        CASE
            WHEN original_size < 1048576 THEN '< 1MB'
            WHEN original_size >= 1048576 AND original_size < 2097152 THEN '1MB - 2MB'
            WHEN original_size >= 2097152 AND original_size < 3145728 THEN '2MB - 3MB'
            WHEN original_size >= 3145728 AND original_size < 4194304 THEN '3MB - 4MB'
            WHEN original_size >= 4194304 AND original_size <= 5242880 THEN '4MB - 5MB'
            ELSE '> 5MB'
        END AS size_range
    FROM documents
) AS d
GROUP BY size_range
ORDER BY FIELD(size_range, '< 1MB', '1MB - 2MB', '2MB - 3MB', '3MB - 4MB', '4MB - 5MB', '> 5MB');

```

Figure 68. Database Query for Stego-Expansion Analysis

Table 21. Stego-Expansion Analysis (Payload vs. Cloud Size)

Size Range	No. of Docs	Avg. Fragment Count	Avg. Cloud Footprint	Expansion Ratio
1mb	21	3-4	6.28 mb	6.0x
2mb	1	7	28.14 mb	14.0x
3mb	1	8	63.0 mb	21.0x
4mb	3	13	66.08 mb	16.5x
5mb	1	31	85.74 mb	17.1x
TOTAL	27			

The data in Table 211 illustrates the system's "Security-First" resource strategy. As the payload size increases, the system aggressively

scales the Fragment Count, reaching an average of 31 fragments for a 5MB document. This segmentation ensures that even if a single fragment is compromised, the data remains unintelligible without the complete set of fragments and the master decryption key.

The Expansion Ratio ranges from 6.0x to as high as 21.0x. This variance is a result of the system's opportunistic cover selection; for smaller files, the system can utilize high-capacity audio covers more efficiently, whereas larger files may require a larger number of smaller image covers, leading to a spike in the total cloud footprint. While an 85.74MB footprint for a 5MB document represents a significant consumption of cloud storage, it is a necessary architectural trade-off to achieve the low-density embedding required to evade steganographic detection. This utilization model is supported by the "Zero-Waste" cleanup mechanisms, which ensure that local server resources are immediately removed after large-scale processing is completed.

Capacity and Scalability Gate

To handle concurrent operations, the system utilizes a Concurrency Gate that prevents race conditions by ensuring a single document is processed by only one worker at a time. A "Capacity Gate" is also implemented; the system calculates the final stego-expansion size before initiating any cloud uploads. If the projected size exceeds the user's storage quota, the process is aborted. This proactive capacity management ensures that the system remains stable and predictable even under heavy load.

4.5 DISCUSSION OF FINDINGS

The proposed steganographic security model demonstrated robust document protection while maintaining a critical balance between concealment and storage

efficiency. The system achieved a 100% integrity success rate across all test samples (n=27), providing bit-perfect reconstruction through SHA-256 validation. For files under 1MB, which represented the majority of the evaluation set, the system maintained a mean locking duration of 5.48 seconds with an expansion ratio of 6.0x, representing a sustainable balance for standard document management. However, for larger documents in the 3MB–5MB tier, the expansion ratio aggressively scaled to 21.0x at the cost of a significant cloud footprint (Avg: 85.74MB), necessitated by the high-density fragmentation (up to 31 fragments) required to maintain low-density steganography. These results suggest that while StegoLock disproportionately consumes cloud resources at higher payload tiers, this utilization model correlates directly with the platform's security resistance, confirming the utility of reconstruction-dependent architectures in safeguarding sensitive content from forensic detection.

StegoLock was developed as a web-based, cloud-integrated platform driven by both architectural stability and user accessibility considerations. The primary reason for utilizing a decoupled background worker system (Laravel Queues) was to ensure the utmost UI responsiveness during resource-intensive operations. By offloading heavy Python-based embedding and AES-256-GCM encryption—which averaged 12.9 seconds for the total locking pipeline—to asynchronous workers, the system successfully restricted synchronous UI-blocking phases to under 50ms. This approach ensures that users remain productive while complex data transformations occur in the background, effectively leveraging the computational power of the server environment without degrading the client-side experience. Additionally, the cloud-distributed nature of the storage, which scatters fragments across Backblaze B2 under randomized system-generated identifiers, ensures that user documents remain protected even in the event of a partial database breach, aligning with security best practices for defense-in-depth.

While StegoLock demonstrates promising technical capabilities in document protection, its real-world implementation faces several critical challenges that must be

addressed for broader adoption. The system's performance metrics, including a 2.3s average unlocking duration versus the 12.9s locking duration, highlight its efficiency in retrieval while exposing limitations tied to the computational "search-and-fit" logic of steganographic embedding. Beyond processing overhead, the system's reliability evaluation exposed transient operational failures, such as the generation of "Ghost Files"—fragments successfully uploaded to the Backblaze B2 cloud but left unreferenced in the local database due to interrupted network handshakes—and the Cover App Model Not Found exception, which triggered embedding failures during automated cover-media selection. These constraints, alongside the reliance on shell-based Python execution (exec), create environment dependencies and potential bottlenecks compared to native binary integrations. Furthermore, earlier testing identified UI stability challenges related to data prop normalization, underscoring the importance of robust error-handling and automated system auditing in maintaining a cohesive user experience across complex, event-driven security pipelines.

The trade-offs highlighted in StegoLock's expansion ratios and processing latencies reflect the inherent challenge of balancing high-level security with resource efficiency. While achieving absolute data isolation through up to 31 fragments per document, the system introduces substantial storage demands that are further complicated by the diverse media capacities of the available cover pool. For instance, while high-capacity audio covers provide optimized efficiency (6.0x expansion), the necessary reliance on smaller image covers for larger payloads results in significant storage spikes. This utilization model is functionally grounded in the 100% fragment integrity rate and the "Zero-Waste" cleanup mechanisms, which ensure that the increased cloud storage consumption is offset by the immediate removal of local server resources. Ultimately, the successful triangulation of user trust with empirical performance metrics confirms that StegoLock effectively bridges the gap between high-level cryptographic complexity and operational simplicity, fulfilling the mandate of a secure, user-centric data storage solution.

CHAPTER V

SUMMARY, CONCLUSIONS, AND RECOMMENDATIONS

This chapter summarizes key findings, the overall conclusions drawn from the results, and recommendations for future work. It reflects on the effectiveness of the proposed reconstruction-dependent security architecture and its impact on protecting digital documents in cloud storage environments. Lastly, it offers insights for improving the system and guiding further research in multi-layered defense-in-depth methodologies utilizing cryptography and steganography.

5.1 SUMMARY

This study aimed to develop StegoLock, a cloud-based web application that protects sensitive digital documents from unauthorized access and forensic detection through a reconstruction-dependent security model. The application utilizes AES-256-GCM encryption, fluid segmentation, and LSB steganographic embedding across heterogeneous cover files, which subtly conceals encrypted data fragments to ensure that the document cannot be reconstructed or decrypted without all distributed parts.

The study evaluated the effectiveness of the StegoLock platform against unauthorized data recovery using a controlled testing environment and a triangulated analysis of system metrics and user evaluations from 30 participants. Three key areas were analyzed: security efficacy (SHA-256 integrity validation), resource utilization (stego-expansion ratio), and computational performance (locking and unlocking latencies). Results demonstrated a clear relationship between payload size and system overhead. For documents under 1MB, the system achieved a mean locking duration of 5.48 seconds with a balanced 6.0x expansion ratio. In contrast, documents in the 3MB–5MB range required up to 31 fragments, increasing the expansion ratio to 21.0x and the locking duration to 19.74 seconds to maintain low-density concealment.

Metric analysis revealed important patterns, where the expansion ratio showed an aggressive increase as payload density neared the capacity limits of available cover

media. While the locking process scaled linearly with file size due to the computational 'search-and-fit' logic of embedding, the unlocking process remained remarkably efficient, averaging 5.07 seconds for the largest file tier. This efficiency correlates with the high user responsiveness rating (GWM 4.61), confirming that the asynchronous architecture effectively mitigates backend complexity. The 100% integrity success rate across all test samples (n=27) validated the reliability of the reassembly pipeline, suggesting that the system maintains bit-perfect reconstruction even under high fragmentation levels.

In conclusion, StegoLock provides a promising solution for organizations seeking to safeguard sensitive digital documents in insecure cloud environments. However, further enhancements are necessary to optimize the expansion ratios for large-scale payloads and address operational dependencies, such as shell-based execution, for broader deployment and enterprise integration.

5.2 CONCLUSIONS

This study aimed to develop StegoLock, which shows that a reconstruction-dependent security architecture integrating cryptography and steganography can effectively safeguard sensitive documents in cloud environments. Through rigorous testing and triangulated analysis, the research demonstrated that segmenting encrypted payloads and concealing them across distributed cloud storage successfully obscures data from unauthorized detection while maintaining bit-perfect integrity. Below are the key conclusions drawn from the study, each corresponding to the project's specific objectives:

1. The study successfully implemented a multi-tiered cryptographic framework using AES-256-GCM encryption and a dual KDF-based key management process (PBKDF2-HMAC-SHA256 and HKDF). Testing confirmed that this implementation effectively protects document confidentiality and ensures that encryption keys are securely derived on demand rather than permanently stored,

reducing the risk of credential exposure. SHA-256 validation results, as discussed in Section 4.5, demonstrated a 100% integrity success rate across all test samples, confirming the mathematical accuracy and robustness of the core cryptographic engine.

2. The development of a segmentation algorithm and LSB-based steganographic embedding successfully achieved Objective 2 by distributing encrypted data across heterogeneous cover files (PNG, WAV, and TXT). Technical findings in Table 25 revealed that the system aggressively scales segmentation—producing up to 31 fragments for a 5MB document—to maintain the low-density concealment required to evade forensic detection. The study concluded that while this approach introduced a storage expansion ratio ranging from 6.0x to 21.0x, the increased cloud footprint is a necessary architectural trade-off to ensure high-level security resistance through physical data separation.
3. The study successfully developed the StegoLock web application, integrating complex backend security pipelines into a responsive, user-friendly platform. The integration of asynchronous background queuing allowed the system to abstract the 12.9-second average locking pipeline into a "One-Click" interface, successfully restricting synchronous UI-blocking phases to under 50ms. The successful cloud deployment and role-based access control (RBAC) validated the platform's ability to provide a secure and collaborative environment for sensitive document management without overwhelming the end-user with technical complexity.
4. The overall effectiveness of StegoLock was evaluated based on ISO/IEC 25010 quality characteristics, achieving a General Weighted Mean (GWM) of 4.53, interpreted as "Strongly Agree." Results summarized in Table 10 confirmed that Performance Efficiency (GWM 4.61), Security (GWM 4.55), and Functional Suitability (GWM 4.55) were the most positively perceived attributes. These findings validated that the reconstruction-dependent model effectively bridges the gap between high-level cryptographic complexity and operational simplicity,

meeting the required standards for a professional-grade secure document management system.

5.3 RECOMMENDATIONS

The findings of this study highlight both the efficacy and technical constraints of StegoLock in its current implementation. While the platform demonstrates a high level of security through its reconstruction-dependent architecture, transition toward greater functional maturity requires addressing scalability, automation, and long-term storage mobility challenges. The following recommendations aim to refine the system's backend pipeline, enhance its proactive security posture, and future-proof its storage model against evolving forensic methodologies. These proposals are structured to align with the operational needs of document owners and the rigorous technical demands of distributed cloud storage.

Firstly, a significant operational constraint identified during testing is the linear scaling of the encryption and embedding pipeline, which can lead to increased latency as document sizes grow. To support greater functional maturity, future system iterations should prioritize Intelligent Payload Distribution. This would involve implementing Dynamic Segmentation algorithms that automatically adjust fragment sizes based on the bit-depth and dimensional capacity of assigned cover files. Such optimizations would allow StegoLock to handle larger datasets with minimal manual configuration, ensuring that the system remains responsive even under high-load conditions.

Second, while the current architecture provides robust static protection, the security of the system could be further hardened against long-term data analysis. Integrating Automated Refresh Cycles is recommended, ensuring that the platform re-encrypts and re-conceals fragments upon each retrieval request, effectively creating a "moving target" for adversaries. Furthermore, upgrading to the Argon2 key derivation standard and implementing Linked Security Fragments would ensure that document segments are cryptographically interdependent. This creates a dynamic security posture

where the compromise of an isolated fragment provides zero advantage to an attacker, as all segments must be correctly sequenced and validated for successful reconstruction.

Also, future research should focus on enhancing the concealment properties and geographic mobility of the stored data. Investigating Dynamic Cloud Relocation, the periodic migration of file fragments between diverse cloud storage providers, would prevent attackers from accumulating a complete dataset over time. Additionally, the exploration of AI-driven Cover Generation could produce high-entropy, natural-looking concealment media that are virtually indistinguishable from ordinary user data. These advancements support long-term data protection by ensuring that stored fragments remain hidden from evolving detection methodologies and sophisticated forensic analysis tools.

Finally, seamless integration into professional workflows is crucial for practical adoption beyond a research environment. To maintain high usability standards, the development of Native Mobile Applications equipped with biometric authentication features is recommended. By exposing the core architecture as a Secure API, organizations could integrate StegoLock's protection layers directly into their existing document management systems. This transition from a research prototype to a practical enterprise-grade solution would provide tangible benefits to organizations handling sensitive digital assets across diverse device types.

These future directions build logically upon the proven efficacy of the core protection technology while addressing the practical limitations identified during the evaluation phase. The most immediate priorities should be computational optimization and the automation of fragment distribution, which would provide the most significant impact on system performance for early adopters. Parallel development of proactive security features and expanded cloud mobility will ensure that the platform remains resilient against emerging cyber threats. Importantly, researchers must recognize that increasing the density of embedded data does not yield proportional security gains; after

a certain threshold, the integrity of the carrier file begins to degrade without adding significant protection. Therefore, future development should prioritize the creation of intelligent systems that dynamically balance security, performance, and concealment quality based on the specific content of the document and its intended use.

REFERENCES

- [1] F. Park, "More than just words on a page: The different ways we record history," Transkribus.org, Sep. 10, 2025. <https://blog.transkribus.org/en/record-history-writing-oral-traditions-physical-objects-multimedia>
- [2] I. Spar, "The Origins of Writing - The Metropolitan Museum of Art," Metmuseum.org, Oct. 2004. <https://www.metmuseum.org/essays/the-origins-of-writing>
- [3] Israt, "PAPER - Technologies of Text," Technologies of Text, Nov. 20, 2023. <https://techoftextf23.kristendoylehighland.com/paper/>
- [4] "2025 Thales Data Threat Report," Thales Cloud Security Products, 2025. <https://cpl.thalesgroup.com/ppc/data-threat-report>
- [5] IBM, "Cost of a data breach report 2025," IBM, 2025. <https://www.ibm.com/reports/data-breach>
- [6] B. Osborne, "2026 Data Breaches: Cybersecurity Incidents - PKWARE®," PKWARE®, Feb. 09, 2026. <https://www.pkware.com/blog/2026-data-breaches>
- [7] I. A. Justina, O. Abiodun Ezekiel, and O. M. Olutomisin, "A Secured Cloud-Based Electronic Document Management System," International Journal of Innovative Research and Development, vol. 11, no. 12, Dec. 2022, doi: <https://doi.org/10.24940/ijird/2022/v11/i12/dec22010>.
- [8] W. Qi, M. Sun, and S. R. A. Hosseini, "Facilitating big-data management in modern business and organizations using cloud computing: A comprehensive study," Journal of Management & Organization, vol. 29, no. 4, pp. 1–27, Apr. 2022, doi: <https://doi.org/10.1017/jmo.2022.17>.
- [9] S. S. Z. Abidin and M. H. Husin, "Improving accessibility and security on document management system: A Malaysian case study," Applied Computing and Informatics, vol. 16, no. 1-2, pp. 137–154, 2020, doi: <https://doi.org/10.1016/j.aci.2018.04.002>.
- [10] A. F. Azmir and L. Wijayanti, "Cloud Computing Opportunities and Challenges in Electronic Document Management," Record and Library Journal, vol. 8, no. 2, pp. 248–258, Dec. 2022, doi: <https://doi.org/10.20473/rlj.v8-i2.2022.248-258>.
- [11] "Money Matters: Google warns Gmail users to change passwords after data breach." Sep. 2025, <https://www.youtube.com/watch?v=KRHnjXnkWJ4> (accessed Jan. 07, 2026).

- [12] R. Koch, "Google suffers data breach, puts out Gmail warning | Proton," Proton, Aug. 26, 2025. <https://proton.me/blog/google-data-breach-gmail-warning>
- [13] "40% of Google Drive files contain sensitive information," Securitymagazine.com, Dec. 06, 2023. <https://www.securitymagazine.com/articles/100205-40-of-google-drive-files-contain-sensitive-information>.
- [14] S. K. Sharma and B. Khuntia, "Integrated security for data transfer and access control using authentication and cryptography technique for Internet of things," International Journal of Knowledge-based and Intelligent Engineering Systems, vol. 24, no. 4, pp. 303–309, Jan. 2021, doi: <https://doi.org/10.3233/kes-190116>.
- [15] Z. Liu, S. Wang, and Y. Liu, "Blockchain-based integrity auditing for shared data in cloud storage with file prediction," Computer Networks, vol. 236, p. 110040, Sep. 2023, doi: <https://doi.org/10.1016/j.comnet.2023.110040>.
- [16] J. Li, J. Wu, G. Jiang, and T. Srikanthan, "Blockchain-based public auditing for big data in cloud storage," Information Processing & Management, vol. 57, no. 6, p. 102382, Nov. 2020, doi: <https://doi.org/10.1016/j.ipm.2020.102382>.
- [17] A. Meryem and B. E. Ouahidi, "Hybrid intrusion detection system using machine learning," Network Security, vol. 2020, no. 5, pp. 8–19, May 2020, doi: [https://doi.org/10.1016/s1353-4858\(20\)30056-8](https://doi.org/10.1016/s1353-4858(20)30056-8).
- [18] V. Hnamte and J. Hussain, "DCNNBiLSTM: An Efficient Hybrid Deep Learning-Based Intrusion Detection System," Telematics and Informatics Reports, vol. 10, p. 100053, Mar. 2023, doi: <https://doi.org/10.1016/j.teler.2023.100053>.
- [19] K. S. Patil, I. Mandal, and C. Rangaswamy, "Hybrid and Adaptive Cryptographic-based secure authentication approach in IoT based applications using hybrid encryption," Pervasive and Mobile Computing, vol. 82, p. 101552, Jun. 2022, doi: <https://doi.org/10.1016/j.pmcj.2022.101552>.
- [20] V. Singhal, D. Singh, and S. Gupta, "Crypto STEGO Techniques to Secure Data Storage Using DES, DCT, Blowfish and LSB Encryption Algorithms," JOURNAL OF ALGEBRAIC STATISTICS, vol. 13, no. 3, pp. 1162–1171, 2022, <https://iftmuniversity.ac.in/iftmuniversity/profile/download/paper/2522.pdf>
- [21] Z. Hu, Y. Wang, and M. He, Advances in Intelligent Systems, Computer Science and Digital Economics IV, Lecture Notes on Data Engineering and Communications Technologies. Cham, Switzerland: Springer Nature Switzerland, 2023, <https://link.springer.com/book/10.1007/978-3-031-24475-9>

- [22] Z. Shabeer et al., “Dual-Attention ResUNet-GAN for Secure Image Steganography: Optimizing the Trade-off Between Imperceptibility and Payload Capacity,” *International Journal of Advanced Computer Science and Applications*, vol. 16, no. 11, 2025, doi: <https://doi.org/10.14569/ijacsa.2025.0161109>.
- [23] Asmaa Mahfoud, I. Omar, and A. Essilini, “Comprehensive Review of Cryptography and Steganography Algorithms,” *Journal of Information Systems Engineering & Management*, vol. Vol. 10, no. (29s), pp. 2468–4376, Jan. 2025, https://www.researchgate.net/publication/391270767_Comprehensive_Review_of_Cryptography_and_Steganography_Algorithms
- [24] D. Das and M. Dutta, “Security Enhancement on Application Oriented Steganographic Schemes with Crypto-Encryption: A Technical Review,” *TTIC*, vol. 4, pp. 16–27, 2020, https://tiutic.org/pdf/volume/Vol_4/Vol4_Article_3.pdf
- [25] S. Mataraarachchi and N. Wedasinghe, “Data Security System for Chat Applications Using Cryptography, Steganography and Image Processing,” *Kdu.ac.lk*, 2018, doi: <https://ir.kdu.ac.lk/handle/345/2510>.
- [26] Mohd and A. Aziz, “Web-Application for securing message using Steganography with LSB Algorithm and Hybrid Encryption / Muhammad Khairul Amin Mohd Nai and Siti Rahayu Abdul Aziz - UiTM Institutional Repository,” *Uitm.edu.my*, 2021, doi: <https://ir.uitm.edu.my/id/eprint/49428/1/49428.pdf>.
- [27] S. Gomathi and C. Radhika, “A secure messaging application using steganography and AES encryption a dual-layer secure messaging system,” *The Scientific Temper*, vol. 16, no. 02, pp. 3803–3811, Mar. 2025, doi: <https://doi.org/10.58414/scientifictemper.2025.16.2.12>.
- [28] S. Jordan, S. S. Zabukovšek, and I. Š. Klančnik, “Document Management System – A Way to Digital Transformation,” *Naše gospodarstvo/Our economy*, vol. 68, no. 2, pp. 43–54, Jun. 2022, doi: <https://doi.org/10.2478/ngoe-2022-0010>.
- [29] D. Jain, “Document Management System,” *Definitions*, 2020. <https://api.semanticscholar.org/CorpusID:140232171>
- [30] A. Orr, “Infostealer malware database exposes millions of iCloud and email passwords,” *AppleInsider*, Jan. 26, 2026. <https://appleinsider.com/articles/26/01/26/infostealer-malware-database-exposes-millions-of-icloud-and-email-passwords>

- [31] J. Fowler, “149M Logins and Passwords Exposed Online Including Financial Accounts, Instagram, Facebook, Roblox, Dating Sites, and More.” ExpressVPN, Jan. 23, 2026. <https://www.expressvpn.com/blog/149m-info-stealer-data-exposed/>
- [32] Dropbox, “Exhibit 99.1 – May 2024 Form 8-K: Security Incident Involving Dropbox Sign,” U.S. Securities and Exchange Commission, May 2024. <https://www.sec.gov/Archives/edgar/data/1467623/000146762324000024/may2024exhibit991.htm>
- [33] R. Lakshmanan, “Dropbox discloses breach of digital signature service,” The Hacker News, May 2024. <https://thehackernews.com/2024/05/dropbox-discloses-breach-of-digital.html>
- [34] R. Lakshmanan, “Microsoft OneDrive File Picker flaw exposes user files to third-party apps,” The Hacker News, May 2025. <https://thehackernews.com/2025/05/microsoft-onedrive-file-picker-flaw.html>
- [35] K. Tran, “OneDrive Security Risks: Microsoft Grants Full Access to Third-Party Apps,” Obsidian Security, Jun. 19, 2025. <https://www.obsidiansecurity.com/blog/onedrive-security-risks>
- [36] “Major AWS S3 Bucket Breach Exposes Data in Cybersecurity Attack,” NHI Management Group, Dec, 2025. <https://nhimg.org/community/nhi-breaches/major-aws-s3-bucket-breach-exposes-data-in-cybersecurity-attack/>
- [37] M. Callahan, “Major API Security Breaches and API Attacks from 2024,” Salt Security, October 15, 2024. <https://salt.security/blog/its-2024-and-the-api-breaches-keep-coming>
- [38] Z. Whittaker, “New zero-day bug in Microsoft SharePoint under widespread attack,” TechCrunch, Jul. 21, 2025. <https://techcrunch.com/2025/07/21/new-zero-day-bug-in-microsoft-sharepoint-under-widespread-attack/>
- [39] V. Krishnakumar, “The Importance of Data Encryption in Cloud Environments,” CloudOptimo, Jun. 19, 2025. <https://www.cloudoptimo.com/blog/the-importance-of-data-encryption-in-cloud-environments/>
- [40] Inam ul Haq, J. Wang, Y. Zhu, and S. Maqbool, “An efficient hash-based authenticated key agreement scheme for multi-server architecture resilient to key compromise impersonation,” *Digital Communications and Networks*, vol. 7, no. 1, pp. 140–150, Feb. 2021, doi: <https://doi.org/10.1016/j.dcan.2020.05.001>.

- [41] B. V. Nair, S. VismayaV, S. S. Muni, and A. Durdu, "Deep learning and chaos: A combined approach to image encryption and decryption," arXiv:2406.16792, June 2024, <https://api.semanticscholar.org/CorpusID:270703289>
- [42] D. Bhavsagar, "Threats and Vulnerabilities in Cloud Computing: A Comprehensive Security Analysis," *International Journal on Science and Technology*, vol. 16, no. 3, Aug. 2025, doi: <https://doi.org/10.71097/ijst.v16.i3.7823>.
- [43] C. Sharma and C. Sharma, "Cloud Computing Security: Threats and Mitigation Strategies," pp. 1–6, Sep. 2024, doi: <https://doi.org/10.1109/sparc61891.2024.10829335>.
- [44] A. T. Olutimehin, S. A. Joseph, A. J. Ajayi, O. C. Metibemu, A. Y. Balogun, and O. O. Olaniyi, "Future-proofing data: Assessing the feasibility of post-quantum cryptographic algorithms to mitigate 'harvest now, decrypt later' attacks," *Archives of Current Research International*, vol. 25, no. 3, pp. 60–80, Feb. 17 2025, doi:10.9734/acri/2025/v25i31098.
- [45] B. Benčina, B. Dowling, V. Maram, and K. Xagawa, "Post-quantum cryptographic analysis of SSH," *Cryptology ePrint Archive*, Paper 2025/684, May 12, 2025, <https://eprint.iacr.org/2025/684>
- [46] Eka Yulianti, Veithzal Rivai Zainal, and A. Hakim, "The Impact of Digital Archiving on Document Management Efficiency in the Directorate of Service and Zones Investment Planning, Ministry of Investment and Downstreaming /BKPM," *Jurnal Ekonomi Manajemen Sistem Informasi*, vol. 6, no. 6, pp. 4036–4043, Jul. 2025, doi: <https://doi.org/10.38035/jemsi.v6i6.5853>.
- [47] Md. Ashikuzzaman, "The Evolution and Impact of Digital Archives on Information Management," *Library & Information Science Education Network*, Apr. 15, 2025. <https://www.lisedunetwork.com/the-evolution-and-impact-of-digital-archives-on-information-management/>
- [48] P. Mell and T. Grance, "The NIST Definition of Cloud Computing Recommendations of the National Institute of Standards and Technology," National Institute of Standards and Technology, Sep. 2011. Available: <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-145.pdf>
- [49] P. Goutham, "Exploring Cloud Computing Services and Applications," *Peer Reviewed, Refereed, Scholarly Indexed, Open Access Journal*) Impact Factor: 8, vol. 206, no. 6, 2025, doi: <https://doi.org/10.15680/IJMRSET.2025.0806037>.
- [50] Ojha, A., & Sharma, S. (2024). An approach for cloud data security based on file splitting. *International Journal of Innovative Research in Computer and*

Communication Engineering, 12(Special Issue 1). http://ijircce.com/special-issues/pdf/2024/27_Approach.pdf

- [51] A. Mittal, "Cloud computing," Proceedings of the IRE, Jun. 2020, Available: https://www.researchgate.net/publication/392400831_Cloud_computing
- [52] S. Holloway, "The Impact of Cloud-Based Information Systems on Collaboration and Productivity in Remote Teams," Dec. 2024, doi: <https://doi.org/10.20944/preprints202412.0958.v1>.
- [53] A. Singh and K. Chatterjee, "Cloud security issues and challenges: A survey," Journal of Network and Computer Applications, vol. 79, pp. 88–115, Feb. 2017, doi: <https://doi.org/10.1016/j.jnca.2016.11.027>.
- [54] Sultan Majid Alshamsi. "Security Vulnerabilities in Cloud Storage: A Comparative Study of Google Drive, Dropbox, and OneDrive." RIT Digital Institutional Repository, 2025, repository.rit.edu/theses/12459/
- [55] Basan, Maine. "How Secure Is Cloud Storage? Features, Risks, & Protection." ESecurity Planet, 18 Jan. 2024, www.esecurityplanet.com/cloud/how-secure-is-cloud-storage/
- [56] Major. "Major Cloud Services such as Google Drive and Dropbox at Risk from Man-In-The-Cloud Attacks - Tech News | HBH." HBH, 2026, hbh.sh/news/639/major-cloud-services-such-as-google-drive-and-dropbox-at-risk-from-man-in-the-cloud-attacks/ Accessed 18 Feb. 2026.
- [57] IJCT. "An Analytical Study on User Trust and Data Privacy in Cloud Storage Services | IJCT Volume 12 - Issue 5 | IJCT-V12I5P64." IJCT Journal, 18 Oct. 2025, ijctjournal.org/analytical-study-user-trust-data-privacy-cloud-storage-services-ijct/ Accessed 18 Feb. 2026.
- [58] R. Mohammad Zahir, "Cloud Storage Security: Risks and Solutions," Nov. 2025, doi: <https://doi.org/10.20944/preprints202511.1778.v1>.
- [59] Arun Prakasam. "Advanced Encryption Standard (AES) Algorithm - TeQ Research - Research Tutorial for Computer Science and Technologies." TeQ Research - Research Tutorial for Computer Science and Technologies - Research Tutorial for Computer Science and Technologies, June 2025, teqresearch.com/advanced-encryption-standard-aes/
- [60] Hidayat, Taufik, and Rahutomo Mahardiko. "A Systematic Literature Review Method on AES Algorithm for Data Sharing Encryption on Cloud Computing."

International Journal of Artificial Intelligence Research, vol. 4, no. 1, 9 Apr. 2020, <https://doi.org/10.29099/ijair.v4i1.154>.

- [61] Ganesh, Rahoul, et al. "A Panoramic Survey of the Advanced Encryption Standard: From Architecture to Security Analysis, Key Management, Real-World Applications, and Post-Quantum Challenges." *International Journal of Information Security*, vol. 24, no. 5, 25 Sept. 2025, <https://doi.org/10.1007/s10207-025-01116-x>.
- [62] P. Gaur, "AES Image Encryption (Advanced Encryption Standard)," *International Journal for Research in Applied Science and Engineering Technology*, vol. 9, no. 12, pp. 1357–1363, Dec. 2021, doi: <https://doi.org/10.22214/ijraset.2021.39542>.
- [63] S. Rana, F. K. Parast, B. Kelly, Y. Wang, and K. B. Kent, "A comprehensive survey of cryptography key management systems," *Journal of Information Security and Applications*, vol. 78, p. 103607, Nov. 2023, doi: <https://doi.org/10.1016/j.jisa.2023.103607>.
- [64] Abdullah, Ali. "Key Derivation: A Dynamic PBKDF2 Model for Modern Cryptographic Systems." *Cryptography*, vol. 9, no. 2, 5 June 2025, pp. 39–39, <https://doi.org/10.3390/cryptography9020039>.
- [65] Hoang, Dinh Linh, et al. "New Proofs for Pseudorandomness of HMAC-Based Key Derivation Functions (RFC 5869)." *Journal of Information Security and Applications*, vol. 93, Sept. 2025, p. 104179, <https://doi.org/10.1016/j.jisa.2025.104179>.
- [66] Krawczyk, Hugo. "Cryptographic Extraction and Key Derivation: The HKDF Scheme." *Cryptology EPrint Archive*, 11 May 2010, eprint.iacr.org/2010/264?u.
- [67] Visconti, Andrea, et al. "Examining PBKDF2 Security Margin—Case Study of LUKS." *Journal of Information Security and Applications*, vol. 46, June 2019, pp. 296–306, <https://doi.org/10.1016/j.jisa.2019.03.016>.
- [68] Legit Security. "What Is Encryption Key Management? Importance and Best Practices." *Legitsecurity.com*, Legit Security, 3 Jan. 2025, www.legitsecurity.com/aspm-knowledge-base/encryption-key-management-best-practices/. Accessed 23 Feb. 2026.
- [69] Aruna, Ms. T N, et al. "Text, Image and Audio Steganography." *International Journal for Research in Applied Science and Engineering Technology*, vol. 11, no. 4, 30 Apr. 2023, pp. 4435–4439, www.ijraset.com/best-journal/text-image-and-audio-steganography, <https://doi.org/10.22214/ijraset.2023.51091>.

- [70] Rahman, Shahid, et al. "A Novel and Efficient Digital Image Steganography Technique Using Least Significant Bit Substitution." *Scientific Reports*, vol. 15, no. 1, 2 Jan. 2025, www.nature.com/articles/s41598-024-83147-3, <https://doi.org/10.1038/s41598-024-83147-3>.
- [71] Sanjalawe, Yousef, et al. "A Deep Learning-Driven Multi-Layered Steganographic Approach for Enhanced Data Security." *Scientific Reports*, vol. 15, no. 1, 8 Feb. 2025, www.nature.com/articles/s41598-025-89189-5, <https://doi.org/10.1038/s41598-025-89189-5>.
- [72] A. M. Deshmukh and R. Chalmeta, "Validation of system usability scale as a usability metric to evaluate voice user interfaces," *PeerJ Computer Science*, vol. 10, p. e1918, Feb. 2024. <https://doi.org/10.7717/peerj-cs.1918>
- [73] Kaviani, M., & Zhandry, M. (2024). Secret sharing with certified deletion. *arXiv*. <https://arxiv.org/html/2405.08117v1>
- [74] Sudha, V., & Ramesh, D. (2022). Development of fragmentation to create big data security. *International Journal of Intelligent Systems and Applications in Engineering (IJISAE)*, 10(3), 298–306. <https://ijisae.org/index.php/IJISAE/article/download/3947/2593/8933>
- [75] Ting, Tang, and Ming Li. "Enhanced Secure Storage and Data Privacy Management System for Big Data Based on Multilayer Model." *Scientific Reports*, vol. 15, no. 1, 2 Sept. 2025, pmc.ncbi.nlm.nih.gov/articles/PMC12405518/, <https://doi.org/10.1038/s41598-025-16624-y>.
- [76] Hu, V. C. (2023). Overview and considerations of access control based on attribute encryption (NIST IR 8450-upd1). National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.IR.8450-upd1>
- [77] Abusukhon, A., Anwar, M. N., Mohammad, B., & Alghannam, B. (2023). Attribute-based encryption schemes for next generation wireless IoT networks: A comprehensive survey. *PubMed Central / MDPI Sensors*, 23(14), 6338. <https://pmc.ncbi.nlm.nih.gov/articles/PMC10346245/>
- [78] National Institute of Standards and Technology. (2020). Roadmap toward criteria for threshold schemes for cryptographic primitives (NIST IR 8214A). U.S. Department of Commerce. <https://csrc.nist.gov/projects/threshold-cryptography>
- [79] Yu, J., Wang, H., & Zhang, Y. (2025). Adaptively secure, splittable, and robust threshold unsigncryption. *Cybersecurity*, 8, Article 32. <https://cybersecurity.springeropen.com/articles/10.1186/s42400-024-00344-3>

- [80] Adee, R., & Mouratidis, H. (2022). A dynamic four-step data security model for data in cloud computing based on cryptography and steganography. *Sensors*, 22(3), 1109. <https://doi.org/10.3390/s22031109>
- [81] Benbouali, A., & Debbabi, M. (2020). Distributed data hiding in multi-cloud storage environment. *Journal of Cloud Computing*, 9, Article 60. <https://doi.org/10.1186/s13677-020-00208-4>
- [82] Haverkamp, I., & Sarmah, D. K. (2024). Evaluating the merits and constraints of cryptography-steganography fusion: a systematic analysis. *International Journal of Information Security*, 23(4), 2607–2635. <https://doi.org/10.1007/s10207-024-00853-9>
- [83] Y. Tong, S. Li, Z. Wang, R. Zhang, and R. Xue, “Adaptively secure, splittable, and robust threshold unsigncryption,” *Cybersecurity*, vol. 8, no. 1, May 2025, doi: <https://doi.org/10.1186/s42400-024-00344-3>.
- [84] Godakanda, M. L. (2023). Optimising a defence-aware threat modelling diagram incorporating a defence-in-depth approach for the Internet-of-Things. [As cited in Wiley Defense in Depth compilation]. <https://onlinelibrary.wiley.com/doi/abs/10.1002/9781394340750.ch3>
- [85] Wai, E., & Lee, C. K. M. (2024). Depth in defense: A multi-layered approach to cybersecurity for SCADA systems in Industry 4.0. In *Science and Technology: Recent Updates and Future Prospects* (Vol. 2, pp. 124–144). B P International. <https://www.researchgate.net/publication/380523433>
- [86] Agboola, T. O., Adegede, J., & Jacob, J. G. (2024). Balancing usability and security in secure system design: A comprehensive study on principles, implementation, and impact on usability. *International Journal of Computing Sciences Research*, 8, 2995–3009. <https://doi.org/10.25147/ijcsr.2017.001.1.199>
- [87] Alqassem, I., Ghani, I., Baharom, F., & Elias, N. F. (2023). Usable security: A systematic literature review. *MDPI Information*, 14(12), 641. <https://doi.org/10.3390/info14120641>
- [88] Bipin Gajbhiye, Shalu Jain, and Om Goel, “Defense in Depth Strategies for Zero Trust Security Models,” *International Journal for Research Publication and Seminar*, vol. 15, no. 3, pp. 293–305, Aug. 2024, doi: <https://doi.org/10.36676/jrps.v15.i3.1497>.
- [89] Ma, X., Hur, J., Gu, M., & Du, N. (2025). Design and optimization of hybrid end-to-end encryption architecture for a secure web application system. *Journal of*

- Web Engineering, 24(07), 1155–1180. <https://doi.org/10.13052/jwe1540-9589.2474>
- [90] M. Rana, Q. Mamun, and R. Islam, “Enhancing IoT Security: An Innovative Key Management System for Lightweight Block Ciphers,” *Sensors*, vol. 23, no. 18, p. 7678, Jan. 2023, doi: <https://doi.org/10.3390/s23187678>.
- [91] Krähenbühl, C., & Perrig, A. (2023). Key management. In V. Mulder, A. Mermoud, V. Lenders, & B. Tellenbach (Eds.), *Trends in Data Protection and Encryption Technologies*. Springer. https://doi.org/10.1007/978-3-031-33386-6_4
- [92] Nabawy, Rabab Mohamed, et al. “Mixed Fragmentation Technique for Securing Structured Data Using Multi-Cloud Environment (MFT-SSD).” *Ad Hoc Networks*, vol. 164, 22 Aug. 2024, p. 103625, www.sciencedirect.com/science/article/pii/S1570870524002361, <https://doi.org/10.1016/j.adhoc.2024.103625>.
- [93] Waseem, S., Krisha, V. S., Urvi Vidhya Sivani, T., & Shamija Sherryl, R. M. R. (2025). File fragmentation and encryption system for cloud based data storage and retrieval. *International Journal of Progressive Research in Engineering Management and Science (IJPREMS)*, 5(4), 3302–3306. <https://doi.org/10.58257/IJPREMS40682>
- [94] M. Čurík and K. Klucznik, "Practical use of secret sharing for enhancing privacy in clouds," *Electronics*, vol. 11, no. 17, p. 2758, 2022. <https://doi.org/10.3390/electronics11172758>
- [95] R. Adees and H. Mouratidis, "A dynamic four-step data security model for data in cloud computing based on cryptography and steganography," *Sensors*, vol. 22, no. 3, p. 1109, 2022. <https://doi.org/10.3390/s22031109>
- [96] M. H. Alkawaz, A. Mohammedali, and S. J. Steven, "Enhancing data security: Implementing steganography in web application," *Journal of Information Systems Engineering and Management*, vol. 10, no. 29s, 2025. <https://doi.org/10.52783/jisem.v10i29s.4596>
- [97] Reddy, K. N., Snehitha, M. S., Reddy, K. S. C., Reddy, K. D., Rajagopal, S. M., and Panda, N., “Secured Steganography: Leveraging Cryptography in a Cloud-Based Toolbox”, in *2024 7th International Conference on Contemporary Computing and Informatics (IC3I)*, 2024, vol. 7, Art. no. 27. doi:10.1109/IC3I61595.2024.10829039.
- [98] Dropbox Inc., "A message about our security," *Dropbox Blog*, 2022. <https://blog.dropbox.com/topics/company/a-message-about-our-security>

- [99] S. Fugkeaw, N. Chinvorarat, N. Charnwutiwong, K. Chaisuwan, and W. Pruktipinyopap, "A dynamic and efficient crypto-steganography system for securing multiple files in cloud," in Proc. 2023 15th Int. Conf. on Knowledge and Smart Technology (KST), IEEE, 2023. <https://doi.org/10.1109/KST57286.2023.10086908>
- [100] Google LLC, "Google Drive security and privacy overview," Google Workspace Admin Help, 2023. <https://support.google.com/a/answer/2490008>
- [101] E. A. W. Hachim, "Cloud data protection based on crypto-steganography approach," Journal La Multiapp, vol. 5, no. 4, pp. 324–331, 2024. <https://doi.org/10.37899/journallamultiapp.v5i4.1337>
- [102] Infosecurity Magazine, "Microsoft OneDrive flaw exposes users to data overreach risks," 2025. <https://www.infosecurity-magazine.com/news/microsoft-onedrive-flaw-exposes/>
- [103] Microsoft Corporation, "OneDrive security and compliance," Microsoft Learn, 2024. <https://learn.microsoft.com/en-us/onedrive/onedrive-security>
- [104] S. K. Patel and S. Chandran, "PDF steganography using hybrid crypto encryption technique," in IoT Based Control Networks and Intelligent Systems, P. P. Joby, V. E. Balas, and R. Palanisamy, Eds., Lecture Notes in Networks and Systems, vol. 528. Springer, 2023, pp. 425–435. https://doi.org/10.1007/978-981-19-5845-8_32
- [105] B. A. Salim, M. A. Aljabery, and H. A. Younis, "AES-based steganography using blockchain: A novel approach for secure text hiding in encrypted images," Informatica, vol. 48, no. 21, pp. 67–78, 2024. <https://doi.org/10.31449/inf.v48i21.6689>
- [106] A. Biryukov, D. Dinu, and D. Khovratovich, "Argon2: The memory-hard, GPU-resistant hash function for password hashing and proof-of-work applications," in Proc. IEEE European Symposium on Security and Privacy (EuroS&P), Saarbrücken, Germany, Mar. 2016, pp. 202–213. [Online]. Available: <https://doi.org/10.1109/EuroSP.2016.31>
- [107] M. Nair and D. Song, "Multi-factor key derivation function: Realizing multi-factor authentication in the key derivation process," in Proc. USENIX Security Symposium, Boston, MA, USA, Aug. 2023. [Online]. Available: <https://arxiv.org/abs/2208.05586>
- [108] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, "Role-based access control models," IEEE Computer, vol. 29, no. 2, pp. 38–47, Feb. 1996. [Online]. Available: <https://doi.org/10.1109/2.485845>

- [109] National Institute of Standards and Technology, "Recommendation for Password-Based Key Derivation: Part 1: Storage Applications," NIST Special Publication 800-132, Dec. 2010. [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-132.pdf>
- [110] National Institute of Standards and Technology, "Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC," NIST Special Publication 800-38D, Nov. 2007. [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-38d.pdf>
- [111] S. Elsheimy, E. E. Hemdan, and B. Elhadidi, "A blind data hiding technique with error correction abilities and a high embedding payload using LSB embedding and block coded modulation," *Journal of Applied Research and Technology*, vol. 11, no. 2, pp. 177-191, 2013. [Online]. Available: [https://doi.org/10.1016/S1665-6423\(13\)71549-5](https://doi.org/10.1016/S1665-6423(13)71549-5)
- [112] S. W. Mossebo Tcheunteu, L. Moyou Metcheke, and R. Ndoundam, "Distributed data hiding in a single cloud storage environment," *Journal of Cloud Computing: Advances, Systems and Applications*, vol. 10, art. no. 43, Aug. 2021. [Online]. Available: <https://doi.org/10.1186/s13677-021-00258-2>
- [113] T. Dierks and E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.2," RFC 5246, Aug. 2008. [Online]. Available: <https://tools.ietf.org/html/rfc5246>
- [114] Backblaze, "Backblaze B2 Cloud Storage: S3-Compatible Object Storage," Backblaze, Inc., 2024. [Online]. Available: <https://www.backblaze.com/cloud-storage>
- [115] S. K. Prabhakar, J. Rao, and P. Khatri, "Agile methodology: An overview and implementation strategies," *Journal of Software Engineering Research and Development*, vol. 5, art. no. 18, Dec. 2017. [Online]. Available: <https://doi.org/10.1186/s40411-017-0051-0>
- [116] N. Khan and S. Oates, "Agile and secure software development: A systematic literature review," *Journal of Systems and Software*, vol. 165, p. 110566, 2020. [Online]. Available: <https://doi.org/10.1016/j.jss.2020.110566>
- [117] W. Mazurczyk and K. Szczypiorski, "Steganography in cloud computing: Security implications and detection mechanisms," *IEEE Transactions on Information Forensics and Security*, vol. 12, no. 12, pp. 2844-2857, Dec. 2017. [Online]. Available: <https://doi.org/10.1109/TIFS.2017.2718542>

- [118] R. K. Patel, S. Desai, and M. Joshi, "Secure steganography techniques: Maintaining carrier file functionality and imperceptibility," *Journal of Cybersecurity and Privacy*, vol. 2, no. 1, pp. 1-18, 2022. [Online]. Available: <https://doi.org/10.3390/jcp2010001>
- [119] D. LeBlanc and S. T. Walsh, "A review of cybersecurity risk assessment methods for critical infrastructure," *Computers & Security*, vol. 96, p. 101873, 2020. [Online]. Available: <https://doi.org/10.1016/j.cose.2020.101873>
- [120] F. Shaikh and M. Alharthi, "Continuous monitoring and improvement of security controls in cloud-based systems," *IEEE Cloud Computing*, vol. 6, no. 3, pp. 54-62, May 2019. [Online]. Available: <https://doi.org/10.1109/MCC.2019.2910940>
- [121] National Security Agency and Cybersecurity and Infrastructure Security Agency, "Secure data in the cloud," *Cybersecurity Information Sheet*, U.S. Dept. of Defense, Mar. 2024. <https://media.defense.gov/2024/Mar/07/2003407862/-1/-1/0/CSI-CLOUDTOP10-SECURE-DATA.PDF>
- [122] Cloud Security Alliance, "Top threats to cloud computing 2024," *Cloud Security Alliance*, Aug. 2024. <https://cloudsecurityalliance.org/artifacts/top-threats-to-cloud-computing-2024>
- [123] National Institute of Standards and Technology, "Advanced Encryption Standard (AES)," *Federal Information Processing Standards Publication 197*, U.S. Dept. of Commerce, updated May 2023. <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.197-upd1.pdf>
- [124] Cybersecurity and Infrastructure Security Agency (CISA), "Transition to Advanced Encryption Standard (AES)," *FPIC Security Subcommittee White Paper*, May 2024. https://www.cisa.gov/sites/default/files/2024-05/23_0918_fpic_AES-Transition-WhitePaper_Final_508C_24_0513.pdf
- [125] M. S. Turan, E. Barker, W. Burr, and L. Chen, "Recommendation for password-based key derivation — Part 1: Storage applications," *National Institute of Standards and Technology, NIST SP 800-132*, Dec. 2010. <https://doi.org/10.6028/NIST.SP.800-132>
- [126] H. Krawczyk and P. Eronen, "HMAC-based Extract-and-Expand Key Derivation Function (HKDF)," *Internet Engineering Task Force, RFC 5869*, May 2010. <https://www.rfc-editor.org/rfc/rfc5869>
- [127] H. A. Ahmed, A. Mahfoud, and O. I. Al-Sanjary, "Comprehensive review of cryptography and steganography algorithms," *Journal of Information Systems*

Engineering and Management, vol. 10, no. 29s, 2025.
<https://doi.org/10.52783/jisem.v10i29s.4471>

- [128] A. Jan, S. A. Parah, M. Hussan, and B. A. Malik, "Double layer security using crypto-stego techniques: a comprehensive review," *Health and Technology*, vol. 12, pp. 9–31, 2022. <https://doi.org/10.1007/s12553-021-00602-1>
- [129] M. A. Nasr, W. El-Shafai, E. M. El-Rabaie, A. S. El-Fishawy, H. M. El-Hoseny, F. E. Abd El-Samie, and N. Abdel-Salam, "A robust audio steganography technique based on image encryption using different chaotic maps," *Scientific Reports*, vol. 14, Art. no. 22054, Sep. 2024. <https://doi.org/10.1038/s41598-024-70940-3>
- [130] M. D. Saleem, S. U. Khan, A. N. Zomaya, and I. Gupta, "DROPS: Division and replication of data in cloud for optimal performance and security," *IEEE Transactions on Cloud Computing*, vol. 6, no. 2, pp. 344–355, Apr.–Jun. 2018. <https://ieeexplore.ieee.org/document/7031887>
- [131] L. Susi, and P. Elfindah, "Evaluating the Effectiveness of Mobile JKN Application in Indonesia: A User-Centric Approach Using the ISO 25010 Quality Model," *Journal of Logistics, Informatics and Service Science*, vol. 11 (2024), no. 10, pp. 485-500, doi: <https://doi.org/10.33168/JLISS.2024.1028>
- [132] ISO25000, "ISO/IEC 25010," iso25000.com, 2022. <https://iso25000.com/index.php/en/iso-25000-standards/iso-25010>
- [133] E. Oye and L. Emilson, "Impact of Cloud Solutions on Remote Work Efficiency," Nov. 01, 2024. https://www.researchgate.net/publication/391212301_Impact_of_Cloud_Solutions_on_Remote_Work_Efficiency
- [134] W. Witanti, "The Role of Cloud Computing Technology in Increasing Work Flexibility in Hybrid Era", *MSJ*, vol. 3, no. 3, pp. 276–285, Aug. 2025. <https://jurnalhafasy.com/index.php/msj/article/view/443>
- [135] K. Woźniak, M. R. Ogiela, and L. Ogiela, "A Two-Phase Embedding Approach for Secure Distributed Steganography," *Sensors*, vol. 25, no. 5, p. 1448, Feb. 2025, doi: <https://doi.org/10.3390/s25051448>.